claude-windows / 0c4fc269-564f-4fb2-9c1e-9389160b6be0

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\0c4fc269-564f-4fb2-9c1e-9389160b6be0.jsonl`
- SHA-256: `55feb3cd4db4b5ff70bed0128a22082db998412d2339a970314866e914814af4`
- Source modified: `2026-07-07T17:39:28+00:00`
- Imported at: `2026-07-08T15:59:10+00:00`
- project: `C--Users-avidu-Projects-badminton-highlight-indexer`
- session_id: `0c4fc269-564f-4fb2-9c1e-9389160b6be0`

Transcript

1. user (2026-07-07T16:46:09.686Z)

Live end-to-end smoke of the Cloud Run "crash-before-marker" fast-fail shipped in PR #515 (squash-merged to master as `ef41473`). Repo: khelsutra-guru/badminton-highlight-indexer ? real code under `C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer` (the primary working dir is an empty stub). `git pull --ff-only` on master first.

WHY THIS TASK EXISTS
PR #515 hardened `CloudRunCompute.run_infer` (backend/compute/cloud_run.py): while the done-marker stays the sole success signal, each bucket-poll now probes the Cloud Run Executions API so a worker that TERMINATED without ever writing a marker (crash / OOM / an old image argparse-aborting on `--skip-validation`) fails fast ? `RemoteExecutionFailed` ? `_dispatch_remote` returns **rc 5** ? instead of spinning the full `deployment.remote_poll_max_wait_sec` (~65 min) into a `PolicyTimeout` (rc 4). The classification LOGIC (`_classify_execution_state`) is unit-tested against real proto-plus `run_v2.Execution` objects, BUT the actual `GoogleCloudRunJobClient.execution_terminal_state` ? `client.get_execution(...)` fetch is `# pragma: no cover` ? it has NEVER run against live Cloud Run. This task closes that gap.

Adversarial review already caught one blocker here pre-merge: google-cloud-run is a proto-plus lib, so a `Timestamp` marshals to a Python `datetime` (`.second`, not `.seconds`) ? an early version keyed on `.seconds` and was a silent no-op. Fixed to key on `completion_time is None`. This smoke is to confirm the REAL get_execution wiring (auth, field names, region) behaves as the unit tests assume.

SETUP (real infra ? costs a few $ of L4 time; see the GPU-resident session handoff memory for exact image digests + rollback levers)
- Live infra: Cloud Run Job `rally-worker` and service `rally-control-plane`, region **asia-southeast1** (note: cloud_run.py's default CLOUD_RUN_REGION is `asia-south1`, but the deployed Job is asia-southeast1 ? use the deployed region, set via the control-plane's CLOUD_RUN_REGION env).
- To manufacture the exact "worker crashes before the marker" condition, pick ONE:
  (a) Pin the `rally-worker` Job to a PRE-#513 image digest (one whose argparse does NOT know `--skip-validation`), then dispatch a real /api/process job with the cloud-serving path so the control-plane sends `--skip-validation` ? the old worker argparse-aborts (exit 2) BEFORE writing the done-marker. This is the literal motivating scenario. Restore the current image digest afterward.
  (b) OR force any pre-marker crash on the current image (e.g. a config/override that makes cmd_infer exit before `_write_done_marker`), if that's cheaper to arrange.

EXPECTED RESULT (the pass criteria)
1. The dispatcher fails FAST ? within ~one poll interval of the crash (`poll_every_n_sec`, default 10s), NOT ~65 min. Time it.
2. Return code is **5** (RemoteExecutionFailed), not 4 (PolicyTimeout) and not a raw traceback. On the API path (backend/orchestration.py) it surfaces as a prompt "cloud dispatch error: cloud-run: job=? execution=? reached terminal state 'Failed' with NO completion marker ?" job-failure, not a stall.
3. The diagnostic names the execution, the terminal state, and the `gcloud run jobs executions

describe ? --region ?` command.
4. Sanity: a NORMAL healthy dispatch still succeeds (rc 0) ? confirm the status probe didn't introduce a false-positive that aborts good runs (watch for the proto-plus unset-completion_time ? None behavior holding on the real API).

CLEAN UP + REPORT
- Restore the `rally-worker` Job image digest if you pinned an old one; tear down any test VM/disk.
- Record the outcome (timing, rc, the actual RemoteExecutionFailed message text) in a runlog under docs/COMPUTE_DECOUPLED_SERVING/runlogs/ and note it on the [[cloud-run-exec-status-fastfail]] memory. If the real API field names / proto-plus marshaling differ from the unit-test assumptions (e.g. get_execution returns something the classifier misreads), that's a real bug ? file it and reference `_classify_execution_state` in backend/compute/cloud_run.py.

BRANCHING SAFETY: branch from origin/master explicitly (`git fetch origin` ? `git checkout -b <name> origin/master`); this repo has concurrent sessions. Any code change needs the pre-merge gate: full suite (`python run_all_tests.py`) + `ruff check .` + `mypy backend training` + coverage floor 70.

2. assistant (2026-07-07T16:46:17.717Z)

I'll start by orienting myself: pulling master, reading the handoff memory, and understanding the code under test. Let me kick off several reads in parallel.

3. user (2026-07-07T16:46:20.187Z)

C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer
---BRANCH---
master
---STATUS---
?? score_golden_gate.py
?? score_local.py

4. user (2026-07-07T16:46:20.508Z)

1 ---
2 name: cloud-run-exec-status-fastfail
3 description: PR #515 (fix/cloud-run-exec-status-fastfail) ? run_infer now polls the Executions API to fail fast when a worker crashes before the done-marker
4 metadata:
5 node_type: memory
6 type: project
7 originSessionId: e83b09d8-e113-4d4d-9411-cd80ae217af9
8 ---
9
10 **Branch `fix/cloud-run-exec-status-fastfail`** (commit `145854e`, **rebased onto current
11 `origin/master` @ 20862d7** = master with #513 skip-validation + #512 validator-decode both merged)
12 ? **pushed; PR https://github.com/Khelsutra/badminton-highlight-indexer/pull/515 OPEN + MERGEABLE**
13 (2026-07-07). Full gate reproduced on the final rebased tree: full suite **1833 passed /1 skipped**, coverage
14 **85.66%** (floor 70), `ruff` + `mypy backend training` clean. Only my 5 files change (verified via
15 `git diff origin/master...HEAD` ? three-dot; a two-dot diff briefly looked like it "deleted" the #512
16 validator files, but that was just being one commit behind, since fixed by rebasing forward).
17
18 **What it fixes:** `CloudRunCompute.run_infer` (backend/compute/cloud_run.py) bucket-polled the
19 done-marker ONLY; a worker that terminated before writing the marker (old image argparse-aborting

```
20      on an unknown flag like `--skip-validation`, OOM, any pre-marker exception) made the
dispatcher
21      poll the full `deployment.remote_poll_max_wait_sec` (~65 min) then fail opaquely.
22
23      **How:** each poll iteration (`_poll_check`) checks the marker FIRST (sole success
signal ? a
24      worker SUCCESS *or FAILURE* marker still wins), and when it's absent probes the Cloud
Run
25      Executions API via the new NON-abstract `CloudRunJobClient.execution_terminal_state`
(default
26      `None` = additive/default-OFF; real impl uses `get_execution` completion_time + per-task
counts).
27      A terminal execution + no marker ? `RemoteExecutionFailed` (new, in
`backend/compute/base.py`)
28      after ONE final marker re-check to close the write race. `_dispatch_remote` maps it to
**rc 5**
29      (distinct from rc 4 poll-timeout, which means the exec may still be running). Probe is
best-effort:
30      a raising/flaky status API is swallowed ? keep polling, never fails a healthy run.
31
32      **Why it exists:** surfaced by the adversarial review of the worker-skip-revalidation PR
33      (`fix/worker-skip-revalidation-on-dispatch`), whose interim mitigation was a "deploy
control-plane
34      + worker together" note. This is the durable fix for the whole "worker crashed before
the
35      done-marker" class.
36
37      **Blocker caught + fixed (commit `ab03af1`, on PR):** the adversarial verification
workflow (4 lenses)
38      found the real-client classifier gated terminality on `completion_time.seconds`, but
google-cloud-run is
39      **proto-plus** ? a Timestamp marshals to a Python `datetime` (`.second`, NOT `.seconds`)
when set / `None`
40      when unset. So `getattr(ct,"seconds",0)` was ALWAYS 0 ? `execution_terminal_state`
returned None for EVERY
41      exec ? the fast-fail was a **silent no-op in prod**; tests passed only because fakes
override it + the real
42      body was `# pragma: no cover`. Fixed: terminality = `completion_time is None` (verified
vs proto-plus 1.28.0);
43      extracted a PURE, unit-tested `_classify_execution_state(exe)` (fake test pins the shape
for CI + an
44      `importorskip` real-`run_v2.Execution` test). **LESSON: don't hide real-SDK logic behind
pragma:no-cover ?
45      extract the pure part and test it.** Only `get_execution` stays untestable.
46
47      **MERGED** 2026-07-07 ? **PR #515 squash-merged to master as `ef41473`** (CI 11/11
green: full suite,
48      ruff, mypy, wheel smoke, license/leak scans, cross-OS golden fixtures). Branch deleted.
Final gate:
49      1835? / cov 85.68%. So the crash-before-marker fast-fail (rc 5) is now on master;
nothing pending.
50
51      **Follow-up FILED (task_069a7a19):** the fast-fail relies on the real-client Executions-
API classifier,
52      which is `# pragma: no cover` for the `get_execution` fetch ? only owner-verifiable on a
real Cloud Run
53      dispatch. Live smoke queued: dispatch `--skip-validation` to a deliberately-old worker
image ? expect
54      rc 5 within a poll interval (not a 65-min hang), and confirm a healthy dispatch still
returns rc 0.
55
```

5. user (2026-07-07T16:46:21.001Z)

```
1       ---
```

```
2       name: session-handoff-gpu-resident
3       description: "Start here to pick up the GPU-resident WASB rollout (issue #506) ?
implementation PR #507 is open; golden gate next"
4       metadata:
5         type: project
6       ---
7
8       # Session Handoff ? GPU-resident WASB inference (rollout)  ·  issue #506 · PR #507
MERGED
9
10      ## SHIPPED 2026-07-07 ? GPU-resident NVDEC decode is LIVE in cloud-serving
11      Full arc done: PR #507 (feature) + PR #508 (cloud-serving preset default =
nvdec_resident) both merged to master (44a2f47, 143dbb9). The `rally-worker` Cloud Run Job
(asia-southeast1) runs the new image `rally-worker:latest` `sha256:465a9250?` (torch 2.6 +
PyNvVideoCodec 2.1.0 + baked NVDEC libs) with `WASB_DECODE_BACKEND=nvdec_resident` set ? **nvdec
is the live serving default**, validated on the real L4 (GX010128 served trajectory reproduced
the VM: 26479?26481 visible; cpu+nvdec+canary smokes all ?). Record:
`docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_gpu-resident-
nvdec_2026-07-07.md`.
12      - **ROLLBACK levers:** `gcloud run jobs update rally-worker --region asia-southeast1
--remove-env-vars WASB_DECODE_BACKEND` (? torch-2.6 cpu path, same image); or `--image ?/rally-
worker@sha256:3bfefaf6?` (? prior torch-1.11 image).
13      - **PRs:** #507 (feature) + #508 (preset default) + #509 (dispatch forwarding + release
hardening) all merged. master @ 66756ad.
14      - **DONE ? control-plane + worker both on image D2 `sha256:09d2edf7` (built from master
66756ad = #507+#508+#509).** Redeployed 2026-07-07: `gcloud run services update rally-control-
plane --image ?:latest` ? rev **00017-qbd** (100% traffic, booted healthy: uvicorn up, cloud-
serving profile ACTIVE, edge-auth on); `gcloud run jobs update rally-worker --image ?:latest`.
So the #509 dispatch **forwarding** + #508 preset default are now LIVE in the control-plane
image. NB the FIRST control-plane deploy this session used the stale `465a9250` (built from
44a2f47, pre-#508/#509) ? rev 00016; superseded by 00017. Rollback: `gcloud run services update-
traffic rally-control-plane --to-revisions rally-control-plane-00015-kp9=100` (last pre-nvdec
rev) or `--to-revisions ?-00016-?` .
15      - **`WASB_DECODE_BACKEND=nvdec_resident` env is still set on the worker Job** ? now
REDUNDANT with the forwarded --set + preset (both say nvdec, env just wins), kept as belt-and-
suspenders + manual override. Safe to remove ONLY after a real /api/process dispatch confirms
the forwarding path live (needs a CF Access JWT ? not verifiable this session; forwarding is
deployed + unit-tested but not live-dispatch-verified).
16      - `DEPLOY_REPORT_?gpu-resident-nvdec_2026-07-07.md` is current: PR #510 added the §5
"Update" (D2 redeploy of control-plane rev 00017 + worker) and corrected §6. All four PRs
merged: #507 #508 #509 #510; master @ 3be0935.
17      - **Wrinkles fixed (#509):** dispatch now forwards
indexing.wasb.{decode_backend,batch_size,prefetch_batches} (worker has no DEPLOYMENT_PROFILE ?
never applied the preset; batch tuning was silently lost too); added `.gcloudignore` (Cloud
Build had none ? relied on the .gitignore fallback to skip 8 GB output/scratch); added
`deploy/cloudrun/post_deploy_smoke.ps1`. See [[gpu-vm-setup-gotchas]].
18      - WASB-C3 weights-license gate still open (unchanged; pre-existing).
19
20      ## Live CUJ 2026-07-07 ? release PROVEN, 3 pre-existing gaps surfaced (all filed as
spawned tasks)
21      Owner ran a real /api/process CUJ ("Video 1 - Badminton.MP4", a 5312x2988 **10-bit
HEVC** 120Mbps clip). Outcomes:
22      - **#509 forwarding PROVEN live**: the cloud dispatch put `--set
indexing.wasb.decode_backend=nvdec_resident --set ?batch_size=64 --set ?prefetch_batches=4` into
the worker exec args. And a sharp-clip green run on the D2 worker (exec `rally-worker-ql56s`,
env-driven nvdec) = 19 rallies, success. Serving path is GREEN for valid footage.
23      - **Gap 1 ? slow validation (task_dcee8bf0):** ~125 random `cap.set(POS_FRAMES)` seeks
across scene_cut(100)/blur(15)/relevance(10), each their own VideoCapture, on 5.3K 10-bit HEVC
on the CPU-only control-plane ? ~6.5 min. Fix: single sequential/keyframe pass.
24      - **Gap 2 ? compute=local footgun (task_81572190):** SPA sent compute=local ? in-process
on the weightless control-plane ? "weights not found" fail. Guard/redirect it.
25      - **Gap 3 ? worker re-validates with DEFAULT thresholds (task_72d98f83):** control-plane
min_blur=8.0 (baked) NOT forwarded; worker re-validates at default 20.0 and REJECTED this soft
clip (sharpness 11.9) ? rc2. This is what failed the CUJ on the cloud path. Fix: skip worker re-
validation on dispatch (control-plane is authoritative) or forward validation config.
```

- **Validation-latency design (my rec):** don't move validation to GPU standalone. (1) stop the double-validation, (2) kill the seeks (sequential/keyframe), (3) end-state = ffprobe metadata pre-gate on control-plane + fold content checks (blur/court) INTO the worker's existing GPU decode pass (decode once, validate+detect together). All 3 gaps are pre-existing, NOT caused by the nvdec release.

(historical) release/rollout phase ? superseded by SHIPPED above

PR #507 squash-merged to master as **44a2f47** (branch deleted). Pre-merge gate reproduced locally: full suite 1764 passed / coverage 85.02% / ruff+mypy clean; CI 11/11 green; 2 optional review follow-ups landed (PyNvVideoCodec==2.1.0 pin in both deploy files; decode_backend persisted in native-runner detection_params). Real-L4 validation + golden gate PASS + latency/resource A/B all done (see PR #507 comment + below). **Remaining = the serving release (image rebuild + gated nvdec flip); decode_backend still defaults cpu so nothing in prod has changed yet.**

Serving release plan (live infra: Job `rally-worker` + service `rally-control-plane`, asia-southeast1; ONE image `rally-worker:latest` drives both ? gen_service_yaml.py:193)

- **Phase A (ship image, no behavior change):** `gcloud builds submit --config=deploy/cloudrun/cloudbuild.yaml --project gen-lang-client-0306026826 .` ? new `rally-worker:latest` (torch2.6+pynvc2.1.0+NVDEC libs). Push does NOT change prod (Job/service pin a digest). Cut over via `gcloud run jobs update rally-worker --region asia-southeast1 --image ?:latest` + redeploy service (`gen_service_yaml.py` ? `services replace`). decode_backend defaults cpu ? cpu path under torch2.6 (validated F1 0.576?0.582). Smoke 1 clip. Rollback: `jobs update --image <old digest>`.
- **Phase B (enable nvdec, gated + reversible):** flip via WORKER env `WASB_DECODE_BACKEND=nvdec_resident` (native runner reads env first ? no code/preset change, instant rollback by removing it). ? THE untested risk: NVDEC on the real Cloud Run L4 ? the image bakes libnvcuvid 580.159.03; Cloud Run's host-driver ABI is unverified there (only tested on a self-managed L4 VM). MUST smoke 1 clip on the Job before trusting it. Then canary; then durable default = add `decode_backend: nvdec_resident` to the cloud-serving preset (presets.py) + rebuild + Launch Report.
- Cost: build ~$0.3; each Job smoke ~$0.4?0.7 (L4). Rollback levers at every step.

You are here

The implementation is DONE and **PR #507 is open** (branch `feat/506-gpu-resident-nvdec`, 2026-07-07): default-OFF `indexing.wasb.decode_backend = "cpu" | "nvdec_resident"`, the validated NVDEC?affine-`grid_sample`?GPU-normalize path wired into `backend/pipeline/detectors/wasb_infer.py` (`NvdecResidentFrontend` + `run_video_nvdec_resident`, same resumable cache keyed by decode_backend), native-runner threading + healthcheck gates (CUDA + PyNvVideoCodec), and the env bump (wasb env ? py3.10 + torch 2.6.0+cu124 + PyNvVideoCodec; libnvcuvid/libnvidia-encode staged in `deploy/cloudrun/Dockerfile` + `deploy/digitalocean/gpu_setup.sh`). Verified locally: 1764 tests green, coverage 85% (floor 70), ruff/mypy/link/leak clean; a 14-agent adversarial review confirmed 7 minor findings, all fixed (notably `padding_mode="zeros"` = cv2 BORDER_CONSTANT for non-16:9 sources). Research evidence unchanged: [[gpu-resident-modernization]], issue #506 comments (F1 0.574 ? baseline 0.582 on GX010128 at SERVED, ~1.7×, GPU 47?79%).

Real-L4 validation ? DONE + golden gate PASS 2026-07-07 (this session, ~$3 of the $100 budget; VM torn down)

Validated end-to-end on an L4 (`rally-506`, torn down clean, no orphan disks) ? full evidence in the PR #507 comment. All GREEN: (1) `gpu_setup.sh` builds the env clean (py3.10 + torch 2.6.0+cu124 cuda True + PyNvVideoCodec 2.1.0 + NVDEC lib extraction); (2) on-box **parity gate PASS** (`pytest -k parity`, 480s ? needed test-name fix 9267255, the documented `-k parity` selected 0 tests); (3) crash/resume byte-identical; (4) cpu?nvdec cache isolation correct; (5) **Cloud Run image builds + `--gpus all` smoke PASS** (baked NVDEC libs decode; native healthcheck ok with nvdec_resident); throughput 43.7 fps / GPU 82% on GX010128.

Golden gate @ SERVED = PASS (10 baseline-comparable clips, 445 rallies): mean ?f1@0.5 **+0.020**, mean ?tolF1 +0.006, worst ?0.009 (**GX010128 must-pass: 0.574 vs 0.582 ?**), 8/10 improved-or-flat. The other 5 golden clips are 4K-sourced (baselines are 1080p) so a vs-baseline number would conflate resolution with decode ? the parity gate already covers nvdec?cpu on identical pixels, so they add nothing to the verdict.

Latency + resource A/B (2nd L4 `rally-506b`, torn down; GX010128, batch 8, telemetry in `gs://?nightly/dev/506/results/bench/`): nvdec **43.7 fps @ GPU 83% / CPU 30%** vs cpu-stream(no-prefetch) 15.0 fps @ GPU 28% / CPU 49%; GPU mem +170 MiB (of 24GB), RAM ~2?3 GB flat.

```
The win is the **bottleneck flip (GPU?, CPU?)**; the 2.9× is vs the untuned streaming path ? vs
prefetch-tuned cpu the realistic end-to-end is ~1.6?1.8× (#506 spike). All in the PR #507
comment.
44
45      ## Staged assets (durable)
46      - **Golden video working set**: all 14 originals in `gs://khelsutra-rally-
corpus/videos/golden/<name>.mp4` + the 4 baseline-source 1080p proxies in `videos/golden_1080p/`
(mahadevpura_2, Badminton_BXH_2, Boxhill_Doubles, GX020094). Frame-count-verified against each
golden trajectory. (GOLDEN_VIDEOS.md could record this GCS location in a follow-up ? kept out of
PR #507 to not muddy its scope.)
47      - Gate trajectories: `gs://khelsutra-rally-nightly/dev/506/results/golden/*_nvdec.csv`.
Re-score with repo-root `score_golden_gate.py` (untracked dev tool, like `score_local.py`;
labels from `?corpus/labels/`, baselines from `?corpus/trajectories/`).
48
49      ## Next steps
50      1. **Merge PR #507** ? real-L4-validated + golden-gate PASS; CI green locally.
51      2. **Flip the default**: owner Launch Report ? set `decode_backend=nvdec_resident` in
the cloud-serving preset (`backend/config/presets.py`). The gate that gated this is now green.
52
53      ## Ramp-up kit
54      - PR #507 (full change description + verification record) and issue #506 (all comments ?
measured evidence).
55      - New tests: `tests/test_wasb_nvdec_resident.py` (grid/cv2 parity incl. non-16:9, cache
keying, CLI guards, the skippable on-box parity gate).
56      - Reference code still in `scratch/gpu_resident/` (`exp_t2_full.py`, `score_local.py`,
`t2setup_env.sh`, `telemetry.sh`).
57
58      ## Key decisions (don't relitigate)
59      - GPU resize MUST be affine `grid_sample` matching `cv2.warpAffine` ? NOT
`F.interpolate` (F1 0.58?0.07; only shows on hard clips). Locked by unit tests now.
60      - `padding_mode="zeros"` (cv2's BORDER_CONSTANT(0)), not "border" ? matters only for
non-16:9 sources; identical to the validated run on 16:9.
61      - PyNvVideoCodec (MIT) for GPU decode; TorchCodec couldn't load on the DL image.
62      - Score with SERVED preset (cap6+R4); raw (0.02,2.0,2.0) is degenerate.
63      - Default-OFF; nvdec/cpu detection caches never cross-replay (manifest keyed by
decode_backend).
64      - WASB weights/NYCU-dataset license (WASB-C3) is a separate launch gate, not a perf
blocker.
65
```

6. assistant (2026-07-07T16:46:24.787Z)

Repo is on `master`, clean except two untracked dev tools. Let me pull, read the code under test, and check my gcloud access in parallel.

7. user (2026-07-07T16:46:30.902Z)

```
---PULL---
Already up to date.
---HEAD---
ef41473 fix(cloud-run): fail fast when a worker execution terminates without a done-marker
(#515)
f5c9dd9 chore(ui): re-vendor SPA bundle with the hosted compute-picker cloud default (#516)
ec2d080 fix(serving): fail fast when compute=local forces weightless in-process detection (#511)
```

8. user (2026-07-07T16:46:36.996Z)

```
/c/Users/avidu/AppData/Local/Google/Cloud SDK/google-cloud-sdk/bin/gcloud
---VERSION---
Google Cloud SDK 571.0.0
beta 2026.05.29
bq 2.1.32
core 2026.05.29
gcloud-crc32c 1.0.0
```

```
---CONFIG---
[accessibility]
screen_reader = False
[billing]
quota_project = gen-lang-client-0306026826
[core]
account = avi.dullu@gmail.com
disable_usage_reporting = False
project = gen-lang-client-0306026826

Your active configuration is: [default]
```

9. assistant (2026-07-07T16:46:41.482Z)

Master confirmed at `ef41473` (#515). gcloud authenticated. Now let me read the code under test.

10. user (2026-07-07T16:46:42.363Z)

```
1       """``CloudRunCompute`` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2       for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4       Flow (the cloud-serving compute path):
5         1. **Dispatch** a Cloud Run Job execution whose container runs ``python -m
backend.worker infer
6            --video-uri <gs> --out-uri <gs> --done-uri <gs> ?`` (the SAME worker, on the L4
GPU).
7         2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage
layer (D16 ? the
8            completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9            it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10           cold-start is absorbed by the async poll. On a poll TIMEOUT (``max_wait_sec`` can
undercut the
11           job's own ``--task-timeout`` budget) the shim does ONE final marker check ? a slow-
but-healthy
12           worker that finished just past the deadline is rescued, not wasted ? then best-
effort CANCELS
13           the still-running execution so the L4 doesn't keep billing until ``--task-
timeout``.
14           *Fast negative path:* while the marker stays the SOLE success signal, each poll
also probes the
15           Cloud Run Executions API ? if the execution has TERMINATED without ever writing a
marker (a
16           worker crash/OOM/argparse-abort before the marker push), the shim fails fast +
diagnostically
17           (``RemoteExecutionFailed``) instead of spinning the full ~65-min budget on a futile
wait.
18        3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
19           ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
20
21      The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` +
native detector)
22      so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
23
24      The Cloud Run trigger is an **injected client** (``CloudRunJobClient``) so the whole
shim is unit-tested
25      with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-
run`` and is
26      owner-verified on the M7 real run.
27      """
28
29      from __future__ import annotations
```

```
30
31      import json
32      import logging
33      import os
34      import tempfile
35      import uuid
36      from abc import ABC, abstractmethod
37      from typing import Any, Callable, List, Optional
38
39      from backend.compute.base import (
40          ComputeBackend,
41          ComputeJobSpec,
42          ComputeResult,
43          RemoteExecutionFailed,
44      )
45      from backend.infrastructure.execution_policy import (
46          DEFAULT_POLICY,
47          ExecutionPolicy,
48          PolicyTimeout,
49          poll_until,
50      )
51      from backend.storage import fetch, get_storage, parse_uri
52
53      LOG = logging.getLogger("compute.cloud_run")
54
55      # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
branch.
56      # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
runner.)
57      _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
58          "deployment.compute_target=local_gpu",
59          "indexing.detector_impl=native",
60      )
61
62
63      class CloudRunJobClient(ABC):
64          """Triggers a Cloud Run **Job** execution with per-execution container arg
overrides.
65
66          Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
implementation
67          overrides the job's container ``args`` for this execution and starts it."""
68
69          @abstractmethod
70          def run_job(
71              self,
72              job_name: str,
73              argv: List[str],
74              *,
75              region: str,
76              project: Optional[str] = None,
77          ) -> str:
78              """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
return an
79              execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""
80
81          @abstractmethod
82          def cancel_execution(self, execution: str) -> None:
83              """Cancel a running execution by the name ``run_job`` returned. Called when the
bucket-poll
84              times out so the abandoned execution doesn't keep billing the GPU until
``--task-timeout``.
85              May raise ? the caller treats every failure as best-effort (the original poll
timeout is
```

```
86              NEVER masked by a cancel error)."""
87
88          def execution_terminal_state(self, execution: str) -> Optional[str]:
89              """Return the execution's TERMINAL state as a short label (``"Failed"`` /
``"Cancelled"`` /
90              ``"Succeeded"``) if it has finished, else ``None`` (still pending/running, or
the state
91              can't be determined).
92
93              ADVISORY, not authoritative: it feeds ``run_infer``'s fast negative path so a
worker that
94              terminated WITHOUT writing the done-marker (crash/OOM/pre-marker abort) fails
fast instead
95              of bucket-polling the full budget. The done-marker stays the ONLY success
signal.
96
97              NON-abstract with a default of ``None`` (additive / default-OFF): a client that
can't ? or
98              chooses not to ? poll the Executions API degrades transparently to today's
marker-only
99              polling. Implementations MAY raise on a transient API error ? ``run_infer``
treats a raise
100             as ``None`` (unknown ? keep waiting), so a flaky status API never fails a
healthy run."""
101             return None
102
103
104     class GoogleCloudRunJobClient(CloudRunJobClient):
105         """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
106         a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""
107
108         def run_job(
109             self,
110             job_name: str,
111             argv: List[str],
112             *,
113             region: str,
114             project: Optional[str] = None,
115         ) -> str:
116             # A None/empty project would build a bogus "projects/None/locations/..." path
(job_path
117             # requires a real str). Fail loudly BEFORE the SDK import so the misconfig is
exercisable
118             # in CI (where google-cloud-run is absent) and so mypy narrows project to str
for job_path.
119             if not project:
120                 raise RuntimeError(
121                     "GOOGLE_CLOUD_PROJECT is unset ? the real Cloud Run dispatch needs a GCP
project id "
122                     "to resolve the job path; set it on the control plane (see
get_compute_backend)."
123                 )
124             try:
125                 from google.cloud import run_v2  # type: ignore[attr-defined]
126             except Exception as exc:  # pragma: no cover - real SDK not present in CI
127                 raise RuntimeError(
128                     "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
129                     "control plane (it is intentionally NOT a CI/test dependency)."
130                 ) from exc
131             client = (
132                 run_v2.JobsClient()
133             )  # pragma: no cover - exercised only on the real M7 run
```

```
            name = client.job_path(project, region, job_name)
            overrides = run_v2.RunJobRequest.Overrides(
                container_overrides=[
                    run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)
                ]
            )
            op = client.run_job(
                request=run_v2.RunJobRequest(name=name, overrides=overrides)
            )
            return (
                getattr(op, "metadata", None)
                and getattr(op.metadata, "name", "")
                or "execution"
            )

        def cancel_execution(self, execution: str) -> None:
            # run_job falls back to the literal string "execution" when the operation
metadata carries
            # no name ? that is NOT a resolvable resource name. Fail loudly BEFORE the SDK
import
            # (mirrors run_job's project guard, #342) so the misuse is exercisable in CI
(where
            # google-cloud-run is absent).
            if "/executions/" not in (execution or ""):
                raise RuntimeError(
                    f"not a fully-qualified Cloud Run execution name: {execution!r} ?
expected "
                    "projects/<p>/locations/<l>/jobs/<j>/executions/<e> (run_job could not
resolve "
                    "one from the operation metadata, so there is nothing to cancel by
name)."
                )
            try:
                from google.cloud import run_v2  # type: ignore[attr-defined]
            except Exception as exc:  # pragma: no cover - real SDK not present in CI
                raise RuntimeError(
                    "google-cloud-run is required to cancel a Cloud Run execution; install
it on the "
                    "control plane (it is intentionally NOT a CI/test dependency)."
                ) from exc
            client = (
                run_v2.ExecutionsClient()
            )  # pragma: no cover - exercised only on a real run
            client.cancel_execution(request=run_v2.CancelExecutionRequest(name=execution))

        def execution_terminal_state(self, execution: str) -> Optional[str]:
            # Advisory status probe (see the ABC docstring): fetch the execution and
classify it via the
            # pure, unit-tested ``_classify_execution_state``. A non-resolvable name /
absent SDK returns
            # None (can't probe ? keep marker-polling) ? unlike run_job/cancel this never
raises on bad
            # input, because the probe is best-effort and must not fail a healthy run.
            if "/executions/" not in (execution or ""):
                return None
            try:
                from google.cloud import run_v2  # type: ignore[attr-defined]
            except Exception:  # pragma: no cover - real SDK not present in CI
                return None
            client = (
                run_v2.ExecutionsClient()
            )  # pragma: no cover - exercised only on a real run
            exe = client.get_execution(  # pragma: no cover - real run only
                request=run_v2.GetExecutionRequest(name=execution)
            )
```

```
189             return _classify_execution_state(exe)  # pragma: no cover - real run only
190
191
192     def _classify_execution_state(exe: Any) -> Optional[str]:
193         """Map a Cloud Run ``run_v2.Execution`` to a terminal state label (``"Failed"`` /
``"Cancelled"``
194         / ``"Succeeded"``), or ``None`` if it is still pending/running. PURE (no
SDK/network) so it is
195         unit-tested against both real proto-plus ``Execution`` objects and lightweight fakes
? the actual
196         ``get_execution`` fetch is the only part that can't run in CI.
197
198         ``google-cloud-run`` is a **proto-plus** library, so a ``google.protobuf.Timestamp``
field is
199         marshalled to a native Python ``datetime`` when set and to ``None`` when unset ? NOT
to a proto
200         ``Timestamp`` with a ``.seconds`` field (a ``datetime`` has ``.second``, singular).
Terminality is
201         therefore "``completion_time`` is not None"; the per-task counts (plain ints) then
classify it,
202         with a cancel taking precedence (a cancelled execution reports ``cancelled_count >
0``)."""
203         if getattr(exe, "completion_time", None) is None:
204             return None  # no terminal completion timestamp yet ? still pending/running
205         if getattr(exe, "cancelled_count", 0):
206             return "Cancelled"
207         if getattr(exe, "failed_count", 0):
208             return "Failed"
209         if getattr(exe, "succeeded_count", 0):
210             return "Succeeded"
211         return "Failed"  # terminal but no positive success ? treat as failed (conservative)
212
213
214     class CloudRunCompute(ComputeBackend):
215         """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
216
217         def __init__(
218             self,
219             *,
220             run_client: CloudRunJobClient,
221             job_name: str,
222             region: str,
223             project: Optional[str] = None,
224             storage_config: Optional[dict] = None,
225             policy: ExecutionPolicy = DEFAULT_POLICY,
226             token: Optional[str] = None,
227             container_overrides: Optional[tuple[str, ...]] = None,
228         ) -> None:
229             self._client = run_client
230             self._job_name = job_name
231             self._region = region
232             self._project = project
233             self._storage_config = storage_config or {}
234             self._policy = policy
235             # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
236             # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
237             self._token = token
238             self._container_overrides = (
239                 _DEFAULT_CONTAINER_OVERRIDES
240                 if container_overrides is None
241                 else tuple(container_overrides)
242             )
243
```

```
        def run_infer(
            self,
            spec: ComputeJobSpec,
            *,
            on_wait: "Callable[[float], None] | None" = None,
        ) -> ComputeResult:
            out_root = spec.out_uri.rstrip("/")
            token = self._token or uuid.uuid4().hex
            done_uri = f"{out_root}/_dispatch/{token}.done"
            argv: List[str] = [
                "infer",
                "--video-uri",
                spec.video_uri,
                "--out-uri",
                spec.out_uri,
                "--done-uri",
                done_uri,
            ]
            if spec.segmenter:
                argv += ["--segmenter", spec.segmenter]
            if spec.skip_validation:
                # The control-plane already validated + only dispatches on PASS ? tell the
worker not to
                # re-decode and re-validate (redundant, and its config may resolve different
thresholds).
                argv.append("--skip-validation")
            for kv in (*spec.config_overrides, *self._container_overrides):
                argv += ["--set", kv]

            execution = self._client.run_job(
                self._job_name, argv, region=self._region, project=self._project
            )
            LOG.info(
                "cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
                self._job_name,
                execution,
                done_uri,
            )

            try:
                marker = poll_until(
                    lambda: self._poll_check(done_uri, str(execution or "")),
                    self._policy,
                    label="cloud-run-infer",
                    logger=LOG,
                    # Live heartbeat (#482): each still-waiting tick reaches the caller so
                    # job.progress moves during the whole GPU run instead of freezing.
                    on_tick=on_wait,
                )
            except PolicyTimeout as timeout_exc:
                # RemoteExecutionFailed (the fast negative path in _poll_check) is NOT a
PolicyTimeout,
                # so it propagates past here uncaught ? the dispatch handler maps it to a
distinct,
                # clearly-labelled failure instead of the ~65-min opaque timeout this branch
handles.
                marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
            video_id = str(marker.get("video_id", ""))
            # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
            # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
            rc = int(marker.get("returncode", 1))
            if not video_id:
                LOG.warning(
```

```
                    "cloud-run: job=%s completion marker present but empty/unreadable (no "
                    "video_id) ? treating as failure",
                    self._job_name,
                )
                rc = rc or 1
            outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
            report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
            LOG.info(
                "cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc
            )
            return ComputeResult(
                returncode=rc,
                outputs_uri=outputs_uri,
                video_id=video_id,
                report=report,
                execution=str(execution or ""),
            )

        def _poll_check(self, done_uri: str, execution: str) -> Optional[dict]:
            """One bucket-poll iteration, hardened with an Executions-API fast negative
path.

            The done-marker is the SOLE success signal and is checked FIRST ? a present
marker always
            wins (even if the execution's status later flips), so a worker that wrote a
SUCCESS *or a
            FAILURE* marker resolves through the normal path with its real returncode. Only
when the
            marker is absent do we consult the execution status: a worker that TERMINATED
without ever
            writing a marker (crash / OOM / an old image argparse-aborting on an unknown
flag) would
            otherwise make this poll spin the full ``remote_poll_max_wait_sec`` (~65 min)
before failing
            opaquely. Detecting that terminal state collapses the whole class to a prompt,
actionable
            failure.

            Returns the marker dict (resolve the poll), ``None`` (keep waiting), or raises
            ``RemoteExecutionFailed`` (abort the poll immediately ? the execution is over,
no marker
            will ever appear)."""
            marker = self._read_marker(done_uri)
            if marker is not None:
                return marker  # success signal ? authoritative even if status later flips
            state = self._terminal_execution_state(execution)
            if state is None:
                return None  # still pending/running, or status unknown ? keep bucket-
polling
            # The execution is terminal but no marker is present. Re-check the marker ONCE
to close the
            # race where the worker wrote it between our two reads (mirrors the post-timeout
rescue in
            # _handle_poll_timeout). If it is genuinely absent, the process is gone and no
marker will
            # EVER appear ? fail fast + diagnostically rather than wait out the whole poll
budget.
            marker = self._read_marker(done_uri)
            if marker is not None:
                return marker
            raise RemoteExecutionFailed(
                f"cloud-run: job={self._job_name} execution={execution or '<unknown>'}
reached "
                f"terminal state {state!r} with NO completion marker ? the worker exited
before "
```

```
                "writing one (crash / OOM / pre-marker abort, or a failed marker upload on
an "
                "otherwise-Succeeded run). The execution is over, so the marker will never
appear; "
                "inspect the Cloud Run Job execution log to diagnose: "
                f"`gcloud run jobs executions describe {execution or '<execution>'} "
                f"--region {self._region}`."
            )

        def _terminal_execution_state(self, execution: str) -> Optional[str]:
            """Best-effort wrapper over the client's ``execution_terminal_state`` probe. A
raising or
            flaky status API must NEVER kill a healthy wait (the done-marker stays the sole
authority),
            so any exception ? and an empty/unresolvable execution name ? degrades to
``None`` (unknown
            ? keep polling) and is DEBUG-logged, exactly today's marker-only behaviour."""
            if not execution:
                return None
            try:
                return self._client.execution_terminal_state(execution)
            except Exception:  # noqa: BLE001 ? advisory probe: never let it kill the
bucket-poll
                LOG.debug(
                    "cloud-run: execution status probe failed for %s ? treating as unknown",
                    execution,
                    exc_info=True,
                )
                return None

        def _handle_poll_timeout(
            self, done_uri: str, execution: str, timeout_exc: PolicyTimeout
        ) -> dict:
            """Poll deadline hit ? rescue a just-finished result, else cancel + re-raise.

            The poll budget (``max_wait_sec``, default 1800s) can undercut the deployed
job's own
            ``--task-timeout`` (3600s), so a slow-but-healthy worker may finish just past
the deadline:
            ONE final marker check rescues that completed GPU run instead of wasting it. If
the marker
            is genuinely absent, the execution is still running ? best-effort cancel it
(otherwise the
            L4 keeps billing until ``--task-timeout``) and re-raise a ``PolicyTimeout``
carrying the
            execution name so the operator can verify the state in the GCP console. A cancel
failure
            NEVER masks the original timeout."""
            try:
                late = self._read_marker(done_uri)
            except Exception:  # noqa: BLE001 ? the final check is best-effort too: absence
? failure
                LOG.warning(
                    "cloud-run: final post-timeout marker check failed ? treating as
absent",
                    exc_info=True,
                )
                late = None
            if late is not None:
                LOG.warning(
                    "cloud-run: job=%s exec=%s done-marker found on the final post-timeout
check "
                    "? rescuing the completed result",
                    self._job_name,
                    execution,
```

```
401                 )
402                 return late
403             try:
404                 self._client.cancel_execution(execution)
405             except Exception as cancel_exc:  # noqa: BLE001 ? best-effort: never mask the
timeout
406                 cancel_note = f"best-effort cancel FAILED ({cancel_exc})"
407                 LOG.warning(
408                     "cloud-run: best-effort cancel of execution=%s failed: %s",
409                     execution,
410                     cancel_exc,
411                     exc_info=True,
412                 )
413             else:
414                 cancel_note = "cancel requested"
415                 LOG.info("cloud-run: requested cancel of execution=%s", execution)
416             raise PolicyTimeout(
417                 f"{timeout_exc} [cloud-run job={self._job_name} "
418                 f"execution={execution or '<unknown>'}; {cancel_note} ? verify the execution
"
419                 "state in the GCP console]"
420             ) from timeout_exc
421
422         # --- storage-backed completion signal (bucket-poll) ---
423         def _read_marker(self, done_uri: str) -> Optional[dict]:
424             """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
425             ref = parse_uri(done_uri)
426             if not get_storage(ref.backend, config=self._storage_config).exists(ref):
427                 return None
428             return (
429                 self._read_json(done_uri) or {}
430             )  # empty dict still terminates the poll (present == done)
431
432         def _read_json(self, uri: str) -> dict:
433             # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
434             # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
435             # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
436             try:
437                 ref = parse_uri(uri)
438                 with tempfile.TemporaryDirectory(prefix="cr-read-") as td:
439                     dst = os.path.join(td, ref.name or "obj.json")
440                     local = fetch(uri, dst, config=self._storage_config)
441                     with open(local, "r", encoding="utf-8") as f:
442                         data: Any = json.load(f)
443                 return data if isinstance(data, dict) else {}
444             except Exception:
445                 LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
446                 return {}
447
```

11. assistant (2026-07-07T16:46:46.435Z)

Now let me look at the dispatch layer (`_dispatch_remote` ? rc 5), the deployment poll config, and how the worker/control-plane are wired.

12. user (2026-07-07T16:46:48.400Z)

```
Found 23 files
backend\worker\__main__.py
deploy\cloudrun\README.md
tests\test_api_cloud_dispatch.py
```

```
tests\test_compute_backend.py
backend\compute\__init__.py
backend\compute\base.py
backend\compute\cloud_run.py
backend\orchestration.py
backend\config\models.py
docs\CONTROL_PLANE_DURABLE_REBUILD.md
tests\test_restart_recovery.py
backend\main.py
backend\api\job_worker.py
tests\test_execution_policy.py
backend\infrastructure\execution_policy.py
tests\test_config_deployment.py
tests\test_main_module_budget.py
docs\COMPUTE_DECOUPLED_SERVING\runlogs\DEPLOY_REPORT_cloud-serving_a1-cuj_2026-07-04.md
docs\archives\past_projects\AUDIT_REPORT_khelsutra-guru_2026-07-02-COMPLETED-2026-07-02.md
docs\archives\past_projects\KHELSUTRA_GURU_HEALTH_REMEDIATION_PLAN-FOLDED-2026-07-02.md
backend\providers\gemini.py
docs\COMPUTE_DECOUPLED_SERVING\README.md
docs\EXECUTION_POLICY.md
```

13. user (2026-07-07T16:46:49.880Z)

```
No matches found
```

14. user (2026-07-07T16:46:53.935Z)

```
1-"""Processing / compile orchestration, extracted from ``backend.main`` (P23; re-cut of #390).
2-
3-The full hash -> validate -> segment -> report pipeline (``_execute_processing``), the
4:Cloud Run dispatch + ingest path (``_maybe_dispatch_remote`` / ``_ingest_remote_segments``),
5-and the shared compile resolution + worker handler (``_resolve_compile`` /
``_execute_compile``).
6-Extracted verbatim from ``backend.main`` to shrink the orchestrator; behavior is unchanged.
7-
--
158-# Trajectory segmenters whose IN-PROCESS detect needs model weights present on THIS box. A
hosted
159-# cloud-serving control-plane bakes NO detector weights into its image (only the worker
Cloud Run Job
160-# GCSFuse-mounts them), so forcing compute="local" for one of these there can never succeed
? the
161:# guardrail in _maybe_dispatch_remote fails such a request FAST instead of the cryptic deep
162-# "WASB weights not found". (Names are the registered segmenter identities; see
backend/pipeline/segmenters.)
163-_WEIGHTS_BACKED_SEGMENTERS = frozenset({"wasb_hybrid", "tracknet_hybrid", "fusion_hybrid"})
164-
--
420-    return cached_process_result(db, cfg, vid, "") is not None
421-
422-
423:def _maybe_dispatch_remote(
424-    cfg,
425-    db,
426-    req: "ProcessRequest",
--
640-            on_wait=lambda elapsed: notify(_wait_progress_message(seg_name, elapsed)),
641-        )
642-    except Exception as e:  # noqa: BLE001 ? any dispatch/poll error is a clean job failure,
not a 500
643:        logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
644:        return _failed(f"cloud dispatch error: {e}", True)
645:    if result.returncode != 0:
646:        return _failed(f"cloud job failed (returncode {result.returncode}).", True)
647-    if result.video_id and result.video_id != video_id:
```

```
648-        # The worker re-hashes the bucket bytes; a divergence means the input changed
between this
649-        # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
diagnosable.
--
774-    val_context: Dict[str, Any] = {}
775-    # Compute-path PROVENANCE (the validation method): which backend actually ran the DETECT
?
776-    # "in_process" once the local segmenter is reached, set on the response as "cloud_run"
by
777:    # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
778-    compute_actual: Optional[str] = None
779-    response: Optional[Dict[str, Any]] = None
780-    try:
--
851-        # DEFAULT-OFF: returns None for every non-cloud target ? the in-process segmenter
below runs
852-        # byte-identically. (player_pool/max_frames aren't threaded to the worker `infer`
CLI yet, so
853-        # they're dropped on the cloud path in this slice ? a documented follow-up.)
854:        remote = _maybe_dispatch_remote(
855-            config, db, req, video_id, seg_name, val_results, play_wm, cand_wm, notify
856-        )
857-        if remote is not None:
```

15. user (2026-07-07T16:47:01.819Z)

```
238-    _execute_processing,
239-    _finalize_reingest,
240-    _ingest_remote_segments,
241:    _maybe_dispatch_remote,
242-    _remote_input_size_bytes,
243-    _report_config_for_input,
244-    _resolve_compile,
--
265-    locale: Optional[str] = None
266-    # SPA compute-picker (item 3): "local" forces in-process even on a cloud-configured
server;
267-    # "cloud" forces a Cloud Run dispatch (requires deployment.cloud_available). None ?
server
268:    # default (deployment.compute_target). Only honoured for these two values; see
_maybe_dispatch_remote.
269-    compute: Optional[str] = None
270-    # Explicit opt-in reprocess path. By default, a video whose content hash already has
stored
271-    # play_segments is returned from cache so refresh/Try again cannot re-bill paid
segmenters.
```

16. user (2026-07-07T16:47:06.363Z)

```
171-        return False  # already logged at ERROR by enforce_startup_checks
172-
173-
174:def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
175-    """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc once
the
176-    outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
177-    container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never re-
dispatch.
178-
179-    A remote job that never writes a completion marker exits via one of two CLEAN codes
(distinct
180-    from the local 2/3), never a raw traceback:
```

```
181:      * rc 4 ? the bucket poll exhausted its budget (``PolicyTimeout``): the execution may
still be
182-        running (the shim best-effort cancels it).
183:      * rc 5 ? the execution has TERMINATED without a marker (``RemoteExecutionFailed``): a
worker
184-        crash/OOM/pre-marker abort caught FAST by the Executions-API probe, so instead of
the
185-        ~65-min opaque wait the operator gets an immediate, log-pointing failure."""
186:    from backend.compute import RemoteExecutionFailed
187:    from backend.infrastructure.execution_policy import PolicyTimeout
188-
189-    spec = ComputeJobSpec(
190-        video_uri=args.video_uri,
--
195-    )
196-    try:
197-        result = compute.run_infer(spec)
198:    except RemoteExecutionFailed as e:
199-        logger.error(
200-            "[worker] remote execution terminated without a completion marker: %s", e
201-        )
202:        return 5
203:    except PolicyTimeout as e:
204-        logger.warning(
205-            "[worker] remote compute did not complete (no marker within budget): %s", e
206-        )
207:        return 4
208-    logger.info(
209-        "[worker] remote compute done: video_id=%s rc=%d outputs=%s",
210-        result.video_id,
--
253-    # dispatcher forces compute_target=local_gpu), so it never recursively re-dispatches.
254-    compute = get_compute_backend(config)
255-    if compute is not None:
256:        return _dispatch_remote(compute, args)
257-
258-    storage_cfg = config.storage.model_dump()
259-
--
277-    # resolves different thresholds than the control-plane's ? observed 2026-07-07: a
5312x2988 clip
278-    # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0, then
the worker
279-    # REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the
fact.
280:    # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote)
skips it; the
281-    # direct ``worker infer`` CLI has no control-plane, so it still validates by default.
282-    if getattr(args, "skip_validation", False):
283-        print(
```

17. assistant (2026-07-07T16:48:25.028Z)

Let me read the worker's marker-writing logic (to know what reliably produces a "terminal without marker" crash) and check my local Python/SDK/ADC setup.

18. user (2026-07-07T16:48:26.520Z)

```
1       """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3           python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4               [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5           python -m backend.worker train ...        # Gen-0 train-from-bucket (follow-up)
6
7       The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
```

```
``local://``
8       is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
local runs
9       are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10      ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11      the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13      This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14      (validator registry ? segmenter ? report), never a forked pipeline.
15      """
16
17      from __future__ import annotations
18
19      import argparse
20      import csv
21      import json
22      import logging
23      import os
24      import re
25      import sys
26      import tempfile
27      from typing import Any, List
28
29      from backend import storage
30      from backend.compute import ComputeJobSpec, get_compute_backend
31      from backend.config import (
32          StartupCheckError,
33          enforce_startup_checks,
34          load_config,
35          probe_cuda_available,
36      )
37      from backend.infrastructure.database import Database
38      from backend.pipeline.segmenters.base import segmenter_registry
39      from backend.pipeline.validators.base import registry as validator_registry
40      from backend.reporting import emit_indexer_report
41      from backend.results import Failure
42      from backend.utils.ffmpeg import ffprobe_bin  # P2a: single ffmpeg/ffprobe resolver
43      from backend.utils.hashing import compute_video_id
44
45      logger = logging.getLogger("backend.worker")
46
47
48      def _coerce(v: str) -> Any:
49          """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
str."""
50          low = v.lower()
51          if low in ("true", "false"):
52              return low == "true"
53          for cast in (int, float):
54              try:
55                  return cast(v)
56              except ValueError:
57                  pass
58          return v
59
60
61      def _apply_overrides(config: Any, sets: List[str]) -> None:
62          """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
63          for kv in sets or []:
64              key, sep, val = kv.partition("=")
65              if not sep:
66                  raise ValueError(f"--set expects k=v, got {kv!r}")
```

```
            parts = key.split(".")
            obj = config
            for p in parts[:-1]:
                obj = getattr(obj, p)
            setattr(obj, parts[-1], _coerce(val))


    def _write_intervals_csv(segments: List[dict], path: str) -> None:
        """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
        artifact (same schema as the rally-annotator labels)."""
        with open(path, "w", newline="") as f:
            w = csv.writer(f)
            w.writerow(["start", "end", "shot_count", "ending_reason"])
            for s in segments:
                w.writerow(
                    [
                        f"{float(s.get('start_time', 0.0)):.3f}",
                        f"{float(s.get('end_time', 0.0)):.3f}",
                        s.get("shot_count", ""),
                        s.get("ending_reason", s.get("shot_count_confidence", "")),
                    ]
                )


    def _write_report_json(
        video_id: str, segments: List[dict], status: str, path: str, video_uri: str = ""
    ) -> None:
        with open(path, "w") as f:
            json.dump(
                {
                    "video_id": video_id,
                    "status": status,
                    "segment_count": len(segments),
                    # The SOURCE the detection ran on (CP-rebuild P3, #485): a restart-wiped
                    # control plane re-hydrating from outputs/<md5>/ restores the video row
                    # with its truthful path ? without this, only the segments recover.
                    "video_uri": video_uri,
                    "segments": [
                        {
                            "start": float(s.get("start_time", 0.0)),
                            "end": float(s.get("end_time", 0.0)),
                        }
                        for s in segments
                    ],
                },
                f,
                indent=2,
            )


    def _serialize_and_push_outputs(
        scratch: str,
        out_uri: str,
        video_id: str,
        segments: List[dict],
        status: str,
        storage_cfg: dict,
        video_uri: str = "",
    ) -> List[str]:
        """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and
push both to
        ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.

        Shared by the SUCCESS path and the failure path (``_fail``) so a detector
```

```
timeout/abort leaves a
130         ``status="failed"`` report.json in the bucket ? the control plane's source of truth
131         (``orchestration.cached_process_result`` / ``probe_process_cache`` treat any
non-``success``
132         report as "no usable result" ? retry, and the restart-recovery poll waits for
report.json to
133         EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``."""
134         out_local = os.path.join(scratch, "outputs", video_id)
135         os.makedirs(out_local, exist_ok=True)
136         _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
137         _write_report_json(
138             video_id,
139             segments,
140             status,
141             os.path.join(out_local, "report.json"),
142             video_uri=video_uri,
143         )
144         out_prefix = out_uri.rstrip("/")
145         pushed: List[str] = []
146         for fn in sorted(os.listdir(out_local)):
147             uri = f"{out_prefix}/outputs/{video_id}/{fn}"
148             storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
149             pushed.append(uri)
150         return pushed
151
152
153     def _run_startup_checks(config: Any) -> bool:
154         """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
155         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
156         cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
157         (returns True, ZERO work) when no deployment.profile is selected.
158
159         The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
160         torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
161         WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
162         no-op of work, not just of output."""
163         profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
164         cuda = (
165             probe_cuda_available() if profile else None
166         )  # torch-free on the default-OFF path
167         try:
168             enforce_startup_checks(config, cuda_available=cuda, logger=logger)
169             return True
170         except StartupCheckError:
```

19. user (2026-07-07T16:48:31.271Z)

```
210             result.video_id,
211             result.returncode,
212             result.outputs_uri,
213         )
214         return result.returncode
215
216
217     def _write_done_marker(
218         done_uri: str,
219         video_id: str,
220         returncode: int,
221         segment_count: int,
```

```
222         status: str,
223         storage_cfg: dict,
224         scratch: str,
225     ) -> None:
226         """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
227         dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
228         try:
229             marker = {
230                 "video_id": video_id,
231                 "returncode": returncode,
232                 "segment_count": segment_count,
233                 "status": status,
234             }
235             local = os.path.join(scratch, "_done.json")
236             with open(local, "w", encoding="utf-8") as f:
237                 json.dump(marker, f)
238             storage.put(local, done_uri, config=storage_cfg)
239             logger.info("[worker] wrote completion marker -> %s", done_uri)
240         except Exception as e:  # never fail a good run because the marker push hiccuped
241             logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
242
243
244     def cmd_infer(args: argparse.Namespace) -> int:
245         config = load_config(args.config) if args.config else load_config()
246         _apply_overrides(config, args.set)
247         if not _run_startup_checks(config):
248             return 2
249
250         # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
251         # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
252         # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
253         # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
254         compute = get_compute_backend(config)
255         if compute is not None:
256             return _dispatch_remote(compute, args)
257
258         storage_cfg = config.storage.model_dump()
259
260         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
261         os.makedirs(scratch, exist_ok=True)
262         config.output.output_dir = scratch
263
264         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
265         local_video = storage.fetch(
266             args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg
267         )
268         print(f"[worker] pulled {args.video_uri} -> {local_video}")
269
270         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
271         video_id = compute_video_id(local_video)
272
273         # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
274         # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate:
it runs the
275         # SAME validators with ITS resolved config and only dispatches on PASS. Re-
validating here is
276         # redundant, decodes the whole video a SECOND time, and is outright WRONG when the
```

```
worker's config
277         # resolves different thresholds than the control-plane's ? observed 2026-07-07: a
5312x2988 clip
278         # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0,
then the worker
279         # REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the
fact.
280         # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote)
skips it; the
281         # direct ``worker infer`` CLI has no control-plane, so it still validates by
default.
282         if getattr(args, "skip_validation", False):
283             print(
284                 "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
285                 "gate and already validated this input."
286             )
287             val_results, val_context = [], {}
288         else:
289             val_results, val_context = validator_registry.run_all_with_context(
290                 local_video, config
291             )
292         failures = [r for r in val_results if not r.passed]
293         db = Database(os.path.join(scratch, config.output.db_name))
294         db.add_video(
295             video_id,
296             local_video,
297             "failed" if failures else "processed",
298             [r.model_dump() for r in val_results],
299         )
300
301         def _fail(rc: int) -> int:
302             # A worker-side FAILURE (validation, unknown segmenter, or a detector
timeout/abort the
303             # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
Write + push
304             # a status="failed" report.json/intervals.csv (0 segments) so the control
plane's bucket
305             # truth (submit-cache probe, restart-recovery) and the alpha-user result surface
see an
306             # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
307             # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote
308             # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
309             try:
310                 _serialize_and_push_outputs(
311                     scratch,
312                     args.out_uri,
313                     video_id,
314                     [],
315                     "failed",
316                     storage_cfg,
317                     str(getattr(args, "video_uri", "") or ""),
318                 )
319             except Exception as e:  # never mask the real failure on an artifact-push hiccup
320                 logger.warning("[worker] could not push failure artifacts: %s", e)
321             # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
322             if getattr(args, "done_uri", None):
323                 _write_done_marker(
324                     args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
325                 )
326             return rc
327
```

```
        segments: List[dict] = []
        segmenter_actual = None
        segmentation_ran = False
        status = "failed"
        try:
            if failures:
                print(
                    "[worker] validation FAILED: "
                    + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
                )
                return _fail(2)
            seg_name = args.segmenter or config.indexing.default_segmenter
            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                print(
                    f"[worker] unknown segmenter: {seg_name!r} "
                    f"(have {list(segmenter_registry.list_available())})"
                )
                return _fail(2)
            segmenter_actual = seg_name
            result = segmenter_cls(db, config).process_video(
                video_id, local_video, max_frames=args.max_frames
            )
            segmentation_ran = True
            if isinstance(result, Failure):
                print(f"[worker] segmenter FAILED: {result.message}")
                return _fail(3)
            segments = result.value if hasattr(result, "value") else result
            status = "success"
            # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
            # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
            # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
            if not segments:
                detector_impl = (
                    os.environ.get("RALLY_DETECTOR_IMPL")
                    or getattr(config.indexing, "detector_impl", None)
                    or "auto"
                )
                logger.warning(
                    "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
                    "The detector substrate yielded no usable trajectories/windows ? check
the detector "
                    "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "
                    "skip_ai_handoff=False (the segmenter bails BEFORE detection ? set the
key or "
                    "indexing.skip_ai_handoff=true; cloud-serving now fails this at
startup), native "
                    "runner UNAVAILABLE (weights/repo/WASB_PYTHON env), the WASB env
produced no "
                    "detections, or the input resolution differs from the tuned proxy
resolution. "
                    "Re-run with -v for the native command + frame counts.",
                    video_id,
                    segmenter_actual,
                    detector_impl,
                )
        finally:
            # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
            emit_indexer_report(
                db=db,
```

```
383                 video_id=video_id,
384                 video_path=local_video,
385                 config=config,
386                 val_results=val_results,
387                 val_context=val_context,
388                 segmenter_requested=args.segmenter,
389                 segmenter_actual=segmenter_actual,
390                 was_reingest=False,
391                 segmentation_ran=segmentation_ran,
392             )
393
394         # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
395         pushed = _serialize_and_push_outputs(
396             scratch,
397             args.out_uri,
398             video_id,
399             segments,
400             status,
401             storage_cfg,
402             str(getattr(args, "video_uri", "") or ""),
403         )
404
405         print(
406             f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):"
407         )
408         for u in pushed:
409             print("   ", u)
410
411         # M6: a remote dispatcher passes --done-uri; write the SUCCESS completion marker so
its bucket-poll
412         # terminates (carries video_id ? the dispatcher's output path + rc=0). Worker
FAILURES (rc 2/3)
413         # write their own marker via _fail() above, so the dispatcher fails FAST either way
(no 30-min hang
414         # on a bad video). DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
415         if getattr(args, "done_uri", None):
416             _write_done_marker(
417                 args.done_uri, video_id, 0, len(segments), status, storage_cfg, scratch
418             )
419         return 0
420
421
422     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
423     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
424     _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
425
426     #: Leading ISO-date prefix on a canonical label name (``labels/<YYYY-MM-
DD>_<name>.rallies.csv``,
427     #: docs/DATA_IN_GCS.md §7.1). It disambiguates recycled GoPro names by authoring date
but is NOT
428     #: part of the clip's join key, so it's stripped when deriving the videos/ stem.
429     _LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")
```

20. assistant (2026-07-07T16:49:09.482Z)

Key finding: the worker's `_fail()` path **does** write a failure marker (rc 2/3), so a *clean* pipeline failure resolves via the marker ? NOT the fast-fail. The rc-5 path only triggers on a crash *before* any marker (argparse abort / uncaught config error). Let me read the dispatcher wiring, config backend construction, and check my local SDK/ADC.

21. user (2026-07-07T16:49:11.510Z)

```
            ~65-min opaque wait the operator gets an immediate, log-pointing failure."""
        from backend.compute import RemoteExecutionFailed
        from backend.infrastructure.execution_policy import PolicyTimeout

        spec = ComputeJobSpec(
            video_uri=args.video_uri,
            out_uri=args.out_uri,
            segmenter=args.segmenter,
            config_overrides=tuple(args.set or ()),
            skip_validation=bool(getattr(args, "skip_validation", False)),
        )
        try:
            result = compute.run_infer(spec)
        except RemoteExecutionFailed as e:
            logger.error(
                "[worker] remote execution terminated without a completion marker: %s", e
            )
            return 5
        except PolicyTimeout as e:
            logger.warning(
                "[worker] remote compute did not complete (no marker within budget): %s", e
            )
            return 4
        logger.info(
            "[worker] remote compute done: video_id=%s rc=%d outputs=%s",
```

22. user (2026-07-07T16:49:11.987Z)

```
      """Pluggable compute layer: in-process (default) + a Cloud Run GPU Job dispatch shim.

      COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is the seam: it returns a
      ``ComputeBackend`` ONLY when a non-local ``deployment.compute_target`` is selected, and
``None``
      otherwise ? ``None`` means "run the worker's existing in-process path" (today's
behaviour, byte
      identical). So the seam is **default-OFF**: with no profile / ``local_gpu`` /
``cpu_mock`` nothing
      changes. Mirrors ``backend/storage``'s lazy factory idiom (cloud SDKs imported only when
needed).
      """

      from __future__ import annotations

      import dataclasses
      import os
      from typing import Any, Optional

      from backend.compute.base import (
          ComputeBackend,
          ComputeJobSpec,
          ComputeResult,
          RemoteExecutionFailed,
      )
      from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy

      __all__ = [
          "ComputeBackend",
          "ComputeJobSpec",
          "ComputeResult",
          "RemoteExecutionFailed",
          "get_compute_backend",
      ]


      def _storage_dict(config: Any) -> dict:
```

```
        storage = getattr(config, "storage", None)
        if storage is None:
            return {}
        if hasattr(storage, "model_dump"):
            return storage.model_dump()  # type: ignore[no-any-return]
        return dict(storage) if isinstance(storage, dict) else {}


    def get_compute_backend(
        config: Any,
        *,
        run_client: Optional[Any] = None,
        policy: ExecutionPolicy = DEFAULT_POLICY,
        token: Optional[str] = None,
        target_override: Optional[str] = None,
    ) -> Optional[ComputeBackend]:
        """Return the compute backend for ``config.deployment.compute_target``, or ``None``
to run
        in-process (the default-OFF path).

        Only ``compute_target == "cloud_run"`` returns a non-None backend (a
``CloudRunCompute``). Its
        Cloud Run coordinates come from the control-plane env (``CLOUD_RUN_JOB`` /
``CLOUD_RUN_REGION``
        / ``GOOGLE_CLOUD_PROJECT``) so no secrets live in config. ``run_client`` is
injectable (tests
        pass a fake; production defaults to the real ``google-cloud-run`` client, owner-
verified on M7).

        ``target_override`` lets a single request force a target (the SPA compute-picker
maps a
        per-request ``compute="cloud"`` to ``target_override="cloud_run"``) without mutating
the
        server's configured default. ``None`` falls back to
``config.deployment.compute_target``.
        """
        target = (
            target_override
            if target_override is not None
            else getattr(getattr(config, "deployment", None), "compute_target", "") or ""
        )
        if target != "cloud_run":
            return None  # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)

        # Lazy import so the cloud SDK shim is only pulled when a cloud target is actually
selected.
        from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient

        # Derive the poll budget from config UNLESS the caller injected an explicit policy ?
        # deployment.remote_poll_max_wait_sec must outlast the worker Job's --task-timeout
so a
        # slow-but-healthy paid GPU run is never abandoned mid-flight (CP-rebuild §4.3; the
old
        # 1800s default undercut real 1080p60 jobs). Identity check on DEFAULT_POLICY keeps
an
        # explicitly-passed policy (tests, callers with bespoke budgets) untouched.
        if policy is DEFAULT_POLICY:
            max_wait = getattr(
                getattr(config, "deployment", None), "remote_poll_max_wait_sec", None
            )
            if max_wait:
                policy = dataclasses.replace(DEFAULT_POLICY, max_wait_sec=float(max_wait))

        job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
        region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")
```

```
87          project = os.environ.get("GOOGLE_CLOUD_PROJECT")
88          client = run_client if run_client is not None else GoogleCloudRunJobClient()
89          return CloudRunCompute(
90              run_client=client,
91              job_name=job_name,
92              region=region,
93              project=project,
94              storage_config=_storage_dict(config),
95              policy=policy,
96              token=token,
97          )
98
```

23. user (2026-07-07T16:49:13.614Z)

```
35:class ComputeJobSpec:
37:    from the SAME input bytes, so the output always lands at
``<out_uri>/outputs/<video_id>/`` ?
40:    video_uri: str  # input (gs://? for cloud; a path/local:// for local)
41:    out_uri: str  # output ROOT (gs://? bucket for cloud)
42:    segmenter: Optional[str] = None
44:    config_overrides: tuple[str, ...] = ()
51:    skip_validation: bool = False
57:    (0 ok / 2 validation-or-config / 3 segmenter-failure), plus **4 = remote dispatch timeout
or an
```

24. user (2026-07-07T16:49:21.599Z)

```
run_v2 OK C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\cloud\run_v2\__init__.py
---ADC---
ya29.a0AT3oNZ-b4ZrdVMspa2sMfdxqz1vp-pjLo
---PYVER---
Python 3.13.13
```

25. assistant (2026-07-07T16:50:41.710Z)

This is very promising. Key facts established:
- `google.cloud.run_v2` imports locally + ADC works ? **I can drive `run_infer` locally with a real `GoogleCloudRunJobClient`, no control-plane / CF-Access JWT needed** (`get_compute_backend` reads `CLOUD_RUN_JOB`/`CLOUD_RUN_REGION`/`GOOGLE_CLOUD_PROJECT` from env).
- `_dispatch_remote` forwards `--set` overrides + `--skip-validation` to the dispatched worker.
- The worker's `_fail()` writes a marker, so I need a *pre-marker hard crash* (argparse abort / uncaught `_apply_overrides` error), not a clean pipeline failure.

Let me set up task tracking and gather the live-infra facts (job image/retries/timeout/command) before dispatching anything.

26. user (2026-07-07T16:50:45.097Z)

```
constructed real client OK: GoogleCloudRunJobClient
classifier importable: _classify_execution_state
```

27. user (2026-07-07T16:50:50.315Z)

```
metadata:
  name: rally-worker
spec:
  template:
    spec:
      template:
        spec:
          containers:
          - image: asia-southeast1-docker.pkg.dev/gen-lang-client-0306026826/rally/rally-
```

```
worker@sha256:8045f54ea9fcbf52426305ab987f122f4fc8bed6b13be93283d21c5cab57a655
          maxRetries: 0
```

28. assistant (2026-07-07T16:51:23.019Z)

maxRetries=0 (terminal-Failed after first crash ? fast) and current live image digest is `sha256:8045f54e?` (matches the deployed guardrail image; I'll record it as the restore baseline). Now I need to confirm that an uncaught `--set` config error in the dispatched worker exits *before* any marker write (not caught+markered by `main()`). Let me read the worker's entry point and arg parser.

29. user (2026-07-07T16:51:24.642Z)

```
429     _LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")
430
431
432     def _label_clip_name(fname: str) -> str:
433         """Derive the clip name (the ``videos/<name>.<ext>`` join key) from a label
filename.
434
435         Strips both the ``.rallies.csv`` / ``.csv`` suffix and an optional leading ISO-date
prefix, so a
436         canonical dated label and a legacy bare label resolve to the SAME bare clip stem:
437
438         * ``2026-06-22_GX020094.rallies.csv`` -> ``GX020094``       (canonical, date-
stamped)
439         * ``GX020094.rallies.csv``            -> ``GX020094``       (legacy bare ? same
stem)
440         * ``GX020094_proxy.rallies.csv``      -> ``GX020094_proxy`` (``_proxy`` is part of
the name, kept)
441         * ``2026-06-21_mahadevpura_1.rallies.csv`` -> ``mahadevpura_1`` (underscores in
<name> survive)
442
443         Without the date strip, ``--trajectory-source detector`` can't match a dated label
to its bare
444         video stem, so every clip is skipped and train aborts with ``<2 videos``
(DATA_IN_GCS §7b #1).
445         """
446         name = fname.replace(".rallies.csv", "").replace(".csv", "")
447         return _LABEL_DATE_PREFIX.sub("", name)
448
449
450     def _probe_video_fps_width(path: str):
451         """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
452
453         The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
454         fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
455         cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
456         inference report (it flags the fallback) but corrupting for REAL training data. So
here we
457         probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
458         rather than guesses.
459         """
460         import json
461         import subprocess
462
463         try:
464             out = subprocess.check_output(
465                 [
466                     ffprobe_bin(),
467                     "-v",
```

```
                    "error",
                    "-select_streams",
                    "v:0",
                    "-show_entries",
                    "stream=r_frame_rate,width",
                    "-of",
                    "json",
                    path,
                ],
                stderr=subprocess.DEVNULL,
            ).decode()
            st = (json.loads(out).get("streams") or [{}])[0]
            num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
            fps = float(num) / float(den or "1")
            width = float(st.get("width") or 0)
            if fps > 0 and width > 0:
                return fps, width
        except (
            subprocess.SubprocessError,
            ValueError,
            KeyError,
            OSError,
            json.JSONDecodeError,
        ):
            pass
        return None


    def _resolve_train_detector(args: argparse.Namespace, config: Any):
        """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
        detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
        stub=CPU mock). Returns the runner, or prints an error + returns None.

        Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
the
        detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
        no shuttle detector and is rejected.
        """
        from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter

        seg_cls = segmenter_registry.get(args.segmenter)
        if not seg_cls:
            print(
                f"[worker] unknown segmenter: {args.segmenter!r} "
                f"(have {list(segmenter_registry.list_available())})"
            )
            return None
        seg = seg_cls(None, config)
        if not isinstance(seg, TrajectoryHybridSegmenter):
            print(
                f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
                f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid / fusion_hybrid)."
            )
            return None
        try:
            runner, _ = seg._resolve_runner(config.indexing)
        except NotImplementedError as e:
            # e.g. detector_impl='native' for a family without a native runner (tracknet, or
            # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
traceback.
            print(f"[worker] detector not available: {e}")
            return None
        ok, msg = runner.healthcheck()
```

```
        if not ok:
            print(f"[worker] detector environment not ready: {msg}")
            return None
        print(f"[worker] detector ready ({args.segmenter}): {msg}")
        return runner


    def cmd_train(args: argparse.Namespace) -> int:
        """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
        to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.

        Two trajectory sources (the train pipeline + harness are identical either way):
          * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
            shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
          * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
            DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
            is the M3.5 "first real training" path; only the trajectory source flips.
        """
        import subprocess

        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        if not _run_startup_checks(config):
            return 2
        storage_cfg = config.storage.model_dump()

        scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
        labels_dir = os.path.join(scratch, "labels")
        traj_dir = os.path.join(scratch, "trajectories")
        videos_dir = os.path.join(scratch, "videos")
        os.makedirs(labels_dir, exist_ok=True)
        os.makedirs(traj_dir, exist_ok=True)

        corpus = args.corpus_uri.rstrip("/")
        label_uris = sorted(
            u
            for u in storage.list_uris(f"{corpus}/labels/", config=storage_cfg)
            if u.lower().endswith(".csv")
        )
        if len(label_uris) < 2:
            print(
                f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
                f"under {corpus}/labels/"
            )
            return 2

        # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
        runner = None
        video_by_stem: dict = {}
        if args.trajectory_source == "detector":
            os.makedirs(videos_dir, exist_ok=True)
            runner = _resolve_train_detector(args, config)
            if runner is None:
                return 2
            for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
                vname = storage.parse_uri(vuri).name
                if not vname.lower().endswith(_VIDEO_EXTS):
                    continue  # skip sidecars (.srt/.json/thumbnails) so they can't shadow
```

```
the video
587                 video_by_stem[os.path.splitext(vname)[0]] = vuri
588
589         print(f"[worker] trajectory source: {args.trajectory_source}")
590         specs: List[dict] = []
591         for uri in label_uris:
592             fname = storage.parse_uri(uri).name  # e.g. 2026-06-22_GX020094.rallies.csv
593             name = _label_clip_name(fname)  # -> GX020094 (date prefix stripped)
594             local_label = storage.fetch(
595                 uri, os.path.join(labels_dir, fname), config=storage_cfg
596             )
597             if args.trajectory_source == "detector":
598                 vuri = video_by_stem.get(name)
599                 if not vuri:
600                     print(
601                         f"[worker] no video for {name!r} under {corpus}/videos/ ? skipping."
602                     )
603                     continue
604                 local_video = storage.fetch(
605                     vuri,
606                     os.path.join(videos_dir, storage.parse_uri(vuri).name),
607                     config=storage_cfg,
608                 )
609                 video_id = compute_video_id(local_video)
610                 # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
611                 # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
612                 meta = _probe_video_fps_width(local_video)
613                 if meta is None:
614                     print(
615                         f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess)."
616                     )
617                     continue
618                 fps, frame_width = meta
619                 traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
620                 if not traj:
621                     print(
622                         f"[worker] detector produced no trajectory for {name!r} ? skipping."
623                     )
624                     continue
625                 specs.append(
626                     {
627                         "name": name,
628                         "trajectory": traj,
```

30. user (2026-07-07T16:51:30.509Z)

```
240:    except Exception as e:  # never fail a good run because the marker push hiccuped
319:        except Exception as e:  # never mask the real failure on an artifact-push hiccup
707:def build_parser() -> argparse.ArgumentParser:
708:    p = argparse.ArgumentParser(
809:    pt.add_argument("--set", action="append", default=[], metavar="k=v")
831:def main(argv=None) -> int:
838:    sys.exit(main())
```

31. user (2026-07-07T16:51:33.674Z)

```
707     def build_parser() -> argparse.ArgumentParser:
708         p = argparse.ArgumentParser(
709             prog="python -m backend.worker",
710             description="Storage-aware worker: pull ? run pipeline ? push.",
711         )
712         p.add_argument(
```

```
713             "-v",
714             "--verbose",
715             action="store_true",
716             help="DEBUG logging (the native detector command, frame counts, per-stage
detail)",
717         )
718         sub = p.add_subparsers(dest="cmd", required=True)
719
720         pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
721         pi.add_argument(
722             "--video-uri", required=True, help="local path / local:// / s3:// / gcs://"
723         )
724         pi.add_argument(
725             "--out-uri",
726             required=True,
727             help="output prefix (outputs/<video_id>/? is appended)",
728         )
729         pi.add_argument(
730             "--segmenter", default=None, help="default: indexing.default_segmenter"
731         )
732         pi.add_argument(
733             "--config", default=None, help="config.json path (default: ./config.json)"
734         )
735         pi.add_argument(
736             "--scratch", default=None, help="local scratch dir (default: a temp dir)"
737         )
738         pi.add_argument("--max-frames", type=int, default=None)
739         pi.add_argument(
740             "--done-uri",
741             default=None,
742             help="M6: storage URI for a completion MARKER (video_id + rc) written on a "
743             "successful run, so a remote (Cloud Run) dispatcher's bucket-poll terminates. "
744             "Default-OFF: omit it for the normal local run.",
745         )
746         pi.add_argument(
747             "--set",
748             action="append",
749             default=[],
750             metavar="k=v",
751             help="config override, e.g. --set indexing.skip_ai_handoff=true",
752         )
753         pi.add_argument(
754             "--skip-validation",
755             action="store_true",
756             help="skip the worker's re-validation: the dispatcher (control-plane) already
ran the "
757             "validators with its resolved config and only dispatches on PASS, so re-
validating here is "
758             "redundant, decodes the video again, and can contradict the gate. Default-OFF:
the direct "
759             "`worker infer` CLI still validates.",
760         )
761         pi.set_defaults(func=cmd_infer)
762
763         pt = sub.add_parser(
764             "train",
765             help="Gen-0 train-from-bucket (labels [+videos] -> trajectories -> harness)",
766         )
767         pt.add_argument(
768             "--corpus-uri",
769             required=True,
770             help="prefix containing labels/ (+ videos/ for "
771             "--trajectory-source detector), e.g. s3://bucket or gcs://bucket",
772         )
773         pt.add_argument(
```

```
            "--out-uri",
            required=True,
            help="output prefix (weights/<version>/ is appended)",
        )
        pt.add_argument("--version", required=True, help="semver for the artifact bundle")
        pt.add_argument(
            "--trajectory-source",
            choices=["synth", "detector"],
            default="synth",
            help="synth = CPU-mock label-consistent trajectories (default, no GPU); "
            "detector = REAL shuttle trajectories via the resolved DetectorRunner "
            "(needs videos/ in the corpus; honours detector_impl).",
        )
        pt.add_argument(
            "--segmenter",
            default="wasb_hybrid",
            help="trajectory hybrid whose detector is used for --trajectory-source
detector",
        )
        pt.add_argument(
            "--floor", default=None, help="floor baseline JSON (local path or storage URI)"
        )
        pt.add_argument("--config", default=None)
        pt.add_argument("--scratch", default=None)
        pt.add_argument(
            "--fps",
            type=float,
            default=30.0,
            help="synth trajectory fps (detector probes the video)",
        )
        pt.add_argument(
            "--frame-width",
            type=float,
            default=1280.0,
            help="synth trajectory frame width (detector probes the video)",
        )
        pt.add_argument("--set", action="append", default=[], metavar="k=v")
        pt.set_defaults(func=cmd_train)
        return p


    def _setup_logging(verbose: bool) -> None:
        """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
diagnostics are
        VISIBLE ? the worker previously ran with no logging config, so every
segmenter/detector log
        (detector choice, 'Running native WASB inference', availability failures) was
silently dropped.
        That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
(the CLI
        entry), not at import, so importing the module never reconfigures a caller's
logging."""
        # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
library
        # configured logging at import), which would leave the level at WARNING and re-
suppress the
        # diagnostics. Forcing reconfiguration means the worker reliably controls its own
verbosity.
        logging.basicConfig(
            level=logging.DEBUG if verbose else logging.INFO,
            format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
            datefmt="%H:%M:%S",
            force=True,
        )

```

```
830
831     def main(argv=None) -> int:
832         args = build_parser().parse_args(argv)
833         _setup_logging(getattr(args, "verbose", False))
834         return args.func(args)
835
836
837     if __name__ == "__main__":
838         sys.exit(main())
```

32. assistant (2026-07-07T16:52:53.744Z)

`main()` runs `args.func(args)` with **no try/except**, so an uncaught error in `cmd_infer` ? `_apply_overrides` (e.g. a `--set` with a nonexistent config section ? `AttributeError`) crashes the worker **before** any marker write, on the current image, no image pinning needed. That's my crash recipe.

Let me nail down the poll policy defaults and the cloud-serving storage/bucket config.

33. user (2026-07-07T16:52:54.901Z)

```
1       """Per-job execution policy: bounded, observable waits for long/flaky ops
(EXECUTION_POLICY.md P1).
2
3       Turns an unbounded blocking call (the GX010125 Gemini hang) into a bounded one:
4       - ``run_bounded`` ? a HARD per-attempt timeout (``op_timeout_sec``, the hang-killer) +
retry-with-backoff
5         for FAILURES (an exception) *and* HANGS (no return in time). A hung attempt's worker
thread is a daemon,
6         so it is abandoned, not joined ? it never blocks the process.
7       - ``poll_until`` ? a bounded poll loop for an in-progress external op
(``poll_every_n_sec`` cadence,
8         ``max_wait_sec`` deadline).
9
10      **Testability:** ``sleep``/``monotonic`` are injected, so the whole module is unit-
tested with zero real
11      waiting (a fake clock advances instantly).
12
13      **Logging discipline** (so polling is VISIBLE but never NOISY): every poll/retry logs at
**DEBUG**; state
14      transitions (start, resolved, hung, gave-up) log at **INFO/WARNING**. INFO stays clean;
enable DEBUG on the
15      ``execution.policy`` logger to watch the poll trail.
16      """
17
18      from __future__ import annotations
19
20      import logging
21      import threading
22      import time as _time
23      from dataclasses import asdict, dataclass
24      from enum import Enum
25      from typing import Callable, Optional, TypeVar
26
27      T = TypeVar("T")
28
29      #: Dedicated logger so callers can dial polling visibility independently of the app's
root level.
30      LOG = logging.getLogger("execution.policy")
31
32
33      class WaitMode(str, Enum):
34          """How a job treats a long wait. ``str``-valued so it serialises straight to JSON on
the job row."""
35
```

```
36          WAIT_AND_RESUME = "wait_and_resume"  # default: bounded wait; over budget ->
reschedule/partial (P2/P3)
37          ABORT = "abort"  # one attempt; on hang/fail, stop immediately
38          BLOCK = "block"  # bounded in-process blocking wait (no reschedule)
39
40
41      class PolicyTimeout(TimeoutError):
42          """An op hung past ``op_timeout_sec`` or never became ready within
``max_wait_sec``."""
43
44
45      @dataclass(frozen=True)
46      class ExecutionPolicy:
47          """Declarative resilience knobs, attachable per job/op (JSON-serialisable; see
EXECUTION_POLICY.md)."""
48
49          mode: WaitMode = WaitMode.WAIT_AND_RESUME
50          poll_every_n_sec: float = 10.0
51          op_timeout_sec: float = (
52              120.0  # HARD per-attempt deadline ? the hang-killer that was missing
53          )
54          max_wait_sec: float = 1800.0  # total wall budget for a poll loop
55          max_retries: int = 5  # retries (=> max_retries+1 attempts) for failures/hangs
56          backoff_base_sec: float = 3.0  # exponential: backoff_base * 2**attempt
57          on_timeout: str = "reschedule"  # reschedule | partial | fail  (consumed by P2/P3)
58          resumable: bool = True
59
60          def to_dict(self) -> dict:
61              d = asdict(self)
62              d["mode"] = self.mode.value
63              return d
64
65          @classmethod
66          def from_dict(cls, d: dict) -> "ExecutionPolicy":
67              known = {f for f in cls.__dataclass_fields__}  # type: ignore[attr-defined]
68              kw = {k: v for k, v in d.items() if k in known}
69              if "mode" in kw and not isinstance(kw["mode"], WaitMode):
70                  kw["mode"] = WaitMode(kw["mode"])
71              return cls(**kw)
72
73
74      #: The sane default (also what a job gets if it declares no policy).
75      DEFAULT_POLICY = ExecutionPolicy()
76
77
78      def _backoff_sec(policy: ExecutionPolicy, attempt: int) -> float:
79          """Deterministic exponential backoff (no jitter ? reproducible tests)."""
80          return policy.backoff_base_sec * (2**attempt)
81
82
83      def run_bounded(
84          fn: Callable[[], T],
85          policy: ExecutionPolicy = DEFAULT_POLICY,
86          *,
87          label: str = "op",
88          logger: Optional[logging.Logger] = None,
89          sleep: Callable[[float], None] = _time.sleep,
90      ) -> T:
91          """Run ``fn()`` under a hard per-attempt timeout, retrying FAILURES and HANGS with
backoff.
92
93          Returns ``fn``'s value on success. Raises ``PolicyTimeout`` if the final attempt
hung, or re-raises the
94          last exception if the final attempt failed. ``mode=ABORT`` => a single attempt (no
retries).
```

```
        """
        log = logger or LOG
        attempts = 1 if policy.mode is WaitMode.ABORT else policy.max_retries + 1
        last_exc: Optional[BaseException] = None
        for attempt in range(attempts):
            box: dict = {}
            done = threading.Event()

            # box/done bound as defaults (not closed over) ? the thread is started + joined
within this
            # same iteration, but explicit binding keeps each attempt isolated and silences
B023.
            def _runner(_box: dict = box, _done: threading.Event = done) -> None:
                try:
                    _box["val"] = fn()
                except (
                    BaseException
                ) as e:  # relay ANY error (incl. timeouts) to the caller thread
                    _box["exc"] = e
                finally:
                    _done.set()

            threading.Thread(target=_runner, name=f"bounded:{label}", daemon=True).start()
            if done.wait(timeout=policy.op_timeout_sec):
                if "exc" in box:
                    last_exc = box["exc"]
                    log.info(
                        "%s: attempt %d/%d failed: %s",
                        label,
                        attempt + 1,
                        attempts,
                        last_exc,
                    )
                else:
                    if attempt:
                        log.info(
                            "%s: succeeded on attempt %d/%d", label, attempt + 1, attempts
                        )
                    return box["val"]  # type: ignore[return-value]
            else:
                last_exc = PolicyTimeout(
                    f"{label} hung > op_timeout_sec={policy.op_timeout_sec}s"
                )
                log.warning(
                    "%s: attempt %d/%d HUNG (>%.0fs) ? abandoning",
                    label,
                    attempt + 1,
                    attempts,
                    policy.op_timeout_sec,
                )
            if attempt < attempts - 1:
                wait = _backoff_sec(policy, attempt)
                log.debug("%s: backing off %.1fs before retry", label, wait)
                sleep(wait)
        assert last_exc is not None
        log.warning("%s: gave up after %d attempt(s): %s", label, attempts, last_exc)
        raise last_exc


    def poll_until(
        check: Callable[[], Optional[T]],
        policy: ExecutionPolicy = DEFAULT_POLICY,
        *,
        label: str = "op",
        logger: Optional[logging.Logger] = None,
```

```
158         sleep: Callable[[float], None] = _time.sleep,
159         monotonic: Callable[[], float] = _time.monotonic,
160         on_tick: Optional[Callable[[float], None]] = None,
161     ) -> T:
162         """Poll ``check()`` every ``poll_every_n_sec`` until it returns a non-``None``
value, or raise
163         ``PolicyTimeout`` after ``max_wait_sec``. ``check`` returns ``None`` while the op is
still in progress.
164
165         Polls log at DEBUG (no INFO spam); the wait start + resolution + give-up log at
INFO/WARNING.
166         ``on_tick(elapsed_sec)`` (optional) fires once per still-waiting iteration ? the
caller's live
167         progress heartbeat (#482). Best-effort by contract: a raising callback is swallowed
168         (DEBUG-logged), never allowed to kill the wait it reports on.
169         """
170         log = logger or LOG
171         start = monotonic()
172         polls = 0
173         log.info(
174             "%s: waiting (poll_every=%.0fs, max_wait=%.0fs)",
175             label,
176             policy.poll_every_n_sec,
177             policy.max_wait_sec,
178         )
179         while True:
180             result = check()
181             if result is not None:
182                 log.info(
183                     "%s: ready after %d poll(s) / %.0fs", label, polls, monotonic() - start
184                 )
185                 return result
186             elapsed = monotonic() - start
187             if elapsed >= policy.max_wait_sec:
188                 log.warning(
189                     "%s: still not ready after %.0fs (max_wait) ? giving up", label, elapsed
190                 )
191                 raise PolicyTimeout(
192                     f"{label} not ready within max_wait_sec={policy.max_wait_sec}s"
193                 )
194             polls += 1
195             log.debug("%s: poll %d, %.0fs elapsed, not ready", label, polls, elapsed)
196             if on_tick is not None:
197                 try:
198                     on_tick(elapsed)
199                 except Exception:  # noqa: BLE001 ? progress reporting must never kill the
wait
200                     log.debug("%s: on_tick callback failed", label, exc_info=True)
201             sleep(policy.poll_every_n_sec)
202
```

34. assistant (2026-07-07T16:53:03.191Z)

Policy defaults: `poll_every_n_sec=10s`, `max_wait_sec=1800s` (overridable to ~65min via `deployment.remote_poll_max_wait_sec`). So after the execution goes terminal, the fast-fail fires within ?10s (one poll). Now let me find the cloud-serving storage bucket and compute config.

35. user (2026-07-07T16:53:05.780Z)

```
28:COMPUTE_TARGETS: tuple[str, ...] = ("local_gpu", "cloud_run", "cpu_mock")
30:# Each profile maps 1:1 to its canonical compute_target. Used to apply the preset and to warn
31:# when config.json explicitly overrides compute_target into a state inconsistent with the
profile.
34:    "cloud-serving": "cloud_run",
```

```
39:# that EXIST today ? M1 is pure/additive: the new ``deployment.{profile,compute_target}``
plus
45:        "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
52:        "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
75:        "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
```

36. user (2026-07-07T16:53:08.711Z)

```
39      # that EXIST today ? M1 is pure/additive: the new
``deployment.{profile,compute_target}`` plus
40      # ``indexing.{detector_impl,skip_ai_handoff}`` and ``storage.backend``. Later milestones
extend
41      # these bundles ? ``input_backend`` (M2) and the configurable output/workspace base (M3,
42      # Launch-Report-gated) ? deliberately NOT set here so M1 never pre-empts a gated flip.
43      PROFILE_PRESETS: dict[str, dict[str, Any]] = {
44          "truly-local": {
45              "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
46              # detector_impl stays "auto" (the runner picks WSL vs native by env on the GPU
box).
47              # Zero-cloud / privacy-first ? run fully local with no Gemini handoff.
48              "indexing": {"skip_ai_handoff": True},
49              "storage": {"backend": "local"},
50          },
51          "cloud-serving": {
52              "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
53              # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172 floor
(D11).
54              # wasb GPU-feed tuning: the parity defaults (batch 8, no prefetch) left the paid
L4
55              # <1% utilized on the first real serving run (2026-07-03) ? the pipeline was
56              # CPU-feed-bound. Serving opts into the measured fast values; local/WSL
unchanged.
57              "indexing": {
58                  "detector_impl": "native",
59                  "skip_ai_handoff": False,
60                  # GPU-resident NVDEC decode (#506/#507): validated on a real L4 (golden gate
PASS,
61                  # F1 0.574 ? baseline 0.582 @ SERVED) AND on the live Cloud Run L4 (the
served
62                  # trajectory reproduced the VM run ? 26479 ? 26481 visible on GX010128),
then rolled
63                  # out + canaried (DEPLOY_REPORT_cloud-serving_gpu-resident-
nvdec_2026-07-07). This makes
64                  # nvdec the code-level serving default so it survives an image rebuild; the
worker's
65                  # WASB_DECODE_BACKEND env was the reversible rollout lever. cuda-only
(validator-enforced).
66                  "wasb": {
67                      "batch_size": 64,
68                      "prefetch_batches": 4,
69                      "decode_backend": "nvdec_resident",
70                  },
71              },
72              "storage": {"backend": "gcs"},
73          },
74          "cpu-mock": {
75              "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
76              # Deterministic CPU mock ? no GPU/WSL, no provider/API. The CI / regression
profile.
77              "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
78              "storage": {"backend": "local"},
79          },
80      }
81
82
```

```
83      def is_known_profile(profile: str) -> bool:
84          """True for a non-empty, recognised profile name."""
85          return bool(profile) and profile in PROFILE_PRESETS
86
87
88      def preset_for(profile: str) -> dict[str, Any]:
89          """Return a deep COPY of the preset bundle for ``profile`` (so callers can't mutate
the
90          module-level table), or ``{}`` for ``""`` / an unknown profile."""
91          return copy.deepcopy(PROFILE_PRESETS.get(profile, {}))
92
93
94      def _truthy(val: Optional[str]) -> bool:
95          """Common env-var truthiness (``1/true/yes/on``, case/space-insensitive)."""
96          if val is None:
97              return False
98          return val.strip().lower() in {"1", "true", "yes", "on"}
99
100
101     def _cuda_available() -> bool:
102         """Best-effort CUDA probe. torch is heavy + optional, so it is imported lazily here
and
103         ONLY when a caller opts into ``probe_cuda``; any import/runtime failure means 'no
GPU'."""
104         try:
105             # Intentionally lazy + local: keeps the config-load path torch-free (torch is
heavy
106             # and optional). Only reached when a caller opts into probe_cuda.
107             import torch
108
109             return bool(torch.cuda.is_available())
110         except Exception:
111             return False
112
113
```

37. assistant (2026-07-07T16:53:38.158Z)

Storage backend for cloud-serving is `gcs`; the bucket comes from the `gs://` URI I supply. Let me (a) verify my crash recipe locally against the real config, (b) confirm the gcs storage backend reads a real bucket path with ADC, and (c) list available buckets ? all read-only/local.

38. user (2026-07-07T16:53:40.349Z)

```
backend\storage\gdrive.py:18:``gcs://`` / ``s3://``, or ``gdown <file-id>`` to a local path.
This backend intentionally does NOT
backend\storage\gdrive.py:89:class GDriveStorage(StorageBackend):
backend\storage\gdrive.py:123:    def fetch_to_local(self, ref, dst=None, *, overwrite=False) ->
str:
backend\storage\gdrive.py:135:    def exists(self, ref) -> bool:
backend\storage\ref.py:8:    gcs://<bucket>/<key>        -> gcs      (gs:// accepted, normalized
to gcs://)
backend\storage\ref.py:27:class StorageStat:
backend\storage\ref.py:42:class StorageRef:
backend\storage\ref.py:56:def parse_uri(uri: str) -> StorageRef:
backend\storage\ref.py:59:    A string with no ``scheme://`` prefix (including Windows
``C:\\...`` paths, which
backend\storage\ref.py:66:    scheme = m.group(1).lower()
backend\storage\ref.py:68:    if scheme == "gs":
backend\storage\ref.py:69:        scheme = "gcs"
backend\storage\ref.py:70:    if scheme in ("local", "gdrive", "gdrive-api"):
backend\storage\ref.py:73:        return StorageRef(scheme, "", rest, s)
backend\storage\ref.py:74:    if scheme in _KNOWN:
backend\storage\ref.py:76:        return StorageRef(scheme, bucket, key, s)
backend\storage\ref.py:77:    raise ValueError(f"unsupported storage URI scheme: {scheme}:// (in
```

```
{s!r})")
backend\storage\ref.py:86:      1. An explicit ``scheme://`` on ``raw`` (``gs://``, ``s3://``,
``local://``, ``gdrive://``) ? as-is.
backend\storage\ref.py:89:          (``"gs://bucket/videos"`` + ``"x.mp4"`` ?
``"gs://bucket/videos/x.mp4"``).
backend\storage\ref.py:91:          ``"gcs://bucket/x"``).
backend\storage\ref.py:98:    if _SCHEME_RE.match(s):  # 1. explicit scheme wins
backend\storage\local.py:19:class LocalStorage(StorageBackend):
backend\storage\local.py:30:    def fetch_to_local(self, ref, dst=None, *, overwrite=False) ->
str:
backend\storage\local.py:50:    def exists(self, ref) -> bool:
backend\storage\gcs.py:37:class GCSStorage(StorageBackend):
backend\storage\gcs.py:56:    def fetch_to_local(self, ref, dst=None, *, overwrite=False) ->
str:
backend\storage\gcs.py:72:    def exists(self, ref) -> bool:
backend\storage\gcs.py:89:                "gcs", prefix.bucket, blob.name,
f"gcs://{prefix.bucket}/{blob.name}"
backend\storage\s3.py:22:class S3Storage(StorageBackend):
backend\storage\s3.py:47:    def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
backend\storage\s3.py:62:    def exists(self, ref) -> bool:
backend\storage\capabilities.py:68:    """Does the storage config name THIS bucket backend? (No
secrets ? only scheme/selector hints.)"""
backend\storage\capabilities.py:73:    schemes = ("gs://", "gcs://") if name == "gcs" else
(f"{name}://",)
backend\storage\capabilities.py:79:    return any(r.startswith(schemes) for r in roots)
backend\storage\__init__.py:55:    config (``input_backend``/``input_root``). Raises
``ValueError`` for an unsupported URI scheme ?
backend\storage\__init__.py:72:    that must be fetched first. Raises ``ValueError`` for an
unsupported scheme (same as
backend\storage\__init__.py:74:    return resolve_input_ref(raw, storage_config).backend ==
"local"
backend\storage\__init__.py:83:    byte-for-byte). A bucket / Drive source
(``gs://``/``s3://``/``gdrive://``, or a bare name under a
backend\storage\__init__.py:89:    if ref.backend == "local":
backend\storage\__init__.py:149:def get_storage(
backend\storage\__init__.py:170:def fetch(
backend\storage\__init__.py:200:def exists(uri: str, *, config: Optional[Dict[str, Any]] = None)
-> bool:
backend\storage\gdrive_api.py:51:class GDriveApiStorage(StorageBackend):
backend\storage\gdrive_api.py:150:    def fetch_to_local(self, ref, dst=None, *,
overwrite=False) -> str:
backend\storage\gdrive_api.py:209:    def exists(self, ref) -> bool:
backend\storage\base.py:18:class StorageBackend(ABC):
backend\storage\base.py:23:    def fetch_to_local(
backend\storage\base.py:44:    def exists(self, ref: StorageRef) -> bool: ...
```

39. assistant (2026-07-07T16:53:48.375Z)

`gs://` normalizes to the `gcs` backend. Let me verify my crash recipe against the real config object and enumerate buckets/objects (for marker-poll reads and a small test video).

40. user (2026-07-07T16:53:55.441Z)

```
config loaded: AppConfig
CRASH as expected: AttributeError -> 'AppConfig' object has no attribute 'zzz_smoke_crash'
```

41. user (2026-07-07T16:53:59.599Z)

```
gen-lang-client-0306026826_cloudbuild
khelsutra-rally-corpus
khelsutra-rally-nightly
khelsutra-rally-serving
```

42. assistant (2026-07-07T16:54:32.770Z)

Crash recipe confirmed against the real `AppConfig`. Buckets: `khelsutra-rally-{corpus,nightly,serving}`. Now let me find where the serving dispatch writes (out-uri bucket + worker service account) and a small test video for the healthy run.

43. user (2026-07-07T16:54:35.471Z)

```
48-# so that raising the drain's worker count (to overlap I/O-bound REMOTE Cloud Run dispatch/poll) never
49-# accidentally parallelizes local work. A single self-hosted box serializes GPU work and Gemini is
50:# rate-fragile; a remote dispatch is a bucket-poll and never holds this lane, so N remote jobs overlap
51-# while local/compile jobs stay 1-at-a-time. It's a plain mutex (no nesting) ? no deadlock; when the
52-# drain is serial (max_workers=1) it's an uncontended no-op = today's behaviour.
--
109-    db, video_id: str, outputs_uri: str, storage_config: dict, *, max_rows: int = 20000
110-) -> int:
111:    """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from the bucket and
112-    persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters call ? so
113-    ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they were produced.
--
157-
158-# Trajectory segmenters whose IN-PROCESS detect needs model weights present on THIS box. A hosted
159:# cloud-serving control-plane bakes NO detector weights into its image (only the worker Cloud Run Job
160-# GCSFuse-mounts them), so forcing compute="local" for one of these there can never succeed ? the
161-# guardrail in _maybe_dispatch_remote fails such a request FAST instead of the cryptic deep
--
192-
193-    True when the control plane is wired for Cloud Run: either the configured default target is
194:    already ``cloud_run`` (the operator chose cloud serving), or the Cloud Run env coordinates
195:    (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT``) and an output bucket (``storage.output_root``)
196-    are present so a forced per-request override can actually dispatch + collect a result.
197-
--
259-
260-
261:def _read_bucket_json(uri: str, storage_config: dict) -> Optional[Dict[str, Any]]:
262-    """Fetch + parse one small JSON object (cloud backends require a real dst ? the
263-    ``_ingest_remote_segments`` pattern). Raises on fetch/parse errors; callers decide."""
--
286-
287-def probe_process_cache(db, cfg, video_id: str) -> bool:
288:    """READ-ONLY: does completed detection for ``video_id`` already exist (DB or bucket)?
289-
290-    Safe to call before the atomic submit claim ? it writes nothing (#492 review blocker 1
--
309-        if not storage.exists(report_uri, config=sc):
310-            return False
311:        report = _read_bucket_json(report_uri, sc)
312-        return bool(report and report.get("status") == "success")
313-    except Exception:  # noqa: BLE001 ? a failed probe degrades to the guarded dispatch path
--
322-) -> Optional[Dict[str, Any]]:
323-    """A ready-to-serve /api/process result when this video's detection ALREADY exists ?
324:    the DB first, then the bucket (``outputs/<md5>/report.json`` with ``status=success``
```

```
325:    re-ingests via ``_ingest_remote_segments``): **the DB is a cache, the bucket is the
326-    source of truth** (#484 instant re-runs; the #485 re-hydrate mechanism). ``None`` ? no
327:    completed work found (or the check failed) ? the caller dispatches normally. The bucket
328-    branch mirrors the real cloud ingest exactly (watermarks + ``_finalize_reingest``), so a
329-    partial earlier ingest is replaced, never double-counted.
--
355-            if not storage.exists(report_uri, config=sc):
356-                return None
357:            report = _read_bucket_json(report_uri, sc)
358-            if not report or report.get("status") != "success":
359-                return None
--
364-                # recover, and the next same-md5 submit re-binds the truthful path via
365-                # add_video's upsert.
```

[Showing results with pagination = limit: 60]

44. user (2026-07-07T16:54:40.576Z)

```
600         sk = getattr(idx_cfg, "skip_ai_handoff", None)
601         if sk is not None:
602             overrides.append(f"indexing.skip_ai_handoff={'true' if sk else 'false'}")
603         # Forward the resolved WASB serving knobs so the PRESET governs the worker
(#506/#507). The
604         # worker Job has no DEPLOYMENT_PROFILE, so it does NOT apply the cloud-serving
preset itself ?
605         # without forwarding, its decode_backend + GPU-feed tuning silently fall back to the
model
606         # defaults (cpu, batch 8) regardless of what the control-plane resolved.
decode_backend is
607         # cuda-only (validator-enforced) and requires a worker image that understands
--decode-backend
608         # (? #507); batch_size/prefetch_batches are forwarded only when set (None = the
parity default).
609         wasb_cfg = getattr(idx_cfg, "wasb", None)
610         if wasb_cfg is not None:
611             decode_backend = getattr(wasb_cfg, "decode_backend", None)
612             if decode_backend:
613                 overrides.append(f"indexing.wasb.decode_backend={decode_backend}")
614             batch_size = getattr(wasb_cfg, "batch_size", None)
615             if batch_size:
616                 overrides.append(f"indexing.wasb.batch_size={batch_size}")
617             prefetch = getattr(wasb_cfg, "prefetch_batches", None)
618             if prefetch:
619                 overrides.append(f"indexing.wasb.prefetch_batches={prefetch}")
620         # Tell the worker to SKIP its redundant re-validation: THIS control-plane is the
authoritative
621         # gate ? it already ran the validators with its resolved config just above (only a
PASS reaches
622         # here; a failure returned "failed" before dispatch). A worker re-run would decode
the video a
623         # second time and, because its config isn't fully forwarded, can resolve DIFFERENT
thresholds and
624         # REJECT a clip this gate already passed (observed 2026-07-07: passed at
min_blur_threshold=8.0,
625         # then worker-rejected at its default 20.0 after a successful GPU dispatch).
626         spec = ComputeJobSpec(
627             video_uri=req.video_path,
628             out_uri=out_root,
629             segmenter=seg_name,
630             config_overrides=tuple(overrides),
631             skip_validation=True,
632         )
633         try:
634             # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
```

```
635             # heartbeat (#482): job.progress moves on every poll tick instead of freezing at
636             # "segmenting" for the whole GPU run (the 2026-07-05 silent 22-min spinner) ?
which
637             # also gives the SPA a stall signal to time out on, instead of a blind wall
clock.
638             result = compute.run_infer(
639                 spec,
640                 on_wait=lambda elapsed: notify(_wait_progress_message(seg_name, elapsed)),
641             )
642         except Exception as e:  # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
643             logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
644             return _failed(f"cloud dispatch error: {e}", True)
```

45. user (2026-07-07T16:54:45.933Z)

```
193-    True when the control plane is wired for Cloud Run: either the configured default target
is
194-    already ``cloud_run`` (the operator chose cloud serving), or the Cloud Run env
coordinates
195:    (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT``) and an output bucket
(``storage.output_root``)
196-    are present so a forced per-request override can actually dispatch + collect a result.
197-
--
203-        os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
204-    )
205:    out_root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
206-    return bool(has_job and out_root)
207-
--
303-        if not sc or (sc.get("backend") or "local") == "local":
304-            return False
305:        output_root = (sc.get("output_root") or "").rstrip("/")
306:        if not output_root:
307-            return False
308:        report_uri = f"{output_root}/outputs/{video_id}/report.json"
309-        if not storage.exists(report_uri, config=sc):
310-            return False
--
348-            if not sc or (sc.get("backend") or "local") == "local":
349-                return None
350:            output_root = (sc.get("output_root") or "").rstrip("/")
351:            if not output_root:
352-                return None
353:            outputs_uri = f"{output_root}/outputs/{video_id}"
354-            report_uri = f"{outputs_uri}/report.json"
355-            if not storage.exists(report_uri, config=sc):
--
365-                # add_video's upsert.
366-                video_path = str(report.get("video_uri") or "") or _first_object_under(
367:                    f"{output_root}/videos/{video_id}/", sc
368-                )
369-            play_wm, cand_wm = db.segment_watermarks(video_id)
--
421-
422-
423:def _maybe_dispatch_remote(
424-    cfg,
425-    db,
--
578-            video_id, staged_uri, "processed", [r.model_dump() for r in val_results]
579-        )
580:    out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
581-    if not out_root:
```

```
582-        return _failed(
583:            "cloud-serving needs storage.output_root set to a bucket (e.g. gs://your-bucket)
"
584-            "? that's where the Cloud Run job writes the result.",
585-            False,
--
1034-    """The bucket URI a compiled reel is mirrored to for a direct signed-URL download
(#463), or "" when
1035-    the output isn't a cloud bucket (truly-local ? serve the local file). Shared by the
compile
1036:    mirror-upload and the /api/highlights redirect:
``<output_root>/highlights/<video_id>/<name>``."""
1037:    root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
1038-    if not root or "://" not in root:
1039-        return ""
```

46. assistant (2026-07-07T16:54:55.554Z)

out-uri = `storage.output_root`. Let me read the deployed control-plane + worker env (to mirror the real output bucket the worker SA can write, and confirm the region env), and enumerate small test videos ? in parallel.

47. user (2026-07-07T16:55:10.651Z)

```
=== CONTROL-PLANE env (storage/output/region) ===
 'value': 'cloud-serving'};{'name': 'CLOUD_RUN_REGION'
 'value': 'asia-southeast1'};{'name': 'GOOGLE_CLOUD_PROJECT'
=== WORKER job SA + region/env ===
492532266377-compute@developer.gserviceaccount.com
 'value': '/gcs/models/wasb_badminton_best.pth.tar'};{'name': 'WASB_DECODE_BACKEND'
```

48. user (2026-07-07T16:55:14.307Z)

```
=== corpus/videos (name + size MiB) ===
     0.0 MiB  (122.49GiB)
    86.0 MiB  gs://khelsutra-rally-corpus/videos/mahadevpura_smoke_180s.mp4
   187.1 MiB  gs://khelsutra-rally-corpus/videos/golden/mahadevpura_singles.mp4
   523.4 MiB  gs://khelsutra-rally-corpus/videos/mahadevpura_1.mp4
   523.4 MiB  gs://khelsutra-rally-corpus/videos/mahadevpura-1.MP4
   603.6 MiB  gs://khelsutra-rally-corpus/videos/GX010143_1080p.mp4
   629.0 MiB  gs://khelsutra-rally-corpus/videos/GX010147_1080p.mp4
   659.1 MiB  gs://khelsutra-rally-corpus/videos/SHANAN/VID_1.mp4
   732.2 MiB  gs://khelsutra-rally-corpus/videos/GX010142_1080p.mp4
  1363.7 MiB  gs://khelsutra-rally-corpus/videos/GX030094_proxy.mp4
  1440.5 MiB  gs://khelsutra-rally-corpus/videos/golden/testlarge_short.mp4
  1482.0 MiB  gs://khelsutra-rally-corpus/videos/golden_1080p/Boxhill_Doubles.mp4
  1482.1 MiB  gs://khelsutra-rally-corpus/videos/golden_1080p/GX020094.mp4
  2234.2 MiB  gs://khelsutra-rally-corpus/videos/golden/GX010128.mp4
  2234.2 MiB  gs://khelsutra-rally-corpus/videos/GX010128.MP4
  2505.8 MiB  gs://khelsutra-rally-corpus/videos/SHANAN/VID_3.mp4
  2665.3 MiB  gs://khelsutra-rally-corpus/videos/SHANAN/VID_2.mp4
  2948.8 MiB  gs://khelsutra-rally-corpus/videos/golden_1080p/mahadevpura_2.mp4
  3491.3 MiB  gs://khelsutra-rally-corpus/videos/golden_1080p/Badminton_BXH_2.mp4
  5505.4 MiB  gs://khelsutra-rally-corpus/videos/golden/GX010141.mp4
```

49. assistant (2026-07-07T16:55:25.993Z)

Worker SA is the broad default compute SA (`492532266377-compute@?` ? can write project buckets). Smallest real video: `mahadevpura_smoke_180s.mp4` (86 MiB). Region env confirmed `asia-southeast1`. Let me grab the exact `output_root` the deployed control-plane uses.

50. user (2026-07-07T16:55:32.125Z)

```
DEPLOYMENT_PROFILE = cloud-serving
```

```
CLOUD_RUN_REGION = asia-southeast1
GOOGLE_CLOUD_PROJECT = gen-lang-client-0306026826
```

51. assistant (2026-07-07T16:56:36.314Z)

Worker SA is the broad default compute SA. **Key insight**: I can validate the `# pragma: no cover` `get_execution` + `_classify_execution_state` path against **real historical executions for free** before spending anything ? the CUJ already produced Succeeded and Failed executions. Let me write a read-only probe that lists past `rally-worker` executions and runs the real classifier against them.

Let me set up task tracking and check the GCS storage backend's init requirements (needed later for the live marker poll).

52. user (2026-07-07T16:56:37.218Z)

```
1       """Google Cloud Storage backend ? first-class object buckets.
2
3       Uses ``google-cloud-storage``, imported lazily so ``import backend.storage`` never pulls
it;
4       install with the ``storage-gcs`` extra. Credentials come from Application Default
Credentials
5       (``GOOGLE_APPLICATION_CREDENTIALS`` pointing at a service-account JSON, or workload
identity) ?
6       never from config. A ``client`` may be injected for tests.
7       """
8
9       from __future__ import annotations
10
11      import base64
12      import binascii
13      import logging
14      from datetime import timedelta
15      from importlib import import_module
16      from typing import Any, Iterator, Optional
17
18      from backend.storage.base import StorageBackend
19      from backend.storage.ref import StorageRef, StorageStat
20
21      logger = logging.getLogger(__name__)
22
23
24      def _md5_hex(md5_b64: "str | None") -> Optional[str]:
25          """GCS reports ``md5Hash`` as base64 of the raw digest; ``compute_video_id`` (the DB
key)
26          is a HEX digest ? convert so the two are directly comparable (#484). Composite
objects
27          have no md5Hash ? None; a malformed value is treated as absent, never raised."""
28          if not md5_b64:
29              return None
30          try:
31              raw = base64.b64decode(md5_b64, validate=True)
32          except (binascii.Error, ValueError):
33              return None
34          return raw.hex() if len(raw) == 16 else None
35
36
37      class GCSStorage(StorageBackend):
38          def __init__(
39              self,
40              *,
41              client: Any = None,
42              project: Optional[str] = None,
43              signed_url_ttl: int = 3600,
44              **_ignored,
```

```
45          ) -> None:
46              self._ttl = signed_url_ttl
47              if client is not None:
48                  self._client = client
49                  return
50              storage_module: Any = import_module("google.cloud.storage")
51              self._client = storage_module.Client(project=project)
52
53          def _blob(self, ref: StorageRef):
54              return self._client.bucket(ref.bucket).blob(ref.key)
55
56          def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
57              if dst is None:
58                  raise ValueError("GCSStorage.fetch_to_local requires a dst path")
59              import os
60
61              dst = str(dst)
62              if os.path.exists(dst) and not overwrite:
63                  return dst
64              os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
65              self._blob(ref).download_to_filename(dst)
66              return dst
67
68          def put(self, local_path, ref, *, content_type=None) -> StorageRef:
69              self._blob(ref).upload_from_filename(str(local_path), content_type=content_type)
70              return ref
71
72          def exists(self, ref) -> bool:
73              return bool(self._blob(ref).exists())
74
75          def stat(self, ref) -> StorageStat:
```

53. assistant (2026-07-07T16:56:47.542Z)

GCS backend uses ADC and ignores extra config ? my owner ADC works for reads. Let me create a task list and write the zero-cost read-only probe that exercises the real `get_execution` + `_classify_execution_state` against historical executions.

54. user (2026-07-07T16:56:54.609Z)

Task #1 created successfully: Probe get_execution + classifier on historical executions (zero-cost)

55. user (2026-07-07T16:57:00.718Z)

Task #2 created successfully: Test B: end-to-end fast-fail (crash before marker) ? rc 5, timed

56. user (2026-07-07T16:57:06.007Z)

Task #3 created successfully: Test C: healthy dispatch still returns rc 0 (no false-positive)

57. user (2026-07-07T16:57:11.006Z)

Task #4 created successfully: Cleanup + report: runlog + memory update

58. user (2026-07-07T16:57:15.562Z)

Chapter marked: "Live smoke: cloud-run fast-fail (#515)". Continue your current work.

59. user (2026-07-07T16:57:19.898Z)

Updated task #1 status

60. assistant (2026-07-07T16:57:38.028Z)

```
"""READ-ONLY smoke of the #515 fast-fail classifier against LIVE historical executions.

Exercises the exact `# pragma: no cover` path ? GoogleCloudRunJobClient.execution_terminal_state
-> run_v2.ExecutionsClient().get_execution -> _classify_execution_state ? against real past
rally-worker executions, at zero cost (no dispatch). Confirms the proto-plus field names /
marshaling the unit tests assume actually hold on the real API.
"""
from __future__ import annotations

from google.cloud import run_v2

from backend.compute.cloud_run import GoogleCloudRunJobClient, _classify_execution_state

PROJECT = "gen-lang-client-0306026826"
REGION = "asia-southeast1"
JOB = "rally-worker"

parent = f"projects/{PROJECT}/locations/{REGION}/jobs/{JOB}"
exec_client = run_v2.ExecutionsClient()

print(f"Listing executions under {parent}\n")
execs = list(exec_client.list_executions(parent=parent))
print(f"total executions: {len(execs)}\n")

client = GoogleCloudRunJobClient()
```

Sort newest first by create_time when available.

```
def _ct(e):
    return getattr(e, "create_time", None) or 0

execs.sort(key=_ct, reverse=True)

rows = []
for e in execs[:12]:
    short = e.name.rsplit("/", 1)[-1]
    completion = getattr(e, "completion_time", None)
    fields = {
        "completion_time": completion,
        "completion_time_type": type(completion).__name__,
        "succeeded_count": getattr(e, "succeeded_count", None),
        "failed_count": getattr(e, "failed_count", None),
        "cancelled_count": getattr(e, "cancelled_count", None),
        "running_count": getattr(e, "running_count", None),
        "task_count": getattr(e, "task_count", None),
    }
    # The pure classifier (unit-tested) directly on the fetched object:
    classified_direct = _classify_execution_state(e)
    # The REAL # pragma: no cover method that does its own get_execution fetch:
    via_client = client.execution_terminal_state(e.name)
    rows.append((short, fields, classified_direct, via_client))
    print(f"exec {short}")
    for k, v in fields.items():
        print(f"    {k:22} = {v!r}")
    print(f"    _classify_execution_state(direct) = {classified_direct!r}")
    print(f"    execution_terminal_state(get)     = {via_client!r}")
    assert classified_direct == via_client, "MISMATCH between direct classify and get_execution
path!"
    print()
```

Summary: do we have at least one Succeeded and one Failed to validate both branche

```
labels = [r[2] for r in rows]
```

```
print("=== SUMMARY ===")
print("terminal labels seen:", sorted({str(x) for x in labels}))
print("have Succeeded:", "Succeeded" in labels)
print("have Failed:   ", "Failed" in labels)
print("have Cancelled:", "Cancelled" in labels)
print("have None(running/pending):", None in labels)
```

61. user (2026-07-07T16:57:38.078Z)

```
File created successfully at: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\0c4fc269-564f-4fb2-9c1e-9389160b6be0\scratchpad\probe_executions.py
(file state is current in your context ? no need to Read it back)
```

62. user (2026-07-07T16:57:46.269Z)

```
Traceback (most recent call last):
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\0c4fc269-564f-4fb2-9c1e-9389160b6be0\scratchpad\probe_executions.py", line 12, in <module>
    from backend.compute.cloud_run import GoogleCloudRunJobClient, _classify_execution_state
ModuleNotFoundError: No module named 'backend'
```

63. assistant (2026-07-07T16:57:58.968Z)

```
Running a script file puts the script's dir (not cwd) on `sys.path`. Let me set `PYTHONPATH` to the repo root.
```

64. user (2026-07-07T16:58:43.510Z)

```
C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end user credentials from Google Cloud SDK without a quota project. You might receive a "quota exceeded" or "API not enabled" error. See the following page for troubleshooting: https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
  warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end user credentials from Google Cloud SDK without a quota project. You might receive a "quota exceeded" or "API not enabled" error. See the following page for troubleshooting: https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
  warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end user credentials from Google Cloud SDK without a quota project. You might receive a "quota exceeded" or "API not enabled" error. See the following page for troubleshooting: https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
  warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end user credentials from Google Cloud SDK without a quota project. You might receive a "quota exceeded" or "API not enabled" error. See the following page for troubleshooting: https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
  warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end user credentials from Google Cloud SDK without a quota project. You might receive a "quota exceeded" or "API not enabled" error. See the following page for troubleshooting: https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
  warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end user credentials from Google Cloud SDK without a quota project. You might receive a "quota exceeded" or "API not enabled" error. See the following page for troubleshooting: https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
```

```
  warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
  warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
  warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
  warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
  warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
  warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
  warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
  warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
Listing executions under projects/gen-lang-client-0306026826/locations/asia-
southeast1/jobs/rally-worker

total executions: 23

exec rally-worker-ql56s
    completion_time        = DatetimeWithNanoseconds(2026, 7, 7, 14, 32, 50, 168345,
tzinfo=datetime.timezone.utc)
    completion_time_type   = 'DatetimeWithNanoseconds'
    succeeded_count        = 1
    failed_count           = 0
    cancelled_count        = 0
    running_count          = 0
    task_count             = 1
    _classify_execution_state(direct) = 'Succeeded'
    execution_terminal_state(get)     = 'Succeeded'

exec rally-worker-2lbz2
    completion_time        = DatetimeWithNanoseconds(2026, 7, 7, 14, 14, 34, 96595,
tzinfo=datetime.timezone.utc)
    completion_time_type   = 'DatetimeWithNanoseconds'
    succeeded_count        = 0
```

```
    failed_count           = 1
    cancelled_count        = 0
    running_count          = 0
    task_count             = 1
    _classify_execution_state(direct) = 'Failed'
    execution_terminal_state(get)     = 'Failed'

exec rally-worker-cnzxw
    completion_time        = DatetimeWithNanoseconds(2026, 7, 7, 12, 29, 38, 837936,
tzinfo=datetime.timezone.utc)
    completion_time_type   = 'DatetimeWithNanoseconds'
    succeeded_count        = 1
    failed_count           = 0
    cancelled_count        = 0
    running_count          = 0
    task_count             = 1
    _classify_execution_state(direct) = 'Succeeded'
    execution_terminal_state(get)     = 'Succeeded'

exec rally-worker-6htsw
    completion_time        = DatetimeWithNanoseconds(2026, 7, 7, 12, 24, 2, 895549,
tzinfo=datetime.timezone.utc)
    completion_time_type   = 'DatetimeWithNanoseconds'
    succeeded_count        = 1
    failed_count           = 0
    cancelled_count        = 0
    running_count          = 0
    task_count             = 1
    _classify_execution_state(direct) = 'Succeeded'
    execution_terminal_state(get)     = 'Succeeded'

exec rally-worker-clkwc
    completion_time        = DatetimeWithNanoseconds(2026, 7, 7, 12, 1, 53, 53809,
tzinfo=datetime.timezone.utc)
    completion_time_type   = 'DatetimeWithNanoseconds'
    succeeded_count        = 1
    failed_count           = 0
    cancelled_count        = 0
    running_count          = 0
    task_count             = 1
    _classify_execution_state(direct) = 'Succeeded'
    execution_terminal_state(get)     = 'Succeeded'

exec rally-worker-p2wd6
    completion_time        = DatetimeWithNanoseconds(2026, 7, 7, 11, 57, 48, 444485,
tzinfo=datetime.timezone.utc)
    completion_time_type   = 'DatetimeWithNanoseconds'
    succeeded_count        = 1
    failed_count           = 0
    cancelled_count        = 0
    running_count          = 0
    task_count             = 1
    _classify_execution_state(direct) = 'Succeeded'
    execution_terminal_state(get)     = 'Succeeded'

exec rally-worker-kzjw6
    completion_time        = DatetimeWithNanoseconds(2026, 7, 6, 18, 38, 47, 907556,
tzinfo=datetime.timezone.utc)
    completion_time_type   = 'DatetimeWithNanoseconds'
    succeeded_count        = 1
    failed_count           = 0
    cancelled_count        = 0
    running_count          = 0
    task_count             = 1
    _classify_execution_state(direct) = 'Succeeded'
```

```
    execution_terminal_state(get)     = 'Succeeded'

exec rally-worker-wh66x
    completion_time        = DatetimeWithNanoseconds(2026, 7, 6, 18, 32, 34, 252574,
tzinfo=datetime.timezone.utc)
    completion_time_type   = 'DatetimeWithNanoseconds'
    succeeded_count        = 1
    failed_count           = 0
    cancelled_count        = 0
    running_count          = 0
    task_count             = 1
    _classify_execution_state(direct) = 'Succeeded'
    execution_terminal_state(get)     = 'Succeeded'

exec rally-worker-8tmr7
    completion_time        = DatetimeWithNanoseconds(2026, 7, 6, 18, 13, 53, 623179,
tzinfo=datetime.timezone.utc)
    completion_time_type   = 'DatetimeWithNanoseconds'
    succeeded_count        = 1
    failed_count           = 0
    cancelled_count        = 0
    running_count          = 0
    task_count             = 1
    _classify_execution_state(direct) = 'Succeeded'
    execution_terminal_state(get)     = 'Succeeded'

exec rally-worker-wf294
    completion_time        = DatetimeWithNanoseconds(2026, 7, 6, 16, 49, 16, 81283,
tzinfo=datetime.timezone.utc)
    completion_time_type   = 'DatetimeWithNanoseconds'
    succeeded_count        = 1
    failed_count           = 0
    cancelled_count        = 0
    running_count          = 0
    task_count             = 1
    _classify_execution_state(direct) = 'Succeeded'
    execution_terminal_state(get)     = 'Succeeded'
```

65. assistant (2026-07-07T16:58:55.514Z)

```
The read-only probe already closes the core gap. Live confirmation:
- `completion_time` marshals to `DatetimeWithNanoseconds` (a `datetime`) when set ? the `is
None` terminality check is correct on the real API (the proto-plus fix holds).
- `succeeded_count`/`failed_count`/`cancelled_count` are plain ints, exactly as
`_classify_execution_state` reads them.
- Real Succeeded (`ql56s` ? the CUJ green run) and real Failed (`2lbz2`) both classify
correctly, and the `# pragma: no cover` `get_execution` path agrees with the direct classifier
on all 23 executions (the assert never fired).

Let me see the summary tail.
```

66. user (2026-07-07T16:59:40.828Z)

```
    succeeded_count        = 1
    failed_count           = 0
    cancelled_count        = 0
    running_count          = 0
    task_count             = 1
    _classify_execution_state(direct) = 'Succeeded'
    execution_terminal_state(get)     = 'Succeeded'

exec rally-worker-2mz8t
    completion_time        = DatetimeWithNanoseconds(2026, 7, 5, 14, 37, 49, 67936,
tzinfo=datetime.timezone.utc)
    completion_time_type   = 'DatetimeWithNanoseconds'
```

```
    succeeded_count        = 1
    failed_count           = 0
    cancelled_count        = 0
    running_count          = 0
    task_count             = 1
    _classify_execution_state(direct) = 'Succeeded'
    execution_terminal_state(get)     = 'Succeeded'

=== SUMMARY ===
terminal labels seen: ['Failed', 'Succeeded']
have Succeeded: True
have Failed:    True
have Cancelled: False
have None(running/pending): False
```

67. assistant (2026-07-07T17:00:53.898Z)

Task 1 is complete ? the core `# pragma: no cover` gap is closed against live infra (both Failed and Succeeded branches, 23/23 classifier agreement). Let me record that and move to Test B.

Critical wiring note I just worked out: I can't pass the bad `--set` to the outer `worker infer` CLI (its own `_apply_overrides` would crash my *local* dispatcher). Instead I'll build `CloudRunCompute` directly and call the **real** `_dispatch_remote(compute, args)` ? the `config_overrides` in the spec are sent only to the remote worker, and `_dispatch_remote` gives me the authentic rc-5 mapping.

68. user (2026-07-07T17:00:54.350Z)

Updated task #1 status

69. user (2026-07-07T17:00:57.508Z)

Updated task #2 status

70. assistant (2026-07-07T17:01:20.161Z)

```
"""Test B ? LIVE end-to-end fast-fail: dispatch rally-worker with a pre-marker crash, expect the
Executions-API probe to fail FAST (RemoteExecutionFailed -> rc 5), NOT a ~65-min PolicyTimeout.

The dispatched worker is forced to crash BEFORE writing any marker by a bad --set
(``zzz_smoke_crash.boom=1`` -> AttributeError in _apply_overrides, before cmd_infer reaches the
pipeline). maxRetries=0 on the Job -> the execution goes terminal Failed after the first crash.
The bad override is carried in the SPEC's config_overrides (sent only to the remote worker via
argv); it is never applied to this local dispatcher. We call the REAL worker
``_dispatch_remote(compute, args)`` so the rc-5 mapping is the authentic code path.

A modest max_wait (600s) is a SAFETY NET: if the probe were broken we'd fall through to a
PolicyTimeout (rc 4) at 10 min instead of hanging ? cleanly distinguishing pass (rc 5, fast)
from
fail (rc 4).
"""
from __future__ import annotations

import dataclasses
import logging
import time
from types import SimpleNamespace

from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient
from backend.infrastructure.execution_policy import DEFAULT_POLICY
from backend.worker.__main__ import _dispatch_remote

PROJECT = "gen-lang-client-0306026826"
REGION = "asia-southeast1"
```

```
JOB = "rally-worker"
OUT_URI = "gs://khelsutra-rally-nightly/dev/smoke515/B"
VIDEO_URI = "gs://khelsutra-rally-corpus/videos/mahadevpura_smoke_180s.mp4"  # never fetched
(crash first)
TOKEN = "smokeB1"
```

--- capture every log record so we can print the exact RemoteExecutionFailed diagnos

```
records: list[logging.LogRecord] = []


class _Capture(logging.Handler):
    def emit(self, record):
        records.append(record)


logging.basicConfig(
    level=logging.INFO, format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
datefmt="%H:%M:%S"
)
logging.getLogger("compute.cloud_run").setLevel(logging.DEBUG)
logging.getLogger("execution.policy").setLevel(logging.DEBUG)
logging.getLogger().addHandler(_Capture())
```

poll_every default 10s; max_wait 600s safety net.

```
policy = dataclasses.replace(DEFAULT_POLICY, max_wait_sec=600.0)

compute = CloudRunCompute(
    run_client=GoogleCloudRunJobClient(),
    job_name=JOB,
    region=REGION,
    project=PROJECT,
    storage_config={"backend": "gcs"},
    policy=policy,
    token=TOKEN,
)

args = SimpleNamespace(
    video_uri=VIDEO_URI,
    out_uri=OUT_URI,
    segmenter=None,
    set=["zzz_smoke_crash.boom=1"],  # -> remote worker AttributeError before any marker
    skip_validation=True,
)

print(f"=== TEST B: dispatching crash-before-marker job to {JOB} ({REGION}) ===")
t0 = time.monotonic()
rc = _dispatch_remote(compute, args)
elapsed = time.monotonic() - t0
```

Pull the exec name + the RemoteExecutionFailed diagnostic from captured logs.

```
exec_name = ""
diag = ""
for r in records:
    msg = r.getMessage()
    if "dispatched job=" in msg and "exec=" in msg:
        exec_name = msg.split("exec=", 1)[1].split(";", 1)[0].strip()
    if "terminated without a completion marker" in msg or "reached " in msg and "terminal state"
in msg:
        diag = msg

print("\n=== TEST B RESULT ===")
print(f"returncode           = {rc}   (expect 5 = RemoteExecutionFailed; 4 = PolicyTimeout would
be a FAIL)")
```

```
print(f"elapsed_sec          = {elapsed:.1f}")
print(f"dispatched execution = {exec_name}")
print(f"diagnostic message   = {diag}")
print(f"VERDICT              = {'PASS' if rc == 5 else 'FAIL'}")
```

71. user (2026-07-07T17:01:20.198Z)

```
File created successfully at: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-
badminton-highlight-indexer\0c4fc269-564f-4fb2-9c1e-9389160b6be0\scratchpad\test_b_fastfail.py
(file state is current in your context ? no need to Read it back)
```

72. assistant (2026-07-07T17:01:30.112Z)

```
Test B is ready. Let me run it in the background (it self-times and bounds itself at 600s),
capturing the full poll trail to a log.
```

73. user (2026-07-07T17:01:33.939Z)

```
Command running in background with ID: bbqxn3ko5. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\0c4fc269-564f-4fb2-9c1e-9389160b6be0\tasks\bbqxn3ko5.output. You will be notified when
it completes. To check interim output, use Read on that file path.
```

74. assistant (2026-07-07T17:02:10.605Z)

```
While Test B runs, let me prepare the healthy-dispatch harness (Test C). I'll use the real
native serving path on the small smoke clip with `skip_ai_handoff=true` so it reliably succeeds
without a Gemini key.
```

75. assistant (2026-07-07T17:02:24.876Z)

```
"""Test C ? LIVE healthy dispatch: a NORMAL good run must still return rc 0, proving the new
Executions-API status probe does NOT false-positive-abort a healthy run.

While the worker is RUNNING, get_execution returns completion_time=None (unset) ->
_classify_execution_state -> None -> _poll_check keeps bucket-polling (no abort). When the
worker
finishes it writes the SUCCESS done-marker -> run_infer resolves rc 0.

Real native serving path on the smallest real clip (mahadevpura_smoke_180s, 86 MiB, 180s), with
skip_ai_handoff=true (no Gemini key needed) + skip_validation (control-plane is the gate).
"""
from __future__ import annotations

import dataclasses
import logging
import time
from types import SimpleNamespace

from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient
from backend.infrastructure.execution_policy import DEFAULT_POLICY
from backend.worker.__main__ import _dispatch_remote

PROJECT = "gen-lang-client-0306026826"
REGION = "asia-southeast1"
JOB = "rally-worker"
OUT_URI = "gs://khelsutra-rally-nightly/dev/smoke515/C"
VIDEO_URI = "gs://khelsutra-rally-corpus/videos/mahadevpura_smoke_180s.mp4"
TOKEN = "smokeC1"

records: list[logging.LogRecord] = []


class _Capture(logging.Handler):
```

```
    def emit(self, record):
        records.append(record)


logging.basicConfig(
    level=logging.INFO, format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
datefmt="%H:%M:%S"
)
logging.getLogger("compute.cloud_run").setLevel(logging.DEBUG)
logging.getLogger("execution.policy").setLevel(logging.DEBUG)
logging.getLogger().addHandler(_Capture())
```

Generous budget for a real GPU run: cold start + fetch + decode + WASB inference.

```
policy = dataclasses.replace(DEFAULT_POLICY, max_wait_sec=1200.0, poll_every_n_sec=15.0)

compute = CloudRunCompute(
    run_client=GoogleCloudRunJobClient(),
    job_name=JOB,
    region=REGION,
    project=PROJECT,
    storage_config={"backend": "gcs"},
    policy=policy,
    token=TOKEN,
)

args = SimpleNamespace(
    video_uri=VIDEO_URI,
    out_uri=OUT_URI,
    segmenter="wasb_hybrid",
    set=["indexing.skip_ai_handoff=true"],  # no Gemini dependency; native WASB still detects
rallies
    skip_validation=True,
)

print(f"=== TEST C: healthy dispatch to {JOB} ({REGION}) on {VIDEO_URI.rsplit('/',1)[-1]} ===")
t0 = time.monotonic()
rc = _dispatch_remote(compute, args)
elapsed = time.monotonic() - t0

exec_name = ""
running_polls = 0
for r in records:
    msg = r.getMessage()
    if "dispatched job=" in msg and "exec=" in msg:
        exec_name = msg.split("exec=", 1)[1].split(";", 1)[0].strip()
    if "not ready" in msg:  # a poll iteration where the probe returned None (still running)
        running_polls += 1

print("\n=== TEST C RESULT ===")
print(f"returncode              = {rc}   (expect 0 = healthy success)")
print(f"elapsed_sec             = {elapsed:.1f}")
print(f"dispatched execution    = {exec_name}")
print(f"running polls (probe=None while running) = {running_polls}")
print(f"VERDICT                 = {'PASS' if rc == 0 else 'FAIL/INVESTIGATE'}")
```

76. user (2026-07-07T17:02:24.923Z)

```
File created successfully at: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-
badminton-highlight-indexer\0c4fc269-564f-4fb2-9c1e-9389160b6be0\scratchpad\test_c_healthy.py
(file state is current in your context ? no need to Read it back)
```

77. assistant (2026-07-07T17:02:27.353Z)

```
Test B finished. Let me read its full log.
```

78. user (2026-07-07T17:02:28.059Z)

```
1       === TEST B: dispatching crash-before-marker job to rally-worker (asia-southeast1) ===
2       C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
3         warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
4       22:31:40 [INFO] compute.cloud_run: cloud-run: dispatched job=rally-worker
exec=projects/gen-lang-client-0306026826/locations/asia-southeast1/jobs/rally-
worker/executions/rally-worker-tgbbf; bucket-polling gs://khelsutra-rally-
nightly/dev/smoke515/B/_dispatch/smokeB1.done
5       22:31:40 [INFO] compute.cloud_run: cloud-run-infer: waiting (poll_every=10s,
max_wait=600s)
6       C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
7         warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
8       C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
9         warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
10      22:31:47 [DEBUG] compute.cloud_run: cloud-run-infer: poll 1, 6s elapsed, not ready
11      C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
12        warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
13      C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
14        warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
15      22:32:03 [DEBUG] compute.cloud_run: cloud-run-infer: poll 2, 23s elapsed, not ready
16      C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
17        warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
18      C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
19        warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
20      C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
21        warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
22      22:32:23 [ERROR] backend.worker: [worker] remote execution terminated without a
completion marker: cloud-run: job=rally-worker execution=projects/gen-lang-
client-0306026826/locations/asia-southeast1/jobs/rally-worker/executions/rally-worker-tgbbf
reached terminal state 'Failed' with NO completion marker ? the worker exited before writing one
(crash / OOM / pre-marker abort, or a failed marker upload on an otherwise-Succeeded run). The
execution is over, so the marker will never appear; inspect the Cloud Run Job execution log to
```

```
diagnose: `gcloud run jobs executions describe projects/gen-lang-
client-0306026826/locations/asia-southeast1/jobs/rally-worker/executions/rally-worker-tgbbf
--region asia-southeast1`.
23
24      === TEST B RESULT ===
25      returncode           = 5   (expect 5 = RemoteExecutionFailed; 4 = PolicyTimeout would be
a FAIL)
26      elapsed_sec          = 47.0
27      dispatched execution = projects/gen-lang-client-0306026826/locations/asia-
southeast1/jobs/rally-worker/executions/rally-worker-tgbbf
28      diagnostic message   = [worker] remote execution terminated without a completion marker:
cloud-run: job=rally-worker execution=projects/gen-lang-client-0306026826/locations/asia-
southeast1/jobs/rally-worker/executions/rally-worker-tgbbf reached terminal state 'Failed' with
NO completion marker ? the worker exited before writing one (crash / OOM / pre-marker abort, or
a failed marker upload on an otherwise-Succeeded run). The execution is over, so the marker will
never appear; inspect the Cloud Run Job execution log to diagnose: `gcloud run jobs executions
describe projects/gen-lang-client-0306026826/locations/asia-southeast1/jobs/rally-
worker/executions/rally-worker-tgbbf --region asia-southeast1`.
29      VERDICT              = PASS
30
```

79. assistant (2026-07-07T17:02:50.076Z)

```
**Test B PASS.** Dispatched `rally-worker-tgbbf`, polled at 6s and 23s (probe returned None ?
still running, no false-abort), then at **47s** detected terminal `Failed` + no marker ? **rc
5** (RemoteExecutionFailed, not rc 4 timeout), with a diagnostic naming the execution, state,
and the exact `gcloud ? executions describe ? --region asia-southeast1` command. That's pass-
criteria 1?3, plus early evidence for #4.

Let me cross-check the fresh execution independently: confirm it's genuinely Failed, that no
marker exists, and pull the crash log to prove it was the pre-marker config abort I engineered.
```

80. user (2026-07-07T17:02:58.757Z)

```
=== describe rally-worker-tgbbf ===
status:
  completionTime: '2026-07-07T17:02:04.425001Z'
  conditions:
  - lastTransitionTime: '2026-07-07T17:02:04.425001Z'
    message: 'Task rally-worker-tgbbf-task0 failed with exit code: 1 and message:
      The container exited with an error.'
    reason: NonZeroExitCode
    status: 'False'
    type: Completed
  - lastTransitionTime: '2026-07-07T17:01:41.783402Z'
    message: Provisioned imported containers.
    status: 'True'
    type: ResourcesAvailable
  - lastTransitionTime: '2026-07-07T17:01:49.758757Z'
    message: Started deployed execution in 7.97s.
    status: 'True'
    type: Started
  - lastTransitionTime: '2026-07-07T17:01:41.274915Z'
    message: Imported container image in 1.02s.
    status: 'True'
    type: ContainerReady
  failedCount: 1

=== marker object existence (expect: does NOT exist) ===
ERROR: (gcloud.storage.ls) One or more URLs matched no objects.
```

81. user (2026-07-07T17:03:03.608Z)

```
=== crash log lines for rally-worker-tgbbf (expect AttributeError: zzz_smoke_crash) ===
```

```
Traceback (most recent call last):
  File "/usr/lib/python3.10/runpy.py", line 196, in _run_module_as_main
    return _run_code(code, main_globals, None,
  File "/usr/lib/python3.10/runpy.py", line 86, in _run_code
    exec(code, run_globals)
  File "/app/backend/worker/__main__.py", line 828, in <module>
    sys.exit(main())
  File "/app/backend/worker/__main__.py", line 824, in main
    return args.func(args)
  File "/app/backend/worker/__main__.py", line 236, in cmd_infer
    _apply_overrides(config, args.set)
  File "/app/backend/worker/__main__.py", line 70, in _apply_overrides
    obj = getattr(obj, p)
  File "/usr/local/lib/python3.10/dist-packages/pydantic/main.py", line 1042, in __getattr__
    raise AttributeError(f'{type(self).__name__!r} object has no attribute {item!r}')
AttributeError: 'AppConfig' object has no attribute 'zzz_smoke_crash'
```

82. assistant (2026-07-07T17:03:28.005Z)

```
**Test B fully confirmed** with airtight independent evidence:
- Execution `rally-worker-tgbbf`: `failedCount: 1`, `NonZeroExitCode` (exit 1), terminal.
- Container log shows the crash was exactly the engineered pre-marker abort: `cmd_infer:236 ?
_apply_overrides:70 ? AttributeError: 'AppConfig' object has no attribute 'zzz_smoke_crash'` ?
before any marker write.
- The marker object genuinely does **not** exist. So this is a true terminal-Failed-with-no-
marker, and the probe caught it in ~47s (one poll interval after terminal) ? rc 5.

Now the healthy sanity run (Test C).
```

83. user (2026-07-07T17:03:28.302Z)

```
Updated task #2 status
```

84. user (2026-07-07T17:03:30.588Z)

```
Updated task #3 status
```

85. user (2026-07-07T17:03:40.458Z)

```
Command running in background with ID: b6cpwpzsb. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\0c4fc269-564f-4fb2-9c1e-9389160b6be0\tasks\b6cpwpzsb.output. You will be notified when
it completes. To check interim output, use Read on that file path.
```

86. assistant (2026-07-07T17:03:57.713Z)

```
While the GPU run proceeds, let me check the runlog directory conventions so I can write a
properly-named runlog.
```

87. user (2026-07-07T17:03:58.294Z)

```
docs\COMPUTE_DECOUPLED_SERVING\runlogs\RUNLOG_truly-local_TEMPLATE.md
docs\COMPUTE_DECOUPLED_SERVING\runlogs\DEPLOY_REPORT_cloud-serving_2026-07-04.md
docs\COMPUTE_DECOUPLED_SERVING\runlogs\DEPLOY_REPORT_cloud-serving_a1-cuj_2026-07-04.md
docs\COMPUTE_DECOUPLED_SERVING\runlogs\DEPLOY_REPORT_cloud-serving_a1-e2e_2026-07-04.md
docs\COMPUTE_DECOUPLED_SERVING\runlogs\DEPLOY_REPORT_cloud-serving_cp-rebuild_2026-07-05.md
docs\COMPUTE_DECOUPLED_SERVING\runlogs\DEPLOY_REPORT_cloud-serving_alpha-all1v1_2026-07-06.md
docs\COMPUTE_DECOUPLED_SERVING\runlogs\README.md
docs\COMPUTE_DECOUPLED_SERVING\runlogs\DEPLOY_REPORT_cloud-serving_gpu-resident-
nvdec_2026-07-07.md
```

88. user (2026-07-07T17:04:05.050Z)

```
1       # runlogs/ ? sample runs & deploy reports for posterity
```

```
2
3       This directory is the **durable record of real runs** of the compute-decoupled serving
profiles ?
4       the same convention proven by the archived
`DECOUPLED_COMPUTE/DEPLOY_REPORT_GCP_2026-06-20.md`. The
5       goal: **the next deploy is one-shot**, and anyone can trace exactly what was run, what
it cost, and
6       what came out.
7
8       ## When to add a file here
9       - Any **real-hardware run** (owner-side L5 in
[`../TESTING_STRATEGY.md`](../TESTING_STRATEGY.md)) of a profile.
10      - Every **default flip** that touches the core ships its Launch Report record here too
(per [[launch-report-convention]]).
11
12      ## Naming
13      `DEPLOY_REPORT_<profile>_<YYYY-MM-DD>.md` ? e.g. `DEPLOY_REPORT_truly-
local_2026-07-01.md`,
14      `DEPLOY_REPORT_cloud-serving_2026-07-01.md`.
15
16      ## Required contents (mirror the DECOUPLED deploy report)
17      1. **TL;DR** ? what ran, profile, ?/?/? per stage (detect / window / stitch / serve).
18      2. **Resources** ? exact machine/VM/Cloud-Run config, region, image; for cloud, the
bucket + SA + budget.
19      3. **Exact commands** ? copy-pasteable (`worker infer ?`, the dispatch call, the
config/profile used).
20      4. **Run results** ? rally counts, **profile-parity numbers** (windows/segments vs the
other profile),
21         and the metering trio **`gpu_active_seconds` / `wall_seconds` / `bytes_out`** (even
before the M8 ledger lands).
22      5. **Reel smoke-test** ? `ffmpeg -f null -` full decode (exit 0) + a 5-frame contact
sheet visually
23         confirming real rallies (court/net/players mid-rally).
24      6. **Gotchas** ? conda-ToS, WASB-C3, GPU stockouts, font fallback, cold-start, NVDEC,
etc.
25      7. **Cost** ? measured/estimated; reconcile probe metrics vs the platform's per-
invocation billing.
26      8. **Teardown / final state** ? for cloud, the `$0` audit (`gcloud compute instances
list` + `disks list` empty); **DELETE, never just stop** ([[gcp-vm-always-delete]]).
27
28      ## DoD coupling
29      `COMPUTE_DECOUPLED_SERVING` is not DONE until **`truly-local` (M5)** and **`cloud-
serving` (M7)** each
30      have **?1 committed report here** (README §9).
31
32      ## Index (append as runs land)
33      | Date | Profile | Result | File |
34      |---|---|---|---|
35      | 2026-07-06 | cloud-serving | alpha-tester "All 1 v 1" timed out ? downsample fix
proven on L4 (720p30: 956 s, 60 rallies) | [DEPLOY_REPORT_cloud-serving_alpha-
all1v1_2026-07-06.md](DEPLOY_REPORT_cloud-serving_alpha-all1v1_2026-07-06.md) |
36      | 2026-07-07 | cloud-serving | GPU-resident NVDEC decode (#506/#507) shipped: image bump
+ cutover, cpu/nvdec smokes, GX010128 trajectory reproduced VM (26479?26481 vis), enabled +
canaried | [DEPLOY_REPORT_cloud-serving_gpu-resident-nvdec_2026-07-07.md](DEPLOY_REPORT_cloud-
serving_gpu-resident-nvdec_2026-07-07.md) |
37
```

89. assistant (2026-07-07T17:04:15.935Z)

Convention understood. Let me peek at Test C's interim progress to confirm the dispatch is polling (not erroring early), then draft the runlog while it runs.

90. user (2026-07-07T17:04:18.275Z)

```
1       === TEST C: healthy dispatch to rally-worker (asia-southeast1) on
```

```
mahadevpura_smoke_180s.mp4 ===
2       C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
3         warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
4       22:33:46 [INFO] compute.cloud_run: cloud-run: dispatched job=rally-worker
exec=projects/gen-lang-client-0306026826/locations/asia-southeast1/jobs/rally-
worker/executions/rally-worker-9tq5g; bucket-polling gs://khelsutra-rally-
nightly/dev/smoke515/C/_dispatch/smokeC1.done
5       22:33:46 [INFO] compute.cloud_run: cloud-run-infer: waiting (poll_every=15s,
max_wait=1200s)
6       C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
7         warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
8       C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
9         warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
10      22:33:52 [DEBUG] compute.cloud_run: cloud-run-infer: poll 1, 6s elapsed, not ready
11      C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
12        warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
13      C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
14        warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
15      22:34:14 [DEBUG] compute.cloud_run: cloud-run-infer: poll 2, 28s elapsed, not ready
16
```

91. assistant (2026-07-07T17:04:51.545Z)

```
Test C is dispatched (`rally-worker-9tq5g`) and polling healthily ? poll 1 @6s and poll 2 @28s
both "not ready" (probe returning None while the exec runs = the no-false-positive behavior).
It'll take several minutes for the GPU run. Let me draft the runlog now (I'll fill in Test C's
final numbers when it lands).
```

92. assistant (2026-07-07T17:05:44.928Z)

RUNLOG ? cloud-serving crash-before-marker fast-fail (PR #515) ? LIVE SMOKE ? 202

TL;DR

```
Live end-to-end smoke of the **Executions-API fast-fail** shipped in PR #515 (squash-merged to
master as `ef41473`). The gap this closes: `CloudRunCompute.run_infer`'s status probe
(`GoogleCloudRunJobClient.execution_terminal_state` ? `run_v2.ExecutionsClient().get_execution`)
is
`# pragma: no cover` ? it had never run against live Cloud Run, so the real API field names /
proto-plus marshaling that `_classify_execution_state` assumes were unverified.

| Stage | What | Result |
|---|---|---|
| A ? probe vs live history | Real `get_execution` + `_classify_execution_state` over 12 of 23
```

historical `rally-worker` executions | ? proto-plus assumptions hold; **Succeeded** + **Failed** branches both classify correctly; direct-classify == get_execution path on all 12 |
| B ? fast-fail (crash before marker) | Dispatch a real worker forced to crash before the marker; time the fail | ? **rc 5** (`RemoteExecutionFailed`) in **47.0 s** (not rc 4 / not ~65 min); diagnostic names exec + state + `gcloud ? describe` cmd |
| C ? healthy dispatch | A normal good run must still succeed (no false-positive) | ? **rc 0** in **RUN_C_ELAPSED s** (`RUN_C_SEGMENTS` segments); probe returned `None` on `RUN_C_POLLS` running-polls, never aborting |

Verdict: PASS. The real `get_execution` wiring (auth, region, field names, proto-plus `Timestamp`?`datetime` marshaling) behaves exactly as the unit tests assume. No bug filed.

Resources

- **Infra (unchanged, live):** Cloud Run **Job** `rally-worker` + service `rally-control-plane`, region **asia-southeast1**, project `gen-lang-client-0306026826`.
  - Worker image digest (**unchanged ? no pin performed**): `asia-southeast1-docker.pkg.dev/gen-lang-client-0306026826/rally/rally-worker@sha256:8045f54ea9fcbf52426305ab987f122f4fc8bed6b13be93283d21c5cab57a655`
  - Worker **maxRetries: 0** (a pre-marker crash goes terminal `Failed` after the first task, no retry churn).
  - Worker SA: `492532266377-compute@developer.gserviceaccount.com`.
- **Driver:** run **locally** (Windows, Python 3.13) importing local master (`ef41473`) `backend.compute.cloud_run` + `backend.worker.__main__._dispatch_remote`, with `google-cloud-run` + `google-cloud-storage` and owner **ADC** (`avi.dullu@gmail.com`). No control-plane API / CF-Access JWT was needed: `CloudRunCompute` was constructed directly (the same object `get_compute_backend` builds from `CLOUD_RUN_JOB`/`CLOUD_RUN_REGION`/`GOOGLE_CLOUD_PROJECT`), pointed at the real Job.
- **Region note:** used the **deployed** region `asia-southeast1` (cloud_run.py's `CLOUD_RUN_REGION` default is `asia-south1`, but the control-plane env sets `asia-southeast1`).

How the "worker crashes before the marker" condition was manufactured

The worker's `_fail()` path writes a *failure* marker (rc 2/3), so a *clean* pipeline failure resolves through the marker ? it does **not** exercise the fast-fail. The fast-fail only fires on a
crash **before any marker**. Rather than pin an old pre-#513 image (argparse-abort on `--skip-validation`), the same class was reproduced on the **current** image with zero image mutation: a bad `--set` in the spec's `config_overrides` (`zzz_smoke_crash.boom=1`) ? `cmd_infer` ? `_apply_overrides` ? `getattr(config, "zzz_smoke_crash")` raises `AttributeError`, which `main()` does **not** catch (`return args.func(args)` has no try) ? the process exits non-zero
before `_write_done_marker` is ever reached. The bad override rides only in the SPEC's `config_overrides` (sent to the remote worker via `--set`), so it never touches the local dispatcher.

Exact commands / harnesses

Read-only probe (Test A) and the two dispatch harnesses live in this session's scratchpad; the essential calls:

```python
# Test A ? the # pragma: no cover path, live, zero cost:
from google.cloud import run_v2
from backend.compute.cloud_run import GoogleCloudRunJobClient, _classify_execution_state
parent = "projects/gen-lang-client-0306026826/locations/asia-southeast1/jobs/rally-worker"
for e in run_v2.ExecutionsClient().list_executions(parent=parent):
    assert _classify_execution_state(e) == GoogleCloudRunJobClient().execution_terminal_state(e.name)

# Tests B/C ? real dispatch through the authentic rc-mapping:
```

```
from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient
from backend.worker.__main__ import _dispatch_remote      # returns 5 on RemoteExecutionFailed,
4 on PolicyTimeout, else result.rc
compute = CloudRunCompute(run_client=GoogleCloudRunJobClient(), job_name="rally-worker",
                          region="asia-southeast1", project="gen-lang-client-0306026826",
                          storage_config={"backend": "gcs"}, policy=<max_wait 600s/1200s>,
token=...)
```

B: args.set=["zzz_smoke_crash.boom=1"], skip_validation=True -> rc 5

C: args.set=["indexing.skip_ai_handoff=true"], segmenter="wasb_hybrid", skip_valida

```
rc = _dispatch_remote(compute, args)
```

Run results

A ? classifier vs live history (read-only, $0)

`run_v2.ExecutionsClient().get_execution(...)` returns a `run_v2.Execution` whose:
- `completion_time` marshals to a **`DatetimeWithNanoseconds`** (a `datetime` subclass) when the
  execution is terminal, and to **`None`** while pending/running ? so
  `_classify_execution_state`'s terminality test `getattr(exe, "completion_time", None) is None`
  is **correct on the real API** (this is the exact proto-plus point the pre-merge adversarial
  review flagged: a `Timestamp` is NOT a proto with `.seconds`; it's a Python `datetime`).
- `succeeded_count` / `failed_count` / `cancelled_count` / `running_count` / `task_count` are
plain
  `int`s, exactly as the classifier reads them.

Both terminal branches were validated against **real** executions:
- **Succeeded** ? `rally-worker-ql56s` (`succeeded_count=1`, the 2026-07-07 CUJ green run) ?
`"Succeeded"`.
- **Failed** ? `rally-worker-2lbz2` (`failed_count=1`) ? `"Failed"`.

The pure `_classify_execution_state(exe)` and the real `execution_terminal_state(name)` (which
does
its own `get_execution` fetch) **agreed on all 12** inspected executions (assert never fired).
Labels
seen: `{Succeeded, Failed}` (no Cancelled/running in the recent window ? the two branches that
drive
the fast-fail and the no-false-positive are both covered).

B ? fast-fail on a real crash-before-marker (rc 5, timed)

- Dispatched execution: **`rally-worker-tgbbf`**.
- Poll trail: `waiting (poll_every=10s, max_wait=600s)` ? poll 1 @6s *not ready* ? poll 2 @23s
  not ready (probe returned `None` ? exec still starting/running) ? **terminal `Failed`
  detected ? `RemoteExecutionFailed`** at **47.0 s** total.
- **`_dispatch_remote` returned rc 5** (RemoteExecutionFailed), not rc 4 (PolicyTimeout), not a
raw
  traceback.
- Diagnostic (verbatim):
  > `cloud-run: job=rally-worker execution=projects/gen-lang-client-0306026826/locations/asia-
southeast1/jobs/rally-worker/executions/rally-worker-tgbbf reached terminal state 'Failed' with
NO completion marker ? the worker exited before writing one (crash / OOM / pre-marker abort, or
a failed marker upload on an otherwise-Succeeded run). The execution is over, so the marker will
never appear; inspect the Cloud Run Job execution log to diagnose:`
  > `` `gcloud run jobs executions describe projects/gen-lang-client-0306026826/locations/asia-
southeast1/jobs/rally-worker/executions/rally-worker-tgbbf --region asia-southeast1`. ``
- Independent confirmation:
  - `executions describe rally-worker-tgbbf` ? `failedCount: 1`, condition `Completed=False`,
    `reason: NonZeroExitCode`, "Task ? failed with exit code: 1", `completionTime` set.
  - Container log shows the engineered pre-marker abort:
    `File ".../backend/worker/__main__.py", line 236, in cmd_infer / _apply_overrides / line 70
getattr(obj, p)`
    ? `AttributeError: 'AppConfig' object has no attribute 'zzz_smoke_crash'`.
  - `gs://khelsutra-rally-nightly/dev/smoke515/B/_dispatch/smokeB1.done` ? **does not exist**
    ("matched no objects"): a genuine terminal-Failed-**with-no-marker**.

C ? healthy dispatch still succeeds (rc 0)

- Dispatched execution: **`rally-worker-9tq5g`** on `mahadevpura_smoke_180s.mp4` (86 MiB, 180 s),
  native `wasb_hybrid`, `skip_ai_handoff=true`, `--skip-validation`.
- **`_dispatch_remote` returned rc RUN_C_RC** in **RUN_C_ELAPSED s**; `RUN_C_SEGMENTS` segments; the
  status probe returned `None` on **RUN_C_POLLS** running-polls (completion_time unset while the exec
  ran) and never false-aborted ? the SUCCESS marker resolved the poll normally.

Gotchas

- **`_fail()` writes a marker**, so a clean validation/segmenter failure is NOT the fast-fail path ?
  the smoke needed a genuine pre-marker *crash* (AttributeError in `_apply_overrides`), which the
  container traceback confirms.
- **Don't pass the crash `--set` to the outer `worker infer` CLI** ? its own `cmd_infer`
  `_apply_overrides` would crash the *local* dispatcher. Carry it in the SPEC's `config_overrides`
  (constructed `CloudRunCompute` + called `_dispatch_remote(compute, args)` directly).
- **Region:** the deployed Job is `asia-southeast1`; the module default `asia-south1` is wrong for
  this infra ? driven from the control-plane's `CLOUD_RUN_REGION` env.
- Harmless ADC "no quota project" `UserWarning` on every google-auth call from a user-cred ADC; the
  calls all succeeded.

Cost

- **A:** $0 (read-only `list_executions` + `get_execution`).
- **B:** one ~40 s **CPU-only** execution ? the worker crashed at config-parse **before** any GPU
  decode, so no L4 GPU-seconds were billed; effectively negligible.
- **C:** one real **L4** run on a 180 s clip (~`RUN_C_ELAPSED` s wall) ? a few cents.
- Total: well under $1.

Teardown / final state

- **No VM / disk** created (driver ran locally) ? nothing to delete; `gcloud compute instances list`
  N/A for this smoke.
- **No image pin** performed ? `rally-worker` image digest **unchanged** (`sha256:8045f54e?`);
  nothing to restore.
- Test objects under `gs://khelsutra-rally-nightly/dev/smoke515/` **deleted** after the run.
- No executions left running (B crashed; C completed).

93. user (2026-07-07T17:05:44.974Z)

File created successfully at: C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\docs\COMPUTE_DECOUPLED_SERVING\runlogs\RUNLOG_cloud-serving_exec-status-fastfail_2026-07-07.md (file state is current in your context ? no need to Read it back)

94. assistant (2026-07-07T17:06:06.205Z)

Runlog drafted (Test C numbers pending). Now I'll wait for the healthy GPU run to complete ? it's dispatched and polling cleanly.

Here's where things stand ? **2 of 3 pass-criteria groups already confirmed against live infra**:

? Core gap closed (the `# pragma: no cover` `get_execution` path). Against 12 real historical `rally-worker` executions, the actual `GoogleCloudRunJobClient.execution_terminal_state ? get_execution ? _classify_execution_state` chain works: `completion_time` marshals to a `DatetimeWithNanoseconds` (a `datetime`) when terminal and `None` while running ? so the `is None` terminality check is correct on the real

API (exactly the proto-plus point the pre-merge review flagged). Both **Succeeded** (`ql56s`, the CUJ green run) and **Failed** (`2lbz2`) classify correctly; direct-classify matched the get_execution path on all 12. **No field-name/marshaling mismatch ? no bug to file.**

? Fast-fail (pass-criteria 1?3). Dispatched `rally-worker-tgbbf` forced to crash before the marker (`AttributeError` in `_apply_overrides`, confirmed in the container traceback; exit 1, `failedCount:1`, marker object genuinely absent). The dispatcher fast-failed in **47.0 s** (polls at 6s/23s "not ready" ? terminal detected) ? **rc 5** (`RemoteExecutionFailed`, *not* rc 4 timeout, *not* ~65 min). The diagnostic named the execution, `terminal state 'Failed'`, and the exact `gcloud run jobs executions describe ? --region asia-southeast1` command.

? Healthy dispatch (pass-criterion 4). `rally-worker-9tq5g` running on `mahadevpura_smoke_180s.mp4` ? the status probe is returning `None` on each running-poll (no false-abort), waiting for the SUCCESS marker ? rc 0. Will finalize the runlog + memory + cleanup once it lands.

No image was pinned (crash reproduced via config on the current image), so there's nothing to restore; the worker digest is unchanged.

95. user (2026-07-07T17:07:10.600Z)

<task-notification>
<task-id>b6cpwpzsb</task-id>
<tool-use-id>toolu_012UBYLo9bf52qYQEsvRWTik</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\0c4fc269-564f-4fb2-9c1e-9389160b6be0\tasks\b6cpwpzsb.output</output-file>
<status>completed</status>
<summary>Background command "Run Test C healthy GPU dispatch (background)" completed (exit code 0)</summary>
</task-notification>

96. assistant (2026-07-07T17:07:13.008Z)

Test C completed. Let me read its result.

97. user (2026-07-07T17:07:15.676Z)

```
15      22:34:14 [DEBUG] compute.cloud_run: cloud-run-infer: poll 2, 28s elapsed, not ready
16      C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end user credentials from Google Cloud SDK without a quota project. You might receive a "quota exceeded" or "API not enabled" error. See the following page for troubleshooting: https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
17        warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
18      C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end user credentials from Google Cloud SDK without a quota project. You might receive a "quota exceeded" or "API not enabled" error. See the following page for troubleshooting: https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
19        warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
20      22:34:35 [DEBUG] compute.cloud_run: cloud-run-infer: poll 3, 49s elapsed, not ready
21      C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end user credentials from Google Cloud SDK without a quota project. You might receive a "quota exceeded" or "API not enabled" error. See the following page for troubleshooting: https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
22        warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
23      C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end user credentials from Google Cloud SDK without a quota project. You might receive a "quota exceeded" or "API not enabled" error. See the following page for troubleshooting: https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
24        warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
25      22:34:56 [DEBUG] compute.cloud_run: cloud-run-infer: poll 4, 70s elapsed, not ready
```

```
26      C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
27        warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
28      C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
29        warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
30      22:35:18 [DEBUG] compute.cloud_run: cloud-run-infer: poll 5, 92s elapsed, not ready
31      C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
32        warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
33      C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
34        warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
35      22:35:39 [DEBUG] compute.cloud_run: cloud-run-infer: poll 6, 113s elapsed, not ready
36      C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
37        warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
38      C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
39        warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
40      22:36:01 [DEBUG] compute.cloud_run: cloud-run-infer: poll 7, 135s elapsed, not ready
41      C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
42        warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
43      C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
44        warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
45      22:36:22 [DEBUG] compute.cloud_run: cloud-run-infer: poll 8, 156s elapsed, not ready
46      C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
47        warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
48      C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
49        warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
50      22:36:44 [DEBUG] compute.cloud_run: cloud-run-infer: poll 9, 178s elapsed, not ready
```

```
51      C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
52        warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
53      C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
54        warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
55      22:37:06 [INFO] compute.cloud_run: cloud-run-infer: ready after 9 poll(s) / 200s
56      C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
57        warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
58      22:37:10 [INFO] compute.cloud_run: cloud-run: job=rally-worker done
(video_id=05e0e577029daafcf425ca70c0c4f0a3 rc=0)
59      22:37:10 [INFO] backend.worker: [worker] remote compute done:
video_id=05e0e577029daafcf425ca70c0c4f0a3 rc=0 outputs=gs://khelsutra-rally-
nightly/dev/smoke515/C/outputs/05e0e577029daafcf425ca70c0c4f0a3/
60
61      === TEST C RESULT ===
62      returncode              = 0   (expect 0 = healthy success)
63      elapsed_sec             = 207.0
64      dispatched execution    = projects/gen-lang-client-0306026826/locations/asia-
southeast1/jobs/rally-worker/executions/rally-worker-9tq5g
65      running polls (probe=None while running) = 9
66      VERDICT                 = PASS
67
```

98. assistant (2026-07-07T17:07:27.322Z)

Test C PASS ? rc 0. The status probe returned `None` on all **9 running-polls** across ~200s of a genuinely-running execution (completion_time unset ? classifier `None`), never false-aborting, then the SUCCESS marker resolved the poll ? rc 0. All four pass-criteria are now met.

Let me grab the segment count for the runlog and confirm the worker image digest is unchanged, then finalize.

99. user (2026-07-07T17:07:36.844Z)

```
=== Test C report.json ===
status: success | segment_count: 19

=== worker image digest NOW (expect unchanged 8045f54e) ===
asia-southeast1-docker.pkg.dev/gen-lang-client-0306026826/rally/rally-
worker@sha256:8045f54ea9fcbf52426305ab987f122f4fc8bed6b13be93283d21c5cab57a655
```

100. assistant (2026-07-07T17:07:48.045Z)

Test C: **status=success, 19 rally segments** (a real detection, genuine healthy success). Worker image digest **unchanged** (`sha256:8045f54e?`) ? no pin, nothing to restore. Let me finalize the runlog.

101. user (2026-07-07T17:07:51.642Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\docs\COMPUTE_DECOUPLED_SERVING\runlogs\RUNLOG_cloud-serving_exec-status-
fastfail_2026-07-07.md has been updated successfully. (file state is current in your context ?
no need to Read it back)
```

102. user (2026-07-07T17:08:01.365Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\docs\COMPUTE_DECOUPLED_SERVING\runlogs\RUNLOG_cloud-serving_exec-status-
fastfail_2026-07-07.md has been updated successfully. (file state is current in your context ?
no need to Read it back)
```

103. user (2026-07-07T17:08:05.931Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\docs\COMPUTE_DECOUPLED_SERVING\runlogs\RUNLOG_cloud-serving_exec-status-
fastfail_2026-07-07.md has been updated successfully. (file state is current in your context ?
no need to Read it back)
```

104. assistant (2026-07-07T17:08:08.572Z)

Now the index row in the runlogs README:

105. user (2026-07-07T17:08:15.947Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\docs\COMPUTE_DECOUPLED_SERVING\runlogs\README.md has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

106. assistant (2026-07-07T17:08:21.264Z)

Now cleanup the test objects. Let me list what's under my test prefix, then delete it.

107. user (2026-07-07T17:08:25.244Z)

```
=== contents under smoke515 (before delete) ===
gs://khelsutra-rally-nightly/dev/smoke515/C/_dispatch/smokeC1.done
gs://khelsutra-rally-
nightly/dev/smoke515/C/outputs/05e0e577029daafcf425ca70c0c4f0a3/intervals.csv
gs://khelsutra-rally-nightly/dev/smoke515/C/outputs/05e0e577029daafcf425ca70c0c4f0a3/report.json
```

108. assistant (2026-07-07T17:08:30.939Z)

Only Test C's objects exist (Test B correctly wrote nothing ? confirming crash-before-marker). Let me delete them and verify no executions are left running.

109. user (2026-07-07T17:08:53.657Z)

```
=== deleting smoke515 test objects ===
Removing objects:

Removing gs://khelsutra-rally-nightly/dev/smoke515/C/_dispatch/smokeC1.done#1783444004661045...
Removing gs://khelsutra-rally-nightly/dev/smoke515/C/outputs/05e0e577029daafcf425ca70c0c4f0a3/in
tervals.csv#1783444003863957...
Removing gs://khelsutra-rally-
nightly/dev/smoke515/C/outputs/05e0e577029daafcf425ca70c0c4f0a3/report.json#1783444004122420...
.........................

=== verify prefix empty ===
ERROR: (gcloud.storage.ls) One or more URLs matched no objects.

=== confirm my two test executions are terminal (not running) ===
rally-worker-tgbbf: 2026-07-07T17:02:04.425001Z         1
rally-worker-9tq5g: 2026-07-07T17:06:52.479313Z 1
```

110. assistant (2026-07-07T17:09:01.076Z)

Cleanup confirmed: smoke515 objects deleted (prefix now empty), both test executions terminal

(`tgbbf` Failed / `9tq5g` Succeeded), none running, worker image unchanged. Let me mark Test C done and update the memory.

111. user (2026-07-07T17:09:01.466Z)

Updated task #3 status

112. user (2026-07-07T17:09:28.545Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\cloud-run-exec-status-fastfail.md has been updated successfully. (file state is current in your context ? no need to Read it back)

113. assistant (2026-07-07T17:09:46.623Z)

Now let me dismiss the satisfied follow-up task and verify no placeholders remain in the runlog.

114. user (2026-07-07T17:09:48.771Z)

No matches found

115. user (2026-07-07T17:09:52.334Z)

No pending task with id task_069a7a19. Task ids are not persisted across app restarts, so a chip from before a restart can no longer be withdrawn. Nothing was changed.

116. assistant (2026-07-07T17:09:59.897Z)

The prior-session task chip can't be withdrawn across restarts (noted as satisfied in the memory instead). I made **no code changes** ? only a docs runlog + README index row (+ the out-of-repo memory) ? so the code pre-merge gate doesn't apply. Let me confirm the working-tree state so I can report exactly what changed.

117. user (2026-07-07T17:10:02.447Z)

```
=== branch ===
feat/validation-result-cache
=== status (docs-only changes expected) ===
 M backend/infrastructure/database.py
 M backend/infrastructure/database_media.py
 M backend/orchestration.py
 M backend/pipeline/validators/base.py
 M docs/COMPUTE_DECOUPLED_SERVING/runlogs/README.md
?? docs/COMPUTE_DECOUPLED_SERVING/runlogs/RUNLOG_cloud-serving_exec-status-fastfail_2026-07-07.md
?? score_golden_gate.py
?? score_local.py
=== diff stat ===
 backend/infrastructure/database.py               | 34 ++++++++++-
 backend/infrastructure/database_media.py         | 62 ++++++++++++++++++++-
 backend/orchestration.py                         | 58 ++++++++++++++++---
 backend/pipeline/validators/base.py              | 50 ++++++++++++++++
 docs/COMPUTE_DECOUPLED_SERVING/runlogs/README.md |  1 +
 5 files changed, 198 insertions(+), 7 deletions(-)
```

118. assistant (2026-07-07T17:11:38.605Z)

?? **Important concurrent-session finding.** At session start the checkout was on `master` (clean, only 2 untracked dev tools). It is now on branch **`feat/validation-result-cache`** with **uncommitted backend changes I did not make** (`database.py`, `database_media.py`, `orchestration.py`, `validators/base.py`, +198 lines). A sibling session switched the branch and is doing feature work in this shared checkout mid-smoke.

This does **not** affect my smoke's validity: the code under test (`cloud_run.py`) and `_dispatch_remote` are unmodified on both branches, and the actual behavior came from the deployed worker image (`8045f54e`) + the real Cloud Run API ? not the sibling's local edits. But per my branching-safety rules I must not entangle my work with, or clobber, their WIP. I'll leave their 4 files untouched and pull my tracked footprint (the README index-row edit) back off their branch, keeping only the untracked runlog file.

119. user (2026-07-07T17:11:49.959Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\docs\COMPUTE_DECOUPLED_SERVING\runlogs\README.md has been updated successfully. (file state is current in your context ? no need to Read it back)

120. user (2026-07-07T17:11:58.498Z)

```
=== branch ===
feat/validation-result-cache
=== status ===
 M backend/infrastructure/database.py
 M backend/infrastructure/database_media.py
 M backend/orchestration.py
 M backend/pipeline/validators/base.py
?? docs/COMPUTE_DECOUPLED_SERVING/runlogs/RUNLOG_cloud-serving_exec-status-fastfail_2026-07-07.md
?? score_golden_gate.py
?? score_local.py
=== confirm README has no diff (my edit reverted) ===
(empty above = clean)
```

121. assistant (2026-07-07T17:12:13.145Z)

My tracked footprint on the sibling branch is now zero ? only the untracked runlog remains (the 4 backend files are the sibling's, untouched). Let me add a short landing-status note to the runlog and reflect the uncommitted state in memory, then close out.

122. user (2026-07-07T17:12:18.994Z)

The file C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\docs\COMPUTE_DECOUPLED_SERVING\runlogs\RUNLOG_cloud-serving_exec-status-fastfail_2026-07-07.md has been updated successfully. (file state is current in your context ? no need to Read it back)

123. user (2026-07-07T17:12:31.089Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\cloud-run-exec-status-fastfail.md has been updated successfully. (file state is current in your context ? no need to Read it back)

124. user (2026-07-07T17:12:33.262Z)

Updated task #4 status

125. assistant (2026-07-07T17:13:07.748Z)

All four pass-criteria are met and everything is cleaned up. Here's the final report.

Live smoke of the #515 crash-before-marker fast-fail ? **PASS** ?

I drove the smoke **locally** (Win, Python 3.13, `google-cloud-run` + owner ADC, importing local master `ef41473`) against the live Job `rally-worker` in **asia-southeast1** ? constructing `CloudRunCompute` directly and calling the real `worker._dispatch_remote(compute, args)`, so no control-plane / CF-Access JWT was needed. **No worker image was pinned** (I reproduced the crash via config on the current image), so nothing needed restoring.

The `# pragma: no cover` gap is closed ? real API matches the unit-test assumptions
Against 12 of 23 historical executions, the actual `execution_terminal_state ? get_execution ? _classify_execution_state` chain works live:
- `completion_time` marshals to a **`DatetimeWithNanoseconds`** (a `datetime`) when terminal, `None` while running ? the `completion_time is None` terminality test is correct (validates the proto-plus fix ? *not* `.seconds`). `succeeded/failed/cancelled_count` are plain ints.
- Both **Succeeded** (`ql56s`, the CUJ green run) and **Failed** (`2lbz2`) classify correctly; direct-classify == the `get_execution` path on all 12. **No field-name/marshaling bug ? nothing to file.**

Pass criteria
#	Criterion	Result
1	Fast-fail within ~one poll interval (not ~65 min)	? **47.0 s** (`rally-worker-tgbbf`; polls at 6s/23s "not ready" ? terminal detected)
2	rc **5** (`RemoteExecutionFailed`), not rc 4, not a traceback	? `_dispatch_remote` returned **5**
3	Diagnostic names execution + terminal state + `gcloud ? describe ? --region`	? verbatim, naming `terminal state 'Failed'` and `gcloud run jobs executions describe ? --region asia-southeast1`
4	Healthy dispatch still rc 0 (no false-positive)	? **rc 0** (`rally-worker-9tq5g`, 19 rallies); probe returned `None` on all **9** running-polls, never aborting

Independent confirmation of the crash: `tgbbf` ? `failedCount:1` / `NonZeroExitCode`; container traceback shows the engineered `AttributeError: 'AppConfig' object has no attribute 'zzz_smoke_crash'` in `_apply_overrides` (before any marker); the marker object genuinely does not exist.

Honest scope note on criterion 2: I exercised the **worker-CLI** rc-5 mapping directly (authentic code path) and captured the exact `RemoteExecutionFailed` text. The control-plane `/api/process` wrapping of that same text into `"cloud dispatch error: ?"` is by code inspection ([orchestration.py:644](backend/orchestration.py:644)) ? that path needs a CF-Access JWT, which the handoff notes isn't available this session. Also: I used region **asia-southeast1** (the deployed env), not the module default `asia-south1`.

Cost & cleanup
- Cost < $1: probe = $0; crash = one ~40s **CPU-only** execution (crashed before GPU); healthy = one ~207s L4 run.
- `gs://khelsutra-rally-nightly/dev/smoke515/**` deleted; no executions left running; worker image digest unchanged (`sha256:8045f54e?`); no VM created.

Two things for your attention
1. **Runlog written** at [RUNLOG_cloud-serving_exec-status-fastfail_2026-07-07.md](docs/COMPUTE_DECOUPLED_SERVING/runlogs/RUNLOG_cloud-serving_exec-status-fastfail_2026-07-07.md), and the memory [[cloud-run-exec-status-fastfail]] updated. It is **untracked (not committed)** ? see below.
2. ?? **Concurrent-session hazard:** the shared checkout was on `master` at session start but a sibling session switched it to branch **`feat/validation-result-cache`** mid-smoke, carrying uncommitted WIP in `database.py`, `database_media.py`, `orchestration.py`, `validators/base.py` (+198 lines, not mine). I **left those untouched** and reverted my README index-row edit so I leave **zero tracked footprint** on their branch. The smoke results are unaffected (the code under test `cloud_run.py` is identical on both branches, and behavior came from the deployed image + real API).

Want me to land the runlog properly? I'd cut a fresh branch off `origin/master`, add the runlog + README index row, and open a docs-only PR ? leaving the sibling's branch alone.

126. user (2026-07-07T17:18:39.622Z)

yes please open a separate branch and send the PR and then merge on all green

127. assistant (2026-07-07T17:19:32.282Z)

On it. Given the confirmed concurrent-session hazard, I will **not** switch the shared

checkout's branch (that would hijack HEAD from the sibling working on `feat/validation-result-cache`). Instead I'll use an **isolated git worktree** off `origin/master`, commit only my two docs files there, and open + merge the PR from that worktree ? leaving the shared checkout and the sibling's WIP completely untouched.

Let me verify tooling and get the latest remote state first.

128. user (2026-07-07T17:19:40.748Z)

```
=== gh auth ===
github.com
  ? Logged in to github.com account avidullu (keyring)
  - Active account: true
  - Git operations protocol: https
  - Token: gho_**********************************
  - Token scopes: 'gist', 'read:org', 'repo', 'workflow'
=== remote ===
origin  https://github.com/Khelsutra/badminton-highlight-indexer.git (fetch)
origin  https://github.com/Khelsutra/badminton-highlight-indexer.git (push)
=== fetch origin ===
=== default branch ===
master
=== origin/master head ===
ef41473 fix(cloud-run): fail fast when a worker execution terminates without a done-marker
(#515)
```

129. assistant (2026-07-07T17:19:59.159Z)

All tooling checks out (gh as `avidullu` with `repo`+`workflow` scopes; default branch `master`; `origin/master` at `ef41473`). Let me create an isolated worktree off `origin/master`.

130. user (2026-07-07T17:20:04.970Z)

```
Preparing worktree (new branch 'docs/runlog-cloud-run-exec-status-fastfail')
branch 'docs/runlog-cloud-run-exec-status-fastfail' set up to track 'origin/master'.
HEAD is now at ef41473 fix(cloud-run): fail fast when a worker execution terminates without a
done-marker (#515)
=== worktree branch/head ===
docs/runlog-cloud-run-exec-status-fastfail
ef41473 fix(cloud-run): fail fast when a worker execution terminates without a done-marker
(#515)
=== shared checkout still on sibling branch (untouched) ===
feat/validation-result-cache
```

131. assistant (2026-07-07T17:20:12.811Z)

Worktree is on the new branch at `ef41473`; the shared checkout stays on `feat/validation-result-cache`, untouched. Now let me copy the runlog into the worktree and adjust its landing note.

132. user (2026-07-07T17:20:16.950Z)

```
copied
-rw-r--r-- 1 avidu 197609 11352 Jul  7 22:50 RUNLOG_cloud-serving_exec-status-
fastfail_2026-07-07.md
```

133. assistant (2026-07-07T17:20:24.479Z)

Now let me update the landing note in the worktree copy (it's being committed now, so the "uncommitted" framing is stale) and add the README index row. Let me read both worktree files.

134. user (2026-07-07T17:20:26.357Z)

```
108     ### C ? healthy dispatch still succeeds (rc 0)
```

```
109     - Dispatched execution: **`rally-worker-9tq5g`** on `mahadevpura_smoke_180s.mp4` (86
MiB, 180 s),
110       native `wasb_hybrid`, `skip_ai_handoff=true`, `--skip-validation`.
111     - **`_dispatch_remote` returned rc 0** in **207 s** (`video_id=05e0e577?`, `report.json`
112       `status=success`, **19** rally segments) ? a real WASB detection, not an empty pass.
113     - The status probe was called on **9** running-polls (poll_every=15s over ~200s) and
returned
114       `None` **every time** while the exec ran (completion_time unset), never false-aborting
? the
115       SUCCESS marker then resolved the poll normally (`ready after 9 poll(s) / 200s`). This
is the
116       decisive no-false-positive check: the proto-plus "unset completion_time ? `None`"
behavior holds
117       on the real API for a genuinely-running execution.
118
119     ## Gotchas
120     - **`_fail()` writes a marker**, so a clean validation/segmenter failure is NOT the
fast-fail path ?
121       the smoke needed a genuine pre-marker *crash* (AttributeError in `_apply_overrides`),
which the
122       container traceback confirms.
123     - **Don't pass the crash `--set` to the outer `worker infer` CLI** ? its own `cmd_infer`
124       `_apply_overrides` would crash the *local* dispatcher. Carry it in the SPEC's
`config_overrides`
125       (constructed `CloudRunCompute` + called `_dispatch_remote(compute, args)` directly).
126     - **Region:** the deployed Job is `asia-southeast1`; the module default `asia-south1` is
wrong for
127       this infra ? driven from the control-plane's `CLOUD_RUN_REGION` env.
128     - Harmless ADC "no quota project" `UserWarning` on every google-auth call from a user-
cred ADC; the
129       calls all succeeded.
130
131     ## Cost
132     - **A:** $0 (read-only `list_executions` + `get_execution`).
133     - **B:** one ~40 s **CPU-only** execution ? the worker crashed at config-parse
**before** any GPU
134       decode, so no L4 GPU-seconds were billed; effectively negligible.
135     - **C:** one real **L4** run on a 180 s clip (~207 s wall, 19 rallies detected) ? a few
cents.
136     - Total: well under $1.
137
```

135. user (2026-07-07T17:20:32.647Z)

```
137
138     ## Teardown / final state
139     - **No VM / disk** created (driver ran locally) ? nothing to delete; `gcloud compute
instances list`
140       N/A for this smoke.
141     - **No image pin** performed ? `rally-worker` image digest **unchanged**
(`sha256:8045f54e?`);
142       nothing to restore.
143     - Test objects under `gs://khelsutra-rally-nightly/dev/smoke515/` **deleted** after the
run.
144     - No executions left running (B crashed ? terminal `Failed`; C completed ? terminal
`Succeeded`).
145
146     ## Landing status (uncommitted)
147     This runlog is **untracked / not committed**. During the smoke the shared checkout was
switched (by a
148     concurrent session) from `master` onto branch **`feat/validation-result-cache`**, which
carries an
149     unrelated sibling's uncommitted WIP (`database.py`, `database_media.py`,
`orchestration.py`,
150     `validators/base.py`). To avoid entangling this docs record with that WIP, no commit was
```

```
made and the
151     `runlogs/README.md` index row was reverted. **To land durably:** cut a branch off
`origin/master`,
152     add this file + the README index row there, and open a PR (docs-only ? no code pre-merge
gate needed).
153
```

136. user (2026-07-07T17:20:40.542Z)

```
The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\0c4fc269-564f-4fb2-9c1e-9389160b6be0\scratchpad\wt-
runlog\docs\COMPUTE_DECOUPLED_SERVING\runlogs\RUNLOG_cloud-serving_exec-status-
fastfail_2026-07-07.md has been updated successfully. (file state is current in your context ?
no need to Read it back)
```

137. assistant (2026-07-07T17:20:44.156Z)

Now the README index row in the worktree. Let me read its index table.

138. user (2026-07-07T17:20:45.880Z)

```
32      ## Index (append as runs land)
33      | Date | Profile | Result | File |
34      |---|---|---|---|
35      | 2026-07-06 | cloud-serving | alpha-tester "All 1 v 1" timed out ? downsample fix
proven on L4 (720p30: 956 s, 60 rallies) | [DEPLOY_REPORT_cloud-serving_alpha-
all1v1_2026-07-06.md](DEPLOY_REPORT_cloud-serving_alpha-all1v1_2026-07-06.md) |
36      | 2026-07-07 | cloud-serving | GPU-resident NVDEC decode (#506/#507) shipped: image bump
+ cutover, cpu/nvdec smokes, GX010128 trajectory reproduced VM (26479?26481 vis), enabled +
canaried | [DEPLOY_REPORT_cloud-serving_gpu-resident-nvdec_2026-07-07.md](DEPLOY_REPORT_cloud-
serving_gpu-resident-nvdec_2026-07-07.md) |
37
```

139. user (2026-07-07T17:20:54.995Z)

```
The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\0c4fc269-564f-4fb2-9c1e-9389160b6be0\scratchpad\wt-
runlog\docs\COMPUTE_DECOUPLED_SERVING\runlogs\README.md has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

140. assistant (2026-07-07T17:21:09.058Z)

Both docs files are ready in the worktree. Let me stage them explicitly (never `-A`), commit, and verify the branch carries only my commit vs `origin/master`.

141. user (2026-07-07T17:21:18.882Z)

```
warning: in the working copy of 'docs/COMPUTE_DECOUPLED_SERVING/runlogs/RUNLOG_cloud-
serving_exec-status-fastfail_2026-07-07.md', LF will be replaced by CRLF the next time Git
touches it
=== staged ===
M  docs/COMPUTE_DECOUPLED_SERVING/runlogs/README.md
A  docs/COMPUTE_DECOUPLED_SERVING/runlogs/RUNLOG_cloud-serving_exec-status-
fastfail_2026-07-07.md
=== HEAD ===
5df8ff2 docs(runlogs): record live smoke of the crash-before-marker fast-fail (#515)
=== branching-safety: commits vs origin/master (must be ONLY mine) ===
5df8ff2 docs(runlogs): record live smoke of the crash-before-marker fast-fail (#515)
=== files in this commit (must be docs-only) ===
5df8ff2 docs(runlogs): record live smoke of the crash-before-marker fast-fail (#515)
 docs/COMPUTE_DECOUPLED_SERVING/runlogs/README.md   |   1 +
 ...loud-serving_exec-status-fastfail_2026-07-07.md | 151 ++++++++++++++++++++
 2 files changed, 152 insertions(+)
```

142. assistant (2026-07-07T17:21:33.423Z)

```
Branching safety verified: commit `5df8ff2` is the **only** commit vs `origin/master`,
containing exactly the 2 docs files. Let me push and open the PR.
```

143. user (2026-07-07T17:21:40.702Z)

```
remote:
remote: Create a pull request for 'docs/runlog-cloud-run-exec-status-fastfail' on GitHub by
visiting:
remote:      https://github.com/Khelsutra/badminton-highlight-indexer/pull/new/docs/runlog-
cloud-run-exec-status-fastfail
remote:
branch 'docs/runlog-cloud-run-exec-status-fastfail' set up to track 'origin/docs/runlog-cloud-
run-exec-status-fastfail'.
To https://github.com/Khelsutra/badminton-highlight-indexer.git
 * [new branch]      docs/runlog-cloud-run-exec-status-fastfail -> docs/runlog-cloud-run-exec-
status-fastfail
```

144. user (2026-07-07T17:21:56.988Z)

```
https://github.com/Khelsutra/badminton-highlight-indexer/pull/517
```

145. assistant (2026-07-07T17:22:03.344Z)

```
PR opened: **https://github.com/Khelsutra/badminton-highlight-indexer/pull/517**. Now let me
check its CI status.
```

146. user (2026-07-07T17:22:10.153Z)

```
=== PR state/mergeability ===
{"draft":false,"mergeState":"UNSTABLE","mergeable":"MERGEABLE","review":"","state":"OPEN"}
=== checks ===
Full test suite (offline, mocked)       pending 0       https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28885365017/job/85684058558
Golden fixtures cross-OS (ubuntu-latest)        pending 0
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28885365017/job/85684058528
Leak scan (PII / attribution guard)     pass    6s      https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28885365017/job/85684058484
Markdown link check (live docs) pass    6s      https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28885365017/job/85684058573
Built wheel smoke (install + launch)    pending 0       https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28885365017/job/85684058542
Dependency license scan (no copyleft in runtime deps)   pending 0
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28885365017/job/85684058531
Golden fixtures cross-OS (macos-latest) pending 0       https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28885365017/job/85684058517
Golden fixtures cross-OS (windows-latest)       pending 0
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28885365017/job/85684058548
Mypy (lenient baseline) pending 0       https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28885365017/job/85684058526
Ruff lint       pending 0       https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28885365017/job/85684058515
Syntax sweep + import smoke (no GPU stack)      pending 0
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28885365017/job/85684058574
```

147. assistant (2026-07-07T17:22:23.221Z)

```
All 11 CI checks kicked off (2 already green: leak scan + markdown link check); PR is
**MERGEABLE**, no required review blocking, mergeState `UNSTABLE` only because checks are still
```

running. Let me set it to auto-merge (squash) so it merges exactly when everything goes green.

148. user (2026-07-07T17:22:25.037Z)

Task #5 created successfully: Merge PR #517 (docs runlog) when CI is all green

149. user (2026-07-07T17:22:28.803Z)

Updated task #5 status

150. user (2026-07-07T17:22:43.191Z)

```
 * branch            master     -> FETCH_HEAD
   ef41473..115b83d  master     -> origin/master
Updating ef41473..115b83d
Fast-forward
 docs/COMPUTE_DECOUPLED_SERVING/runlogs/README.md   |   1 +
 ...loud-serving_exec-status-fastfail_2026-07-07.md | 151 ++++++++++++++++++++
 2 files changed, 152 insertions(+)
 create mode 100644 docs/COMPUTE_DECOUPLED_SERVING/runlogs/RUNLOG_cloud-serving_exec-status-
fastfail_2026-07-07.md
```

151. assistant (2026-07-07T17:23:04.756Z)

The output suggests the merge may have already gone through (no required status checks gating). Let me verify the PR state and ? importantly ? whether CI actually passed, since you asked to merge on green.

152. user (2026-07-07T17:23:09.127Z)

```
Exit code 1
=== PR #517 state ===
Unknown JSON field: "merged"
Available fields:
  additions
  assignees
  author
  autoMergeRequest
  baseRefName
  baseRefOid
  body
  changedFiles
  closed
  closedAt
  closingIssuesReferences
  comments
  commits
  createdAt
  deletions
  files
  fullDatabaseId
  headRefName
  headRefOid
  headRepository
  headRepositoryOwner
  id
  isCrossRepository
  isDraft
  labels
  latestReviews
  maintainerCanModify
  mergeCommit
  mergeStateStatus
  mergeable
```

```
  mergedAt
  mergedBy
  milestone
  number
  potentialMergeCommit
  projectCards
  projectItems
  reactionGroups
  reviewDecision
  reviewRequests
  reviews
  state
  statusCheckRollup
  title
  updatedAt
  url
```

153. user (2026-07-07T17:23:19.102Z)

```
=== PR #517 ===
{"autoMerge":null,"commit":"115b83dd710b7f8ba3e08a5151a19824da313470","mergedAt":"2026-07-
07T17:22:37Z","mergedBy":"avidullu","state":"MERGED"}
=== check rollup (name/state/conclusion) ===
Ruff lint: COMPLETED SUCCESS
Leak scan (PII / attribution guard): COMPLETED SUCCESS
Markdown link check (live docs): COMPLETED SUCCESS
Mypy (lenient baseline): COMPLETED SUCCESS
Syntax sweep + import smoke (no GPU stack): COMPLETED SUCCESS
Full test suite (offline, mocked): IN_PROGRESS
Golden fixtures cross-OS (ubuntu-latest): COMPLETED SUCCESS
Golden fixtures cross-OS (windows-latest): COMPLETED SUCCESS
Golden fixtures cross-OS (macos-latest): COMPLETED SUCCESS
Dependency license scan (no copyleft in runtime deps): COMPLETED SUCCESS
Built wheel smoke (install + launch): COMPLETED SUCCESS
```

154. assistant (2026-07-07T17:23:54.540Z)

```
The PR **merged** (squash ? `115b83d`) ? but I need to flag something honestly: `--auto` merged
as soon as the PR was mergeable because the repo's branch protection doesn't *require* the
status checks. At merge time **10 of 11 checks were green** (Ruff, Mypy, all 3 golden-fixtures
OSes, wheel smoke, license scan, syntax sweep, leak scan, markdown links), but **"Full test
suite (offline, mocked)" was still running**. It's a docs-only diff (2 markdown files) so it
can't affect the suite, but that's not literally "on all green" yet. Let me watch that last
check to completion and confirm it's green, and clean up the shared checkout in parallel.
```

155. user (2026-07-07T17:23:57.289Z)

```
Command running in background with ID: bo4a10bws. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\0c4fc269-564f-4fb2-9c1e-9389160b6be0\tasks\bo4a10bws.output. You will be notified when
it completes. To check interim output, use Read on that file path.
```

156. assistant (2026-07-07T17:24:01.750Z)

```
While CI finishes, let me remove the now-redundant untracked runlog from the shared checkout
(it's committed on master now) and confirm the sibling's WIP is still intact.
```

157. user (2026-07-07T17:24:05.513Z)

```
removed untracked runlog from shared checkout
=== shared checkout status (sibling WIP must be intact, no stray runlog) ===
feat/validation-result-cache
 M backend/infrastructure/database.py
 M backend/infrastructure/database_media.py
```

```
 M backend/orchestration.py
 M backend/pipeline/validators/base.py
 M tests/test_async_jobs.py
 M tests/test_database.py
 M tests/test_job_worker_loop.py
 M tests/test_locale_analytics.py
?? score_golden_gate.py
?? score_local.py
?? tests/test_validation_cache.py
```

158. assistant (2026-07-07T17:24:18.844Z)

The sibling's WIP is fully intact (they've since added more test files ? actively working) and my stray runlog is gone; **zero contamination**. Let me finish cleanup by removing the worktree while CI runs.

159. user (2026-07-07T17:24:28.815Z)

```
=== worktrees before ===
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer
ef41473 [feat/validation-result-cache]
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/0c4fc269-564f-4fb2-9c1e-9389160b6be0/scratchpad/wt-runlog     115b83d [master]
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/cfg-wt        20b2d5a [claude/wasb-config-validation]
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/compile-wt    b09a1d4 [claude/compile-reliability]
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/pr458-pre-wt  355dad6 (detached HEAD)
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/pr458-wt      5de1ef2 (detached HEAD)
error: failed to delete '.git/worktrees/a1-master-worktree': Permission denied
error: failed to delete '.git/worktrees/bhi-wt-worker-reval': Permission denied
error: failed to delete '.git/worktrees/fix-worktree': Permission denied
error: failed to delete '.git/worktrees/master-compare': Permission denied
error: failed to delete '.git/worktrees/pr465': Permission denied
error: failed to delete '.git/worktrees/pr465-current-merge': Permission denied
error: failed to delete '.git/worktrees/pr465-merge': Permission denied
error: failed to delete '.git/worktrees/pr467': Permission denied
error: failed to delete '.git/worktrees/pr469': Permission denied
error: failed to delete '.git/worktrees/wt-465fix': Permission denied
error: failed to delete '.git/worktrees/wt-a1-runlog': Permission denied
error: failed to delete '.git/worktrees/wt-claudemd-r25': Permission denied
error: failed to delete '.git/worktrees/wt-cloudrun-cancel': Permission denied
error: failed to delete '.git/worktrees/wt-cloudrun-dispatch': Permission denied
error: failed to delete '.git/worktrees/wt-cp-rebuild-spec': Permission denied
error: failed to delete '.git/worktrees/wt-dockerfile-auth': Permission denied
error: failed to delete '.git/worktrees/wt-docs': Permission denied
error: failed to delete '.git/worktrees/wt-docs-reconcile': Permission denied
error: failed to delete '.git/worktrees/wt-drain-concurrency': Permission denied
error: failed to delete '.git/worktrees/wt-ffmpeg-timeouts': Permission denied
error: failed to delete '.git/worktrees/wt-gdrive-api-safety': Permission denied
error: failed to delete '.git/worktrees/wt-idempotent-process': Permission denied
error: failed to delete '.git/worktrees/wt-input-jail': Permission denied
error: failed to delete '.git/worktrees/wt-output-dir-config': Permission denied
error: failed to delete '.git/worktrees/wt-p1-genyaml': Permission denied
error: failed to delete '.git/worktrees/wt-p2-upload-stream': Permission denied
error: failed to delete '.git/worktrees/wt-p4-md5cache': Permission denied
error: failed to delete '.git/worktrees/wt-p5a-pollknob': Permission denied
error: failed to delete '.git/worktrees/wt-p6-heartbeat': Permission denied
error: failed to delete '.git/worktrees/wt-pr481-review': Permission denied
error: failed to delete '.git/worktrees/wt-retention-sweep': Permission denied
error: failed to delete '.git/worktrees/wt-review-492-mergecheck': Permission denied
```

```
error: failed to delete '.git/worktrees/wt-review-493-mergecheck': Permission denied
error: failed to delete '.git/worktrees/wt-scratch-reclaim': Permission denied
error: failed to delete '.git/worktrees/wt-signed-download': Permission denied
error: failed to delete '.git/worktrees/wt-source-cache': Permission denied
error: failed to delete '.git/worktrees/wt-upload-dir': Permission denied
error: failed to delete '.git/worktrees/wt-upload-staging': Permission denied
=== branch cleanup ===
error: branch 'docs/runlog-cloud-run-exec-status-fastfail' not found
(local branch already gone via --delete-branch)
=== worktrees after ===
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer
ef41473 [feat/validation-result-cache]
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/cfg-wt        20b2d5a [claude/wasb-
config-validation]
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/compile-wt    b09a1d4 [claude/compile-
reliability]
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/pr458-pre-wt  355dad6 (detached HEAD)
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/pr458-wt      5de1ef2 (detached HEAD)
```

160. assistant (2026-07-07T17:24:42.046Z)

My worktree `wt-runlog` was removed cleanly and the local branch is gone (the permission-denied noise is git trying to prune *other* sessions' stale worktree registrations, which I don't own and correctly left alone). Cleanup is complete; the sibling's WIP is untouched.

Now waiting on the final CI check (the full test suite) to confirm green ? the watcher will notify me when the run finishes.

161. user (2026-07-07T17:24:44.713Z)

```
1       Refreshing run status every 3 seconds. Press Ctrl+C to quit.
2
3       * docs/runlog-cloud-run-exec-status-fastfail CI Khelsutra/badminton-highlight-
indexer#517 · 28885365017
4       Triggered via pull_request about 2 minutes ago
5
6       JOBS
7       ? Leak scan (PII / attribution guard) in 6s (ID 85684058484)
8         ? Set up job
9         ? Run actions/checkout@v6
10        ? Run actions/setup-python@v6
11        ? Leak scan
12        ? Post Run actions/setup-python@v6
13        ? Post Run actions/checkout@v6
14        ? Complete job
15      ? Ruff lint in 7s (ID 85684058515)
16        ? Set up job
17        ? Run actions/checkout@v6
18        ? Run actions/setup-python@v6
19        ? Install ruff
20        ? Ruff check
21        ? Post Run actions/setup-python@v6
22        ? Post Run actions/checkout@v6
23        ? Complete job
24      ? Golden fixtures cross-OS (macos-latest) in 14s (ID 85684058517)
25        ? Set up job
26        ? Run actions/checkout@v6
27        ? Run actions/setup-python@v6
28        ? Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
29        ? Golden-fixtures characterization (cross-OS determinism guard)
30        ? Post Run actions/setup-python@v6
```

```
      ? Post Run actions/checkout@v6
      ? Complete job
    ? Mypy (lenient baseline) in 31s (ID 85684058526)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
      ? Install mypy + typed core deps
      ? Mypy
      ? Post Run actions/setup-python@v6
      ? Post Run actions/checkout@v6
      ? Complete job
    ? Golden fixtures cross-OS (ubuntu-latest) in 25s (ID 85684058528)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
      ? Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
      ? Golden-fixtures characterization (cross-OS determinism guard)
      ? Post Run actions/setup-python@v6
      ? Post Run actions/checkout@v6
      ? Complete job
    ? Dependency license scan (no copyleft in runtime deps) in 35s (ID 85684058531)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
      ? Install runtime deps + pip-licenses
      ? Report dependency licenses
      ? Fail on any copyleft (GPL/AGPL/LGPL/SSPL) in the runtime closure
      ? Post Run actions/setup-python@v6
      ? Post Run actions/checkout@v6
      ? Complete job
    ? Built wheel smoke (install + launch) in 1m10s (ID 85684058542)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
      ? System libs for opencv-python
      ? Build the wheel
      ? Install the built wheel + CPU torch (out-of-band index)
      ? Launch the installed console script from a non-repo dir
      ? Post Run actions/setup-python@v6
      ? Post Run actions/checkout@v6
      ? Complete job
    ? Golden fixtures cross-OS (windows-latest) in 55s (ID 85684058548)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
      ? Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
      ? Golden-fixtures characterization (cross-OS determinism guard)
      ? Post Run actions/setup-python@v6
      ? Post Run actions/checkout@v6
      ? Complete job
    * Full test suite (offline, mocked) (ID 85684058558)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
      ? System libs for opencv-python
      ? Install CPU-only torch (no CUDA wheels on a CPU runner)
      ? Install trackers (git-only dep ? PyPI wheel is broken)
      ? Install package (editable) + dev tooling
      * Run full suite (with coverage floor)
      * Post Run actions/setup-python@v6
      * Post Run actions/checkout@v6
    ? Markdown link check (live docs) in 6s (ID 85684058573)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
```

```
  ? Check internal markdown links
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job
? Syntax sweep + import smoke (no GPU stack) in 21s (ID 85684058574)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? Install minimal deps
  ? Run import smoke tests
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job

ANNOTATIONS
- The macos-latest label will migrate to macOS 26 beginning June 15, 2026. For more
information see https://github.com/actions/runner-images/issues/14167
Golden fixtures cross-OS (macos-latest): .github#1

Refreshing run status every 3 seconds. Press Ctrl+C to quit.

* docs/runlog-cloud-run-exec-status-fastfail CI Khelsutra/badminton-highlight-
indexer#517 · 28885365017
Triggered via pull_request about 2 minutes ago

JOBS
? Leak scan (PII / attribution guard) in 6s (ID 85684058484)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? Leak scan
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job
? Ruff lint in 7s (ID 85684058515)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? Install ruff
  ? Ruff check
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job
? Golden fixtures cross-OS (macos-latest) in 14s (ID 85684058517)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
  ? Golden-fixtures characterization (cross-OS determinism guard)
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job
? Mypy (lenient baseline) in 31s (ID 85684058526)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? Install mypy + typed core deps
  ? Mypy
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job
? Golden fixtures cross-OS (ubuntu-latest) in 25s (ID 85684058528)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
```

```
159       ? Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
160       ? Golden-fixtures characterization (cross-OS determinism guard)
161       ? Post Run actions/setup-python@v6
162       ? Post Run actions/checkout@v6
163       ? Complete job
164     ? Dependency license scan (no copyleft in runtime deps) in 35s (ID 85684058531)
165       ? Set up job
166       ? Run actions/checkout@v6
167       ? Run actions/setup-python@v6
168       ? Install runtime deps + pip-licenses
169       ? Report dependency licenses
170       ? Fail on any copyleft (GPL/AGPL/LGPL/SSPL) in the runtime closure
171       ? Post Run actions/setup-python@v6
172       ? Post Run actions/checkout@v6
173       ? Complete job
174     ? Built wheel smoke (install + launch) in 1m10s (ID 85684058542)
175       ? Set up job
176       ? Run actions/checkout@v6
177       ? Run actions/setup-python@v6
178       ? System libs for opencv-python
179       ? Build the wheel
180       ? Install the built wheel + CPU torch (out-of-band index)
181       ? Launch the installed console script from a non-repo dir
182       ? Post Run actions/setup-python@v6
183       ? Post Run actions/checkout@v6
184       ? Complete job
185     ? Golden fixtures cross-OS (windows-latest) in 55s (ID 85684058548)
186       ? Set up job
187       ? Run actions/checkout@v6
188       ? Run actions/setup-python@v6
189       ? Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
190       ? Golden-fixtures characterization (cross-OS determinism guard)
191       ? Post Run actions/setup-python@v6
192       ? Post Run actions/checkout@v6
193       ? Complete job
194     * Full test suite (offline, mocked) (ID 85684058558)
195       ? Set up job
196       ? Run actions/checkout@v6
197       ? Run actions/setup-python@v6
198       ? System libs for opencv-python
199       ? Install CPU-only torch (no CUDA wheels on a CPU runner)
200       ? Install trackers (git-only dep ? PyPI wheel is broken)
201       ? Install package (editable) + dev tooling
202       * Run full suite (with coverage floor)
203       * Post Run actions/setup-python@v6
204       * Post Run actions/checkout@v6
205     ? Markdown link check (live docs) in 6s (ID 85684058573)
206       ? Set up job
207       ? Run actions/checkout@v6
208       ? Run actions/setup-python@v6
209       ? Check internal markdown links
210       ? Post Run actions/setup-python@v6
211       ? Post Run actions/checkout@v6
212       ? Complete job
213     ? Syntax sweep + import smoke (no GPU stack) in 21s (ID 85684058574)
214       ? Set up job
215       ? Run actions/checkout@v6
216       ? Run actions/setup-python@v6
217       ? Install minimal deps
218       ? Run import smoke tests
219       ? Post Run actions/setup-python@v6
220       ? Post Run actions/checkout@v6
221       ? Complete job
222
223     ANNOTATIONS
```

```
    - The macos-latest label will migrate to macOS 26 beginning June 15, 2026. For more
information see https://github.com/actions/runner-images/issues/14167
    Golden fixtures cross-OS (macos-latest): .github#1

    Refreshing run status every 3 seconds. Press Ctrl+C to quit.

    * docs/runlog-cloud-run-exec-status-fastfail CI Khelsutra/badminton-highlight-
indexer#517 · 28885365017
    Triggered via pull_request about 2 minutes ago

    JOBS
    ? Leak scan (PII / attribution guard) in 6s (ID 85684058484)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
      ? Leak scan
      ? Post Run actions/setup-python@v6
      ? Post Run actions/checkout@v6
      ? Complete job
    ? Ruff lint in 7s (ID 85684058515)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
      ? Install ruff
      ? Ruff check
      ? Post Run actions/setup-python@v6
      ? Post Run actions/checkout@v6
      ? Complete job
    ? Golden fixtures cross-OS (macos-latest) in 14s (ID 85684058517)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
      ? Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
      ? Golden-fixtures characterization (cross-OS determinism guard)
      ? Post Run actions/setup-python@v6
      ? Post Run actions/checkout@v6
      ? Complete job
    ? Mypy (lenient baseline) in 31s (ID 85684058526)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
      ? Install mypy + typed core deps
      ? Mypy
      ? Post Run actions/setup-python@v6
      ? Post Run actions/checkout@v6
      ? Complete job
    ? Golden fixtures cross-OS (ubuntu-latest) in 25s (ID 85684058528)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
      ? Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
      ? Golden-fixtures characterization (cross-OS determinism guard)
      ? Post Run actions/setup-python@v6
      ? Post Run actions/checkout@v6
      ? Complete job
    ? Dependency license scan (no copyleft in runtime deps) in 35s (ID 85684058531)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
      ? Install runtime deps + pip-licenses
      ? Report dependency licenses
      ? Fail on any copyleft (GPL/AGPL/LGPL/SSPL) in the runtime closure
      ? Post Run actions/setup-python@v6
      ? Post Run actions/checkout@v6
      ? Complete job
```

```
    ? Built wheel smoke (install + launch) in 1m10s (ID 85684058542)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
      ? System libs for opencv-python
      ? Build the wheel
      ? Install the built wheel + CPU torch (out-of-band index)
      ? Launch the installed console script from a non-repo dir
      ? Post Run actions/setup-python@v6
      ? Post Run actions/checkout@v6
      ? Complete job
    ? Golden fixtures cross-OS (windows-latest) in 55s (ID 85684058548)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
      ? Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
      ? Golden-fixtures characterization (cross-OS determinism guard)
      ? Post Run actions/setup-python@v6
      ? Post Run actions/checkout@v6
      ? Complete job
    * Full test suite (offline, mocked) (ID 85684058558)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
      ? System libs for opencv-python
      ? Install CPU-only torch (no CUDA wheels on a CPU runner)
      ? Install trackers (git-only dep ? PyPI wheel is broken)
      ? Install package (editable) + dev tooling
      * Run full suite (with coverage floor)
      * Post Run actions/setup-python@v6
      * Post Run actions/checkout@v6
    ? Markdown link check (live docs) in 6s (ID 85684058573)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
      ? Check internal markdown links
      ? Post Run actions/setup-python@v6
      ? Post Run actions/checkout@v6
      ? Complete job
    ? Syntax sweep + import smoke (no GPU stack) in 21s (ID 85684058574)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
      ? Install minimal deps
      ? Run import smoke tests
      ? Post Run actions/setup-python@v6
      ? Post Run actions/checkout@v6
      ? Complete job

    ANNOTATIONS
    - The macos-latest label will migrate to macOS 26 beginning June 15, 2026. For more
information see https://github.com/actions/runner-images/issues/14167
    Golden fixtures cross-OS (macos-latest): .github#1

    Refreshing run status every 3 seconds. Press Ctrl+C to quit.

    * docs/runlog-cloud-run-exec-status-fastfail CI Khelsutra/badminton-highlight-
indexer#517 · 28885365017
    Triggered via pull_request about 2 minutes ago

    JOBS
    ? Leak scan (PII / attribution guard) in 6s (ID 85684058484)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
```

```
      ? Leak scan
      ? Post Run actions/setup-python@v6
      ? Post Run actions/checkout@v6
      ? Complete job
    ? Ruff lint in 7s (ID 85684058515)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
      ? Install ruff
      ? Ruff check
      ? Post Run actions/setup-python@v6
      ? Post Run actions/checkout@v6
      ? Complete job
    ? Golden fixtures cross-OS (macos-latest) in 14s (ID 85684058517)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
      ? Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
      ? Golden-fixtures characterization (cross-OS determinism guard)
      ? Post Run actions/setup-python@v6
      ? Post Run actions/checkout@v6
      ? Complete job
    ? Mypy (lenient baseline) in 31s (ID 85684058526)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
      ? Install mypy + typed core deps
      ? Mypy
      ? Post Run actions/setup-python@v6
      ? Post Run actions/checkout@v6
      ? Complete job
    ? Golden fixtures cross-OS (ubuntu-latest) in 25s (ID 85684058528)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
      ? Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
      ? Golden-fixtures characterization (cross-OS determinism guard)
      ? Post Run actions/setup-python@v6
      ? Post Run actions/checkout@v6
      ? Complete job
    ? Dependency license scan (no copyleft in runtime deps) in 35s (ID 85684058531)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
      ? Install runtime deps + pip-licenses
      ? Report dependency licenses
      ? Fail on any copyleft (GPL/AGPL/LGPL/SSPL) in the runtime closure
      ? Post Run actions/setup-python@v6
      ? Post Run actions/checkout@v6
      ? Complete job
    ? Built wheel smoke (install + launch) in 1m10s (ID 85684058542)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
      ? System libs for opencv-python
      ? Build the wheel
      ? Install the built wheel + CPU torch (out-of-band index)
      ? Launch the installed console script from a non-repo dir
      ? Post Run actions/setup-python@v6
      ? Post Run actions/checkout@v6
      ? Complete job
    ? Golden fixtures cross-OS (windows-latest) in 55s (ID 85684058548)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
```

```
      ? Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
      ? Golden-fixtures characterization (cross-OS determinism guard)
      ? Post Run actions/setup-python@v6
      ? Post Run actions/checkout@v6
      ? Complete job
    * Full test suite (offline, mocked) (ID 85684058558)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
      ? System libs for opencv-python
      ? Install CPU-only torch (no CUDA wheels on a CPU runner)
      ? Install trackers (git-only dep ? PyPI wheel is broken)
      ? Install package (editable) + dev tooling
      * Run full suite (with coverage floor)
      * Post Run actions/setup-python@v6
      * Post Run actions/checkout@v6
    ? Markdown link check (live docs) in 6s (ID 85684058573)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
      ? Check internal markdown links
      ? Post Run actions/setup-python@v6
      ? Post Run actions/checkout@v6
      ? Complete job
    ? Syntax sweep + import smoke (no GPU stack) in 21s (ID 85684058574)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
      ? Install minimal deps
      ? Run import smoke tests
      ? Post Run actions/setup-python@v6
      ? Post Run actions/checkout@v6
      ? Complete job

    ANNOTATIONS
    - The macos-latest label will migrate to macOS 26 beginning June 15, 2026. For more
information see https://github.com/actions/runner-images/issues/14167
    Golden fixtures cross-OS (macos-latest): .github#1

    Refreshing run status every 3 seconds. Press Ctrl+C to quit.

    * docs/runlog-cloud-run-exec-status-fastfail CI Khelsutra/badminton-highlight-
indexer#517 · 28885365017
    Triggered via pull_request about 2 minutes ago

    JOBS
    ? Leak scan (PII / attribution guard) in 6s (ID 85684058484)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
      ? Leak scan
      ? Post Run actions/setup-python@v6
      ? Post Run actions/checkout@v6
      ? Complete job
    ? Ruff lint in 7s (ID 85684058515)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
      ? Install ruff
      ? Ruff check
      ? Post Run actions/setup-python@v6
      ? Post Run actions/checkout@v6
      ? Complete job
    ? Golden fixtures cross-OS (macos-latest) in 14s (ID 85684058517)
      ? Set up job
```

```
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
  ? Golden-fixtures characterization (cross-OS determinism guard)
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job
? Mypy (lenient baseline) in 31s (ID 85684058526)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? Install mypy + typed core deps
  ? Mypy
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job
? Golden fixtures cross-OS (ubuntu-latest) in 25s (ID 85684058528)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
  ? Golden-fixtures characterization (cross-OS determinism guard)
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job
? Dependency license scan (no copyleft in runtime deps) in 35s (ID 85684058531)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? Install runtime deps + pip-licenses
  ? Report dependency licenses
  ? Fail on any copyleft (GPL/AGPL/LGPL/SSPL) in the runtime closure
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job
? Built wheel smoke (install + launch) in 1m10s (ID 85684058542)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? System libs for opencv-python
  ? Build the wheel
  ? Install the built wheel + CPU torch (out-of-band index)
  ? Launch the installed console script from a non-repo dir
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job
? Golden fixtures cross-OS (windows-latest) in 55s (ID 85684058548)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
  ? Golden-fixtures characterization (cross-OS determinism guard)
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job
* Full test suite (offline, mocked) (ID 85684058558)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? System libs for opencv-python
  ? Install CPU-only torch (no CUDA wheels on a CPU runner)
  ? Install trackers (git-only dep ? PyPI wheel is broken)
  ? Install package (editable) + dev tooling
  * Run full suite (with coverage floor)
  * Post Run actions/setup-python@v6
```

```
      * Post Run actions/checkout@v6
    ? Markdown link check (live docs) in 6s (ID 85684058573)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
      ? Check internal markdown links
      ? Post Run actions/setup-python@v6
      ? Post Run actions/checkout@v6
      ? Complete job
    ? Syntax sweep + import smoke (no GPU stack) in 21s (ID 85684058574)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
      ? Install minimal deps
      ? Run import smoke tests
      ? Post Run actions/setup-python@v6
      ? Post Run actions/checkout@v6
      ? Complete job

    ANNOTATIONS
    - The macos-latest label will migrate to macOS 26 beginning June 15, 2026. For more
information see https://github.com/actions/runner-images/issues/14167
    Golden fixtures cross-OS (macos-latest): .github#1

    Refreshing run status every 3 seconds. Press Ctrl+C to quit.

    * docs/runlog-cloud-run-exec-status-fastfail CI Khelsutra/badminton-highlight-
indexer#517 · 28885365017
    Triggered via pull_request about 2 minutes ago

    JOBS
    ? Leak scan (PII / attribution guard) in 6s (ID 85684058484)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
      ? Leak scan
      ? Post Run actions/setup-python@v6
      ? Post Run actions/checkout@v6
      ? Complete job
    ? Ruff lint in 7s (ID 85684058515)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
      ? Install ruff
      ? Ruff check
      ? Post Run actions/setup-python@v6
      ? Post Run actions/checkout@v6
      ? Complete job
    ? Golden fixtures cross-OS (macos-latest) in 14s (ID 85684058517)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
      ? Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
      ? Golden-fixtures characterization (cross-OS determinism guard)
      ? Post Run actions/setup-python@v6
      ? Post Run actions/checkout@v6
      ? Complete job
    ? Mypy (lenient baseline) in 31s (ID 85684058526)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
      ? Install mypy + typed core deps
      ? Mypy
      ? Post Run actions/setup-python@v6
      ? Post Run actions/checkout@v6
```

```
      ? Complete job
    ? Golden fixtures cross-OS (ubuntu-latest) in 25s (ID 85684058528)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
      ? Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
      ? Golden-fixtures characterization (cross-OS determinism guard)
      ? Post Run actions/setup-python@v6
      ? Post Run actions/checkout@v6
      ? Complete job
    ? Dependency license scan (no copyleft in runtime deps) in 35s (ID 85684058531)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
      ? Install runtime deps + pip-licenses
      ? Report dependency licenses
      ? Fail on any copyleft (GPL/AGPL/LGPL/SSPL) in the runtime closure
      ? Post Run actions/setup-python@v6
      ? Post Run actions/checkout@v6
      ? Complete job
    ? Built wheel smoke (install + launch) in 1m10s (ID 85684058542)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
      ? System libs for opencv-python
      ? Build the wheel
      ? Install the built wheel + CPU torch (out-of-band index)
      ? Launch the installed console script from a non-repo dir
      ? Post Run actions/setup-python@v6
      ? Post Run actions/checkout@v6
      ? Complete job
    ? Golden fixtures cross-OS (windows-latest) in 55s (ID 85684058548)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
      ? Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
      ? Golden-fixtures characterization (cross-OS determinism guard)
      ? Post Run actions/setup-python@v6
      ? Post Run actions/checkout@v6
      ? Complete job
    * Full test suite (offline, mocked) (ID 85684058558)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
      ? System libs for opencv-python
      ? Install CPU-only torch (no CUDA wheels on a CPU runner)
      ? Install trackers (git-only dep ? PyPI wheel is broken)
      ? Install package (editable) + dev tooling
      * Run full suite (with coverage floor)
      * Post Run actions/setup-python@v6
      * Post Run actions/checkout@v6
    ? Markdown link check (live docs) in 6s (ID 85684058573)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
      ? Check internal markdown links
      ? Post Run actions/setup-python@v6
      ? Post Run actions/checkout@v6
      ? Complete job
    ? Syntax sweep + import smoke (no GPU stack) in 21s (ID 85684058574)
      ? Set up job
      ? Run actions/checkout@v6
      ? Run actions/setup-python@v6
      ? Install minimal deps
      ? Run import smoke tests
```

```
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job

ANNOTATIONS
- The macos-latest label will migrate to macOS 26 beginning June 15, 2026. For more information see https://github.com/actions/runner-images/issues/14167
Golden fixtures cross-OS (macos-latest): .github#1

Refreshing run status every 3 seconds. Press Ctrl+C to quit.

* docs/runlog-cloud-run-exec-status-fastfail CI Khelsutra/badminton-highlight-indexer#517 · 28885365017
Triggered via pull_request about 2 minutes ago

JOBS
? Leak scan (PII / attribution guard) in 6s (ID 85684058484)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? Leak scan
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job
? Ruff lint in 7s (ID 85684058515)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? Install ruff
  ? Ruff check
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job
? Golden fixtures cross-OS (macos-latest) in 14s (ID 85684058517)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
  ? Golden-fixtures characterization (cross-OS determinism guard)
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job
? Mypy (lenient baseline) in 31s (ID 85684058526)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? Install mypy + typed core deps
  ? Mypy
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job
? Golden fixtures cross-OS (ubuntu-latest) in 25s (ID 85684058528)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
  ? Golden-fixtures characterization (cross-OS determinism guard)
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job
? Dependency license scan (no copyleft in runtime deps) in 35s (ID 85684058531)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? Install runtime deps + pip-licenses
```

```
  ? Report dependency licenses
  ? Fail on any copyleft (GPL/AGPL/LGPL/SSPL) in the runtime closure
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job
? Built wheel smoke (install + launch) in 1m10s (ID 85684058542)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? System libs for opencv-python
  ? Build the wheel
  ? Install the built wheel + CPU torch (out-of-band index)
  ? Launch the installed console script from a non-repo dir
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job
? Golden fixtures cross-OS (windows-latest) in 55s (ID 85684058548)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
  ? Golden-fixtures characterization (cross-OS determinism guard)
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job
* Full test suite (offline, mocked) (ID 85684058558)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? System libs for opencv-python
  ? Install CPU-only torch (no CUDA wheels on a CPU runner)
  ? Install trackers (git-only dep ? PyPI wheel is broken)
  ? Install package (editable) + dev tooling
  * Run full suite (with coverage floor)
  * Post Run actions/setup-python@v6
  * Post Run actions/checkout@v6
? Markdown link check (live docs) in 6s (ID 85684058573)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? Check internal markdown links
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job
? Syntax sweep + import smoke (no GPU stack) in 21s (ID 85684058574)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? Install minimal deps
  ? Run import smoke tests
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job

ANNOTATIONS
- The macos-latest label will migrate to macOS 26 beginning June 15, 2026. For more
information see https://github.com/actions/runner-images/issues/14167
Golden fixtures cross-OS (macos-latest): .github#1

Refreshing run status every 3 seconds. Press Ctrl+C to quit.

* docs/runlog-cloud-run-exec-status-fastfail CI Khelsutra/badminton-highlight-
indexer#517 · 28885365017
Triggered via pull_request about 2 minutes ago

```

```
JOBS
? Leak scan (PII / attribution guard) in 6s (ID 85684058484)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? Leak scan
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job
? Ruff lint in 7s (ID 85684058515)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? Install ruff
  ? Ruff check
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job
? Golden fixtures cross-OS (macos-latest) in 14s (ID 85684058517)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
  ? Golden-fixtures characterization (cross-OS determinism guard)
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job
? Mypy (lenient baseline) in 31s (ID 85684058526)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? Install mypy + typed core deps
  ? Mypy
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job
? Golden fixtures cross-OS (ubuntu-latest) in 25s (ID 85684058528)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
  ? Golden-fixtures characterization (cross-OS determinism guard)
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job
? Dependency license scan (no copyleft in runtime deps) in 35s (ID 85684058531)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? Install runtime deps + pip-licenses
  ? Report dependency licenses
  ? Fail on any copyleft (GPL/AGPL/LGPL/SSPL) in the runtime closure
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job
? Built wheel smoke (install + launch) in 1m10s (ID 85684058542)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? System libs for opencv-python
  ? Build the wheel
  ? Install the built wheel + CPU torch (out-of-band index)
  ? Launch the installed console script from a non-repo dir
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
```

```
  ? Complete job
? Golden fixtures cross-OS (windows-latest) in 55s (ID 85684058548)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
  ? Golden-fixtures characterization (cross-OS determinism guard)
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job
* Full test suite (offline, mocked) (ID 85684058558)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? System libs for opencv-python
  ? Install CPU-only torch (no CUDA wheels on a CPU runner)
  ? Install trackers (git-only dep ? PyPI wheel is broken)
  ? Install package (editable) + dev tooling
  * Run full suite (with coverage floor)
  * Post Run actions/setup-python@v6
  * Post Run actions/checkout@v6
? Markdown link check (live docs) in 6s (ID 85684058573)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? Check internal markdown links
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job
? Syntax sweep + import smoke (no GPU stack) in 21s (ID 85684058574)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? Install minimal deps
  ? Run import smoke tests
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job

ANNOTATIONS
- The macos-latest label will migrate to macOS 26 beginning June 15, 2026. For more
information see https://github.com/actions/runner-images/issues/14167
Golden fixtures cross-OS (macos-latest): .github#1

Refreshing run status every 3 seconds. Press Ctrl+C to quit.

* docs/runlog-cloud-run-exec-status-fastfail CI Khelsutra/badminton-highlight-
indexer#517 · 28885365017
Triggered via pull_request about 2 minutes ago

JOBS
? Leak scan (PII / attribution guard) in 6s (ID 85684058484)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? Leak scan
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job
? Ruff lint in 7s (ID 85684058515)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? Install ruff
  ? Ruff check
```

```
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job
? Golden fixtures cross-OS (macos-latest) in 14s (ID 85684058517)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
  ? Golden-fixtures characterization (cross-OS determinism guard)
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job
? Mypy (lenient baseline) in 31s (ID 85684058526)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? Install mypy + typed core deps
  ? Mypy
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job
? Golden fixtures cross-OS (ubuntu-latest) in 25s (ID 85684058528)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
  ? Golden-fixtures characterization (cross-OS determinism guard)
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job
? Dependency license scan (no copyleft in runtime deps) in 35s (ID 85684058531)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? Install runtime deps + pip-licenses
  ? Report dependency licenses
  ? Fail on any copyleft (GPL/AGPL/LGPL/SSPL) in the runtime closure
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job
? Built wheel smoke (install + launch) in 1m10s (ID 85684058542)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? System libs for opencv-python
  ? Build the wheel
  ? Install the built wheel + CPU torch (out-of-band index)
  ? Launch the installed console script from a non-repo dir
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job
? Golden fixtures cross-OS (windows-latest) in 55s (ID 85684058548)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
  ? Golden-fixtures characterization (cross-OS determinism guard)
  ? Post Run actions/setup-python@v6
  ? Post Run actions/checkout@v6
  ? Complete job
* Full test suite (offline, mocked) (ID 85684058558)
  ? Set up job
  ? Run actions/checkout@v6
  ? Run actions/setup-python@v6
  ? System libs for opencv-python
```

```
990       ? Install CPU-only torch (no CUDA wheels on a CPU runner)
991       ? Install trackers (git-only dep ? PyPI wheel is broken)
992       ? Install package (editable) + dev tooling
993       * Run full suite (with coverage floor)
994       * Post Run actions/setup-python@v6
995       * Post Run actions/checkout@v6
996     ? Markdown link check (live docs) in 6s (ID 85684058573)
997       ? Set up job
998       ? Run actions/checkout@v6
999       ? Run actions/setup-python@v6
1000      ? Check internal markdown links
1001      ? Post Run actions/setup-python@v6
1002      ? Post Run actions/checkout@v6
1003      ? Complete job
1004    ? Syntax sweep + import smoke (no GPU stack) in 21s (ID 85684058574)
1005      ? Set up job
1006      ? Run actions/checkout@v6
1007      ? Run actions/setup-python@v6
1008      ? Install minimal deps
1009      ? Run import smoke tests
1010      ? Post Run actions/setup-python@v6
1011      ? Post Run actions/checkout@v6
1012      ? Complete job
1013
1014    ANNOTATIONS
1015    - The macos-latest label will migrate to macOS 26 beginning June 15, 2026. For more
information see https://github.com/actions/runner-images/issues/14167
1016    Golden fixtures cross-OS (macos-latest): .github#1
1017
1018
```

162. assistant (2026-07-07T17:24:58.017Z)

PR **#517 is merged** (squash `115b83d` ? master) and all cleanup is done ? worktree removed, local/remote branches deleted, stray runlog cleared, sibling WIP untouched. 10 of 11 CI jobs are green (Ruff, Mypy, all 3 golden-fixtures OSes, wheel smoke, license scan, syntax sweep, leak scan, markdown links); only the **Full test suite** is still executing its test step. I'm watching that run to completion (with `--exit-status`) so I can confirm the final green ? the watcher will report back when it finishes.

163. user (2026-07-07T17:24:59.396Z)

Updated task #5 description

164. assistant (2026-07-07T17:25:06.063Z)

I'll wait for the run watcher to report the final result. Everything substantive is complete ? the smoke passed all four criteria, the runlog is merged to master, and the shared checkout is clean with the sibling's work untouched. I'll confirm the last check the moment it lands.

165. user (2026-07-07T17:25:20.989Z)

```
<task-notification>
<task-id>bo4a10bws</task-id>
<tool-use-id>toolu_01Mce5NoL83NMg2VgjoqY15h</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-
highlight-indexer\0c4fc269-564f-4fb2-9c1e-9389160b6be0\tasks\bo4a10bws.output</output-file>
<status>completed</status>
<summary>Background command "Watch CI run 28885365017 to completion (background)" completed
(exit code 0)</summary>
</task-notification>
```

166. assistant (2026-07-07T17:25:25.434Z)

The watcher exited 0 (with `--exit-status`, that means the run succeeded). Let me confirm the definitive conclusion.

167. user (2026-07-07T17:25:33.517Z)

```
=== run conclusion ===
{"conclusion":"success","status":"completed"}
=== the full test suite job specifically ===
Full test suite (offline, mocked): completed success
=== master head now ===
115b83d docs(runlogs): record live smoke of the crash-before-marker fast-fail (#515) (#517)
```

168. assistant (2026-07-07T17:25:42.970Z)

All green confirmed. Let me close out the task.

169. user (2026-07-07T17:25:43.021Z)

Updated task #5 status

170. assistant (2026-07-07T17:25:58.235Z)

Done ? merged, all green ?

PR [#517](https://github.com/Khelsutra/badminton-highlight-indexer/pull/517) is squash-merged to `master` as **`115b83d`**, and CI run 28885365017 has since completed **`success` ? all 11 checks green**, including the **Full test suite (offline, mocked)** and Ruff / Mypy / golden-fixtures (3 OS) / wheel smoke / license scan.

One honesty note on "on all green": because the repo's branch protection doesn't *require* the status checks, `gh --auto` merged the moment the PR was mergeable ? at which point 10/11 were green and the full suite was still running. I then watched that last check to completion and it passed, so the merged commit is verified green (and it couldn't have been otherwise ? the diff is 2 markdown files).

How it was landed safely (given the concurrent-session hazard): I did **not** touch the shared checkout's branch. I created an isolated `git worktree` off `origin/master`, committed only the two docs files there (verified `origin/master..HEAD` contained solely my commit, docs-only), pushed, PR'd, and merged from the worktree. The shared checkout stayed on the sibling's `feat/validation-result-cache` throughout ? their WIP is fully intact and never entangled with mine.

Cleanup complete: worktree removed, local + remote `docs/?` branches deleted, the redundant untracked runlog cleared from the shared checkout, GCS test objects deleted, no executions running, worker image digest unchanged (`sha256:8045f54e?`).

Net result of the whole task

The `# pragma: no cover` `get_execution` fast-fail path is now **verified against live Cloud Run** ? no bug found:
- **Classifier vs real API:** `completion_time` ? `datetime` when terminal / `None` while running (proto-plus fix holds); Succeeded + Failed branches both correct on real executions.
- **Fast-fail:** crash-before-marker ? **rc 5** in **47 s** (not rc 4, not ~65 min), with a diagnostic naming the execution, state, and `gcloud ? describe` command.
- **No false-positive:** healthy dispatch ? **rc 0** (19 rallies); probe returned `None` on all 9 running-polls.

The record is at [RUNLOG_cloud-serving_exec-status-fastfail_2026-07-07.md](docs/COMPUTE_DECOUPLED_SERVING/runlogs/RUNLOG_cloud-serving_exec-status-fastfail_2026-07-07.md) (now on master) and the [[cloud-run-exec-status-fastfail]] memory is updated.